The Payments page, as built

Every screen a clerk reaches under `/payments`, audited against the running code and then against the running app: what is there, what is reachable, what each field is really served by, and what the product itself says it cannot do.

READ AT	COMMIT	PRODUCT NOW	SCREENS DRAWN	DRAWINGS	TABS	NEW SCREENS	NEED NO BACKEND	JOURNEYS WALKED
2.322.0	e75771af	*Could not check*	35	38	7	17	2	6 of 9

RULE VIOLATIONS	FIELDS SERVED	CROSS-MODULE FACTS	SEAMS NOT HEALTHY
18	48 of 62	7	2 of 6

Read at one moment · re-verify before acting

Every as-built statement here was true at **2.322.0**, commit `e75771af`, on **2026-09-21**, and must be re-verified before it is trusted. The design system was read from `07-frontend/src/styles/tokens.css` and `07-frontend/src/features/payments/payments.css` at that same commit.

Whether the product has moved on: could not check. `origin/HEAD` does not resolve in this checkout and more than twenty remote branches exist with no obvious default, so naming one as "the product" would be a guess. This describes `claude/upbeat-cannon-xln69g` at that commit and nothing else.

Four findings in the source were **already stale when read** — the currency code on two bodies, `GET /write-offs`, and two of this module's own file exceptions. Each is **struck in place, not deleted**, with the answer printed beneath it, because a deleted row invites the same finding next quarter.

What this warning does not cover: descriptions go stale, rulings stay decided, and open questions stay open until somebody answers them — not until the code changes. `OQ-1340` and `OQ-153` are both still open here regardless of what any file now says.

How much of this was seen, and how much was read

The app was driven. The full stack came up under the repository's own `08-qa/01-e2e/stack/boot_stack.sh` — PostgreSQL 16, 389 migrations, the demo fixture, Django, and the **built** SPA served same-origin on one port. Signed in through the real sign-in screen as the fixture's published owner (**Dana Whitfield, System Admin**, tenant `brazosbend`) and driven with Playwright. Seeded state as walked: **30 accounts owing, $8,870.58 outstanding, zero payments recorded.**

6 of 9 journeys were walked. The three that were not — recording a payment, reversing one, correcting a reference — all need a payment row, and **the payments fixture refused**

to seed. It refused by name rather than crashing: *"no multi-property cheque was recorded: one of the landlord's three properties owes nothing, so a cheque covering all three would be a cheque for less than it claims and the tender's own sum check would refuse it."*

So the single most severe finding here — **PAY-A**, an internal key printed to the clerk on the primary action's happy path — **is proved by reading every link of its chain and was never seen on screen.** A described click is not a walk, and in a finished document the two are indistinguishable unless someone says which is which. §03 says which is which, screen by screen.

Description
A statement about what the code does today, including an honest statement that something is missing. It commits nobody to anything.

Commitment
A change specification: a named piece of work, what it waits on, the screens it changes, and what a person could do and see once it is done.

If a thing is not in a commitment card, nobody has undertaken to build it. Four state chips, and only four: EXISTS BEHIND NEW READ NEW BUILD BLOCKED. A screen no specification covers carries NO COMMITMENT — that absence is the finding, not an oversight. The IDs `PAY-A`…`PAY-P` are a **new series minted for this document** and deliberately not allocated into the repository's live `US-`/`BR-`/`DEF-`/`OQ-` series: an analysis may propose and never allocate, and borrowing a live ID makes work look scheduled when it is not.

00

How to read this document

Every statement here is one of five things, and the marker says which. A lane that cannot tell them apart will either build what nobody committed to or skip what was. **The machine-readable copy is the JSON block in this section** — `#blueprint-manifest` — which carries every task, screen, dropdown source, ruling and open question as data, so nothing has to be parsed out of prose.

Marker	Means	What to do with it
TASK	**Committed work.** Carries a plan id `Pn.m`	**Build it.** The plan gives it a reference to open and an outcome that says when it is done
TODAY	**What is true in the code now**, with a file and line	**Do not re-derive it, and do not change it without a TASK.** It was read, not assumed
GAP	**A defect or a missing piece**	**Information only.** Actionable *only* if it also carries a TASK — several gaps here are deliberately not scheduled
RULED	**A Product Owner decision**, with	**Binding. Never re-ask** — CLAUDE.md hard-won law 13: a re-ask manufactures inconsistency and then blames the human for it

	its date	
OPEN	**Nobody has answered**, with the question's id	**Never resolve it by choosing a value.** Stub with the id, per the prime directive
SOURCE	**Where a control's values come from**	**No drawing in this document authorises a literal array.** The full set is in §01

GAP

The GAP / TASK split is the one that matters

This audit found **eighteen rule breaches**, several dead controls, two duplicated implementations and one orphaned route. **Some are in the plan and some are deliberately not.** Without the split a lane reads a finding as an instruction — or skips one that was scheduled — and either way the plan and the code diverge silently.

So: a GAP with no TASK beside it is reported, not scheduled. If you think it should be, that is a conversation with the Product Owner, not a decision to make while building.

TODAY

The manifest is derived from this document, not written beside it

A hand-kept index is a second source of truth that drifts. This one is **generated from the marked HTML**, so a task with no id, a screen with no class or a dropdown with no source is **absent from the manifest and its absence is visible**.

It has already earned its place twice. Building it surfaced six screens still carrying the pre-standardisation class words (*Exists*, *Not built*, *No address today*) and a register row still calling 7·3c an *Update* after it became a *Remove*. Regenerating it after the notice screen landed caught the register's stale ids for 7·3b. **None of the three was visible by reading.**

01

The standards every screen is drawn to

Each is an existing ratified rule with a home in the repository, a failure evidenced in the product, or a convention users already expect. **Nothing here is new design philosophy.** Precedence, and it decides every judgement in §05: **B14 wins**, then r10's `UI-RULES.md` — B14's source, not its authority — then the repo's own build file for how those are obeyed in this tree.

What the rule set names	What the shipped app declares	Where the drawings take it from
`.lt` · `.page-head` · `.date-input` · `.btn-danger` · `.btn-sm` · `.br-steps`	**None of them.** They live in `05-prototype/r10-2026-08-16/styles.css`, whose runtime CLAUDE.md forbids porting	Layout only — never presented as a class the product has
Archivo 400/600/800, `--font-heading-weight: 800`	Declared in `tokens.css` and **never loaded**. Every screen has rendered in system-ui since the shell was built	**Archivo**, because the missing webfont is a filed gap, not a decision
Page-head title **40px** — measured on a running app at an earlier commit	**32px.** Measured with `getComputedStyle` on this build, and B14 rule 40 ratified it on 2026-09-12: *"use 32 pixels — for every ordinary screen"*	**32px.** B14 is the authority and the running app agrees with it
`--radius-sm/md/lg`	**0px, all three.** Square at every size, on purpose	0px. **Radius is the tell** — rounded cards would prove no extraction happened
Payments screens' own classes	`features/payments/payments.css` — 589 lines, 66 classes, imported by seven components. **Not in `route-b-migrated.txt`**	Shapes only. Lawful legacy: `G31` requires exactly what these files do

Figure 1 · The three places a drawing can go wrong, and where each literal was actually taken from. Two of these rows are divergences resolved *against* the guidance this audit follows — the title size and the webfont — and both are resolved in favour of what the code and the running app say.

A · Which reference owns these screens

> TODAY **Determined, not assumed**
>
> CLAUDE.md's `DEF-GOV-5248` demotes r10 to visual style alone *for any module that has a blueprint PDF*. **Payments has none.** Checked with `git ls-tree -r --name-only <ref> | grep -i 'pdf$'` across every remote ref, not just this branch: seven blueprints exist — Assets, Accounts, Billing, Integration, Premises & Units, Customer Portal, Plans & Rates — and `git log --all --diff-filter=A` shows no Payments blueprint has ever existed in the repository's history.
>
> So for these nine screens the demotion does not fire, and `05-prototype/r10-2026-08-16/SMARTBillHub.dc.html` remains the reference for **layout, copy, flows and states**, with B14 overriding wherever the two differ.

B · The design system, extracted rather than invented

Family	Pinned literal	Source
Colour	bg `#f3f2f2` · surface `#eae9e9` · ink `#201e1d` · accent `#ec3013` · neutral ramp 100–900 · success / warning / danger / info	`tokens.css`
Type	page-head **32px** · h4 20 · h5 16 · body 15 · md 14 · base 13 · sm 12 · xs 11 · KPI metric 26 · table body **13.5px**, header 12px at `.08em`	`tokens.css` · B14 rule 40
Font	Archivo, heading weight **800**	`tokens.css`
Space	4 · 8 · 12 · 16 · 20 · 24 · 32, and nothing off the scale	`tokens.css`
Radius	**0px at every size**	`tokens.css`
Shell	240px inverted-ink left rail, **13 items**, 56px topbar, accent active item	`AppShell.tsx` `SHELL_NAV`, counted in the running app

The thirteen rail items, read from `SHELL_NAV` and confirmed by counting anchors in the running app — never retyped: Dashboard · Plans & Rates · Premises · Assets · Accounts · Reads · Billing · Payments · Work · Finance · Runbooks · Reports · Settings.

> `TODAY` **Light drawings inside a theme-aware document**
>
> This page follows your theme. **The wireframes do not.** They are pictures of a light product, and a picture of a light product that restyles itself in a dark client is no longer a picture of the product — so every colour inside a drawing frame is pinned to the shipped value whatever theme you read in. Holding a drawing beside the running screen stays a fair comparison either way.

C · The rules this audit judges against

B14 carries **34** binding rules, numbered **1–21 and 31–43**. That count was taken from B14's own status table rather than from any pointer to it, as CLAUDE.md instructs; rules 22–30 were never minted and 30 was declined.

Rule	What it says	Enforced?
12	Table toolbar: `[Search…] [filter chips ×] [+ Filter ▾] … [▦ Columns]`	Ratified only
13	Sorting: headers show ⇅; click cycles asc → desc → clear	Ratified only
14	Column selector: ▦ opens show/hide with reorder and Reset	Ratified only
15	Row actions are **icons** with tooltips, never text links. Max 2 visible + overflow	Q27 — ENFORCED

15	Row actions are **icons** with tooltips, never text links. Max 2 visible + overflow	G37 — ENFORCED
16	Pagination footer on every list table. Wizard summary tables exempt	shared DataTable
17	Empty state on every list: what is missing, and the action that fills it	Ratified only
20	One table type across the product	Ratified only
31	**Nothing internal is ever visible to a utility user** — not an id, not a rule number, not a path	Ratified only
32	A screen has a description budget; rationale is zero words on screen	Ratified only
33	On-screen text capped by what it is for: label ≤3 words · fact ≤6 · instruction ≤6 per sentence	Ratified only
34	One pair of words for *held here* and *in the books* — **Recorded** and **In the books**, in all eleven catalogues	Ratified only
40	Page-head title is **32px**, set on the element	Ratified only
43	A piece of UI is built as a **component**; standalone code inside a page is not reuse	Ratified only

> GAP **Why a green suite proves nothing here**
>
> **Twelve of these thirteen would never be caught by any machine in this repository**, because no gate reads rendered output. Every violation in §05 sits under a fully green test suite, a clean `tsc` and a passing `run_gates`. The enforcement column exists so a reader can see which rows a machine would ever have caught: **one**, and the screens pass it.

D · Two product rules that fail invisibly

Rule	What it says	On these screens
NFR-3 · NFR-5	Universal server pagination; no hard caps anywhere	**Breached twice** — Outstandings and Collections. **Honoured** on the payments list
Hard rule 2	Money stored to 4 decimals, percentages 6, both **display 2**; never float	Honoured on every money cell traced here; all route through `MoneyCell`. The one browser-side sum uses BigInt rather than floating point
Hard rule 4	Financial immutability, narrowed for exactly three columns on `payments`	**Honoured exactly, at both layers.** The best-built thing in the module — screen 05

The eighteen breaches, in one place

Each one is stated again on the screen it belongs to. It is counted here because **spread across thirty-five screens the total disappears**, and the total is the finding.

Rule	×	What is broken, and where	Caught by?
31	3	Raw message keys reach a clerk as screen text — the result panel after a payment, the problem list on two screens, the bad-debt panel	No
12	2	No table toolbar at all on the debtor list; a stub of one on the ladder	No
13	2	A static label reading "Sorted by balance" instead of a control; no sort on the ladder	No
14	2	No column chooser on either debtor screen	No
16	3	No pagination footer on the debtor list, the ladder or the write-off queue	No
11	3	**No consequence preview on any of the three money forms.** The kit's own `ConsequenceBox` is rendered at 52 places across seventeen areas — including the screen where a *customer* pays — and **zero times in staff payments**	No
6	1	**The reversal commits through a primary button.** The row action that opens the dialog IS danger-tiered — the warning is on the door, not on the act	**Partly**
NFR-3	2	The debtor list fetches every debtor and sorts in Python; the ladder is unpaginated and runs one query per debtor. **The second is not named anywhere else**	No

GAP **Seventeen of the eighteen cannot be caught by any machine in this repository**

No gate reads rendered output. Every row above sits under a green suite, a clean type-check and a passing gate run. **Green is not evidence here.**

And the eighteenth is worse than unchecked — it is checked everywhere except there. Rule 6 has rendered-output enforcement on two shared components, and B14's own status column names the survivor: *"a commit button that composes NEITHER shared component."* The reverse dialog is that button. **It is not a rule nobody got to; it is the gap the rule's own record names.**

SOURCE **Where every dropdown's values come from**

No drawing in this document authorises a literal array in code. 27 value-bearing controls across the 38 drawings, each one classified. **Where a control has no source, that is the finding, not the design.**

The repo argues this better than I would

`paymentsShared.tsx` on the tender list: the value and options are *"served from `GET /payments methods[]` — never a hardcoded set"*, and it says what happens when the list is empty: ***"an empty list is rendered as an empty picker, never as the six seeds: it means the tenant deactivated every method, and inventing the seeds back would offer codes the server refuses."***

That is the standard for every row below. A picker that invents its own options offers values the server will reject, and the clerk finds out at the commit.

Screen	Control	Class	Where the values come from	Reference	Note
2·1	**Source filter**	SIGNED	`payments.source`'s CHECK — three values	`P0.1`	Counter · Digital · Post
2·1	**Batch filter**	DERIVED	The batches this tenant has, from the list's own rows	—	one at a time
2·1b	**Why (reversal)**	SIGNED	`REVERSAL_REASONS` — CR-059 CS-31's four	`endpoints.ts:15608`	bank return · chargeback · dispute · recorded wrongly
2·3a	**Account**	DERIVED	`GET /accounts?q=` — the search's own matches	`AccountPickerFields`	never a list of all accounts
2·3a	**What this money is for**	SIGNED	`PAYMENT_PURPOSES` — `BR-PAY-012`, four values	`endpoints.ts:14745`	bill · deposit · plan_down · prepay
2·3a	**Method**	SERVED	`GET /payments methods[]` — **never a literal array**	`paymentsShared.tsx:239`	**An empty list renders an empty picker**, not the six seeds
2·3a	**Kind**	SIGNED	`PAYMENT_TYPES` — three values	`endpoints.ts:14740`	payment · credit · adjustment
2·3a	**Apply to**	SIGNED	`ALLOCATION_CHOICES` — `BR-PAY-001`'s four staff options	`paymentsShared.tsx:224`	⚠ **`newest_first` is deliberately absent** — `BR-PAY-010` makes the order a tenant SETTING
2·3b	**Method (tender)**	SERVED	The same `methods[]`	`paymentsShared.tsx:239`	refused once the payment is banked
2·3c	**What is wrong**	SERVED	`GET /payments/reason-codes` — *"the tenant's live reason list"*	`endpoints.ts:14998`	the reasons a credit or adjustment may carry
2·3c	**Apply to**	SIGNED	`ALLOCATION_CHOICES`	`paymentsShared.tsx:224`	as 2·3a
2·3d	**Why (cancel)**	NOT SERVED	No endpoint exists	`P3.2`	**The task creates it.** Do not seed a literal list
2·3e	**Why (reversal)**	SIGNED	`REVERSAL_REASONS`	`endpoints.ts:15608`	the reason decides which flow runs

2·3f	**Why (move)**	SERVED	`GET /payments/reason-codes`	`endpoints.ts:14998`	—
2·3g	**Apply to**	SIGNED	`ALLOCATION_CHOICES`	`paymentsShared.tsx:224`	as 2·3a
2·3h	**Send by**	SERVED	The channels `payment_received` declares	`rule_catalog.py:425`	email · sms · a print floor. **Multi-select**, B14 rule 9
3·3a	**Service centre**	SERVED	`GET settings/reference-data/service-centres` — the **back-office** read	`published_content.back_office_centres`	includes hidden centres; hidden is a state, not a deletion
4·3a	**Loads into**	SIGNED	`TARGETS` — **eight after `P1.1`**	`import_runs.py:356`	premises · units · 3 asset targets · accounts · parts · **payments**
4·3a	**Source**	SIGNED	The runner's source cards — **five after the Scan card**	`WORKFLOW_SOURCES` · `kinds.ts:724`	File · **Scan** · SFTP · API · Manual. The last three are held with *"Not switched on yet. Upload a file instead."*
4·3a	**What does this file mean?**	SIGNED	`WRITE_MODES` — and **payments passes ONE of the two**	`WriteModeChoice.tsx` · hard rule 4	Insert new only. *Insert & update* is refused by name — a payment on file is never updated
4·3a	**Mapping template**	SERVED	The tenant's own templates for this target	the runner's read	**The stage payments' own pipeline never had**
6·3a	**Reason (refund)**	NOT SERVED	A literal list today	`P4.4`	**Make it a lookup** or state why it is closed
6·3a	**How**	SIGNED	`REFUND_METHODS` — two values	`endpoints.ts:15529`	check · ach. **Not the tender list** — money out is not money in
6·3c	**Reason (credit)**	SERVED	Billing's own credit reasons	`corrections.execute_credit_only`	billing owns the vocabulary, payments opens the door
7·1	**Ageing band filters**	DERIVED	The five bands, computed from the posting date	`outstandings.py`	each card is a filter
7·3d	**Which notice**	SERVED	`GET /notice-tones` — **and which LETTER each tone sends**	`settings_api.py:673`	⚠ ***Firm* sends the regulated 10-day notice**, not a firmer reminder. Day numbers from tenant config, never constants

7·1	**Why (write-off)**	SERVED	`GET /payments/reason-codes`	`endpoints.ts:14998`	⚠ a CHECK constraint refuses two reasons the prototype drew

SERVED	**10** — an endpoint field. The list is whatever the tenant has configured, and an empty one is an empty picker.
SIGNED	**12** — a closed enum with a rule id. **A tenant cannot add a value and a lane must not either.**
DERIVED	**3** — computed from data already on screen. No list to hardcode.
NOT SERVED	**2** — no endpoint exists. **The named task creates one; it does not seed a literal.**

The two rows to read twice

2·3a *Apply to* is SIGNED with a deliberate hole. `ALLOCATION_CHOICES` carries four of `BR-PAY-001`'s options and **`newest_first` is deliberately not among them** — `BR-PAY-010` makes the order a tenant SETTING, and offering it on the form would let one clerk silently override what the tenant configured once. **A lane that "completes" the enum breaks a rule.**

6·3a *Reason* is a literal today and should not be. It is the one control on this list whose options are written into the front end with no rule id behind them. **`P4.4` makes it a lookup or states why it is closed** — either is fine; leaving it a literal is not.

02

Development plan

Six increments, each shippable on its own, and every one a task table. No dates — the order is set by what depends on what, and by putting the cheapest value first. Every task carries **what it involves, the reference a developer looks up, and the outcome that says it is done**; every phase names what a clerk can *newly do*, because a phase that cannot answer that is a phase nobody can sign off.

How to read the task tables

Kind is DB, Backend, Front end, Settings, Test, E2E or Other — **all in one table on purpose**, because a migration and the test that proves it are one piece of work and splitting them into separate lists is how the second half gets dropped. **Reference** is a file and line, a rule id, or a journey — the thing to open, not a description of it. **Outcome** is what is true afterwards, phrased so it can be checked rather than asserted.

A ⚠ in a task is a decision or a hazard, not a step: something to settle or verify before the code is written.

Phase 1 is deliberately not the counter, and that is the whole shape of this plan

The counter is the biggest gap. **Imports is the biggest return**: four finished, tested engines with no screen, standing between a clerk and 4,000 to 6,000 receipts a month they key by hand. It needs **one dict entry** of backend and nothing else. **Ship the reachability before the new engines.**

The Summary is not a phase — it grows with every phase

1·1's worklist has five rows and each belongs to a different tab: import rows needing an account, uncounted tills, refunds waiting, failed payments, overdue accounts. **Building it once at the end means building it against four things that do not exist yet.** So each phase adds its own row, and the landing screen gets more useful as the work lands rather than all at once. Its counter band ships in phase 2 with the till, and its source split in phase 2 with the column.

Two things gate everything, and neither is in payments

The ledger's batching rule. Six Counter screens and the Batch column all assume a batch is one person's session. The database enforces day × method today, so **a counter close has nowhere valid to write until that constraint is replaced** — phase 0, task 6, and a ledger change.

Billing's barcode. The **Scan paper** source on 4·3a is a keyboard input over a bill that carries no code — the document code declares “no remittance barcode” on every result. **The scan screen is cheap and useless without it**, which is why they ship in the same phase.

Phase 0 Foundations 17 TASKS

Nothing a clerk can see, and everything else waits on it.

Ships	No screen. The shell, the kit, four migrations and one settings row.
What a clerk can newly do	Nothing. **The only phase with no user value**, which is why it is kept as short as it can be.
Task mix	6 db · 1 backend · 5 front end · 1 settings · 3 test · 1 e2e
Done when	**The seven tabs render and are reachable, a payment carries a source, a counter session can be written, and a posting date lands on the tenant's books.**

Done when	**The seven tabs render and are reachable, a payment carries a source, a counter session can be written, and a posting date lands on the tenant's day.**

TASK **Every row below is committed work**

Task	Kind	What	What it involves	Reference	Outcome
P0.1	DB	`payments.source`	A text column with a CHECK over `counter | digital | post`.	`payments/0029` · `models.py` Payment	**Every payment says where it came from.** The row has no such field today.
P0.2	DB	**Backfill the source**	Derive it: non-null `webhook_key` → digital · named by a `payment_import_rows.payment_id` → post · otherwise counter.	`webhook_key` · `PaymentImportRow.payment_id`	No nulls, and **a row matching none of the three fails loudly** rather than defaulting.
P0.3	DB	`counter_sessions`	Owner, service centre, opening float at `NUMERIC(18,6)`, opened_at, closed_at, closed_by, close_kind, counted totals per method, difference, note.	`payments/0030` · B2 §2	A till can be written. **Six screens in phase 2 wait on this one table.**
P0.4	DB	**One open till per person**	A **partial unique index** on (tenant, owner) `WHERE closed_at IS NULL`.	`payments/0030`	**The database refuses a second open till** — not a service check that a second caller can skip.
P0.5	DB	**RLS on the new table**	Its own migration, as `0028` is for counter receipts: `FORCE ROW LEVEL SECURITY` and the tenant policy.	`0028_rls_cr094_counter_receipts.py` · hard rule 1	**A tenant table with no RLS migration is hard rule 1 broken.**
P0.6	DB	**Re-key the deposit batch**	Drop `uq_deposit_batches_day_method` and key on the session. ⚠ **A data migration, not just DDL** — existing batches ARE day × method.	`bankrec_models.py:300` · BR-PAY-025	A batch is one person's session. **Decide the legacy rows before writing it** — a dropped unique key with no replacement lets two batches collide silently.
P0.7	BACKEND	`isoToday()` **reads the tenant timezone**	It returns `new Date().toISOString()` — the UTC day — while the setting exists and the date component honours it.	`PaymentsPage.tsx:677` · `DateCell.tsx:154` · Company → Regional	**A 17:30 shift in UTC−7 stops stamping its payments into tomorrow.**
P0.8	FRONT END	**The seven-tab strip**	It is four today — Summary, Record payments, Outstandings, Collections. Every screen in §03 draws seven.	`paymentsShared.tsx` `PAYMENTS_TABS`	Seven tabs render on every payments screen.
P0.9	FRONT	**Routes and shell doors**	Counter, Imports and Settlements need routes, and **doors in the shell**	`App.tsx:1316` ·	**Every new tab is reachable**

	FRONT END	**Routes and shell doors**	Counter, Imports and Settlements need routes, and **doors in the shell** — the nav renders no submenus, so a tab is reachable only from the page's own strip.	`SHELL_NAV` · PO 2026-08-10	**Every new tab is reachable.** Three screens in this audit were built and unreachable; this is how that is avoided by design.
P0.10	FRONT END	**Kit: the selection toolbar**	One control for bulk selection — the toolbar's own row, its contents swapped.	`DataTable.tsx:519` · B14 rule 12	**Two lists use it and the third inherits** rather than inventing a fourth pattern.
P0.11	FRONT END	**Kit: ageing cards and the stage rail**	Two small components the drawings already use.	`WORKFLOW_STAGES` · `kinds.ts` · B14 rules 32, 36	Payments draws the runner's rail and the debtor bands from the kit, not from copies.
P0.12	FRONT END	**`ConsequenceBox` into payments**	It is rendered at 52 call sites across seventeen areas — **and zero times in staff payments**.	`ConsequenceBox.tsx` · B14 rule 11	The rule-11 box exists in this module for the first time. **Three violations in §06 are fixed by it.**
P0.13	SETTINGS	**One number series**	The settlement series. Counter batches reuse `deposit_batch`, imports reuse `import_run`.	Configure → Number series (30 exist)	**The entire Settings change for all 35 screens.**
P0.14	TEST	**RLS probe, on PostgreSQL**	Both directions on `counter_sessions`. ⚠ **The suite defaults to in-memory SQLite**, where the guarantee cannot exist — 192 skips once read as a pass.	`config/settings/test.py` · hard rule 1	Tenant A cannot read or write tenant B's till, proved where the policy actually runs.
P0.15	TEST	**Constraint and backfill cases**	The partial index refuses a second OPEN till and **permits a second CLOSED one**. Four backfill cases including the loud failure.	—	The easy-to-miss half is covered.
P0.16	TEST	**Migration forwards and backwards**	Migration ordering is a 🔒 sensitive path.	CLAUDE.md sensitive paths	The deposit re-key is reversible.
P0.17	E2E	**No new journey**	Nothing a user does changes — the honest statement for a foundations phase.	`08-qa/01-e2e/journeys/` (354)	**All 354 still pass.** The source backfill is the reason to run them.

Phase 1 Make what exists reachable 23 TASKS

Four finished engines get a screen. Almost no new backend.

Ships 4·1 Batches · 4·2 Run · 4·3a Import · 2·3f Move · 2·3g Reapply · 2·1b the selection toolbar · 7·3c the retirement

What a clerk can newly do	**Import a lockbox or store file** — the 4,000 to 6,000 receipts a month keyed by hand today. **Move a payment** and **re-apply it**, both of which the product refuses in words while the endpoint is served.
Task mix	5 backend · 2 db · 7 front end · 2 other · 4 test · 3 e2e
Done when	**A clerk uploads a store file, maps its columns, fixes the rows that did not match, commits it, and the payments appear on the one list with their run. And there is exactly one write-off queue.**

TASK Every row below is committed work

Task	Kind	What	What it involves	Reference	Outcome
P1.1	BACKEND	**An eighth `ImportTarget`**	`payments` as a dict entry: column set, aliases, default mapping template, dedupe key, rule set.	`import_runs.py:356 TARGETS · BR-MET-096`	**One dict entry.** The runner has absorbed seven targets on exactly this shape.
P1.2	BACKEND	**The writer seam**	LOAD resolves **payments'** `import_writer`, as `parts` resolves work's.	`resolve("work", …, kind="import_writer") · G78`	**Metering writes nothing of ours** — structurally, not by care.
P1.3	DB	**Check the target constraint**	⚠ If `import_runs.target` carries a CHECK, an eighth value is a migration. If it is validated in Python only, it is not.	`import_runs.py:3236`	**Read it before assuming.** Cheaper to verify than to be wrong about.
P1.4	DB	**Keep the old staging tables**	`payment_import_batches` and `payment_import_rows` hold the provenance of every payment they created.	`payment_import.py`	**Dropped in a later phase, with that stated** — phase 0's backfill still reads them.
P1.5	FRONT END	**Mount the existing imports list**	With `target=payments`. Three copies already exist for premises, assets and accounts.	`PremisesImportsTab.tsx · AccountImportsTab.tsx`	**A fourth caller, not a fourth implementation.**
P1.6	FRONT END	**Screens for Move and Reapply**	`movePayment`, `movePreview`, `reapplyPayment` and `reapplyPlan` are written and **called by nothing**.	`endpoints.ts:15843, :15950`	Two finished engines become usable. **No backend at all.**
P1.7	FRONT END	**Delete the Reapply refusal**	The bulk bar says “Reapplying is not built yet” over a served endpoint.	`paymentsBulk.tsx:637`	**A control that lies stops lying.**
P1.8	FRONT	**The selection toolbar on**	With the eligible count and the per-row confirm.	`bulkActionRefusal ·`	**The button reads *Reverse 2*,**

	END	**2·1b**		`paymentsBulk.tsx:336`	**never *Reverse 3*.**
P1.9	OTHER	**Retire `payment_import.py`**	Six endpoints and two staging tables doing four of the runner's five stages, with **no Transform**. Or amend the rule.	`BR-MET-096` · `assets/services.py:2132`	**One import pipeline.** A PO call — assets' staged path was retired for exactly this.
P1.10	OTHER	**Delete the orphaned write-off route**	The queue already lives as a working panel in Revenue & AR.	`App.tsx:1435` · `RevenueArPage.tsx:713`	**One write-off queue, not two.**
P1.11	TEST	**The target end to end, on PostgreSQL**	Upload, map, validate, commit — **and one bad row among four hundred**, proving the savepoint.	hard-won law 14	**Cannot be proved on SQLite**, which has no aborted-transaction state.
P1.12	TEST	**The ambiguous match stays for a person**	Already has tests; they must pass against the new target.	`payment_import.py` rule 2	A name-and-amount guess never commits on its own.
P1.13	TEST	**A guard on the deletion**	A test that fails if the orphaned route is mounted again.	—	**The deletion cannot come back.**
P1.14	E2E	**NEW `J-PAY-lockbox-morning`**	A store's file with its own column names. Upload, map, twenty-three rejects, resolve, commit.	`08-qa/01-e2e/journeys/`	**The payments appear on the one list with their run and Source · Post.**
P1.15	E2E	**NEW `J-PAY-wrong-account`**	Move a payment taken on the wrong account.	—	**Both customers' balances move** — the one that got it wrongly owes again.
P1.16	E2E	**Regression**	`J2-read-to-cash`.	—	A hand-recorded payment behaves identically.
P1.17	FRONT END	**Mount the runner, not a copy of it**	`WorkflowRunner` and its panels through the ONE import path. `ImportWorkflowPage.tsx` is the reference — it mounts the kit and carries **zero stylesheet classes**.	`components/workflow/index.ts` · `CR-054` task 18	**No hand-rolled stage, rail or reject table.** The kit's own header records what happened the last time a screen took half of it.
P1.18	FRONT END	**`WriteModeField` on step 1**	**One control for every import in the product.** Payments passes ONE option — `options` is a prop, and *Insert & update* would overwrite a payment on file.	`CR-042` · `BR-MET-210` · hard rule 4	**A clerk who has used the premises importer finds the same control here**, and the refused option states its reason.

P1.19	FRONT END	**A Scan paper source card**	A fifth card on Extract. A sack of store cheques is a SOURCE, not a second screen — everything after it is the runner.	`WORKFLOW_SOURCES` · `SourceCards`	**One screen fewer.** Scanning at the window stays on 2·3a, where the money is taken there and then.
P1.20	BACKEND	**A sample CSV for the target**	**PO ruling 2026-08-21:** *"we will need sample CSV files to be downloaded at every import at step 1"*. The file is the SERVER's — header row derived from the target's real columns, plus one example row.	`CR-025` · `BR-MET-088` · `metering.import.sampleCsv`	**It comes with the column set**, not as a separate build — but a target registered without it shows a button that fetches nothing.
P1.21	BACKEND	**Seven catalogue keys for the target**	`target` · `kicker` · `blurb` · `next` · `exit` · `return` · `rollback` — the shape every other target already fills.	`kinds.ts` · `en.json`	**A target registered without them shows an empty promise** where the others show a sentence.
P1.22	BACKEND	**The rollback sentence is a REFUSAL**	A posted payment cannot be unwritten. Reads is the only other target that refuses outright.	hard rule 4 · `workflow.reads.rollback`	**No rollback point, no *Roll back this run* button** — and **Discard this batch** before the commit is the only way to throw a run away.
P1.23	TEST	**The refusal renders, and the button does not**	Assert the payments target shows its rollback sentence and offers no rollback control.	hard-won law 7	**A refusal nothing asserts is a refusal that rots** — law 2's mirror, in the one place money lands in bulk.

Phase 2 The counter 19 TASKS

The biggest single gap: six screens behind one table.

Ships	3·1 Counters · 3·2 My counter · 3·3a–d the four acts · 2·1 Source and Batch · 2·3a the banner
What a clerk can newly do	**Open a till, take a shift's money into one batch, and balance the drawer.** None of it is possible today — no session, no float, no count.
Task mix	3 backend · 2 db · 4 front end · 2 other · 5 test · 3 e2e
Done when	**A clerk opens a counter at 08:55, takes seventeen payments, closes it at 17:30 with a counted drawer, and a supervisor can see the three tills nobody counted.**

TASK Every row below is committed work

Task	Kind	What	What it involves	Reference	Outcome
P2.1	BACKEND	`POST /counters`	Opens the session **and its batch in one act**, and refuses a second open counter for the same user.	PO ruling 2026-09-21	**One act, not two.** The batch exists before the money does.
P2.2	BACKEND	**Close, force-close, declare**	Three endpoints, three different rows.	`reconciliation.cut_deposit_batches`	A shift can be sealed with a count, without one, or counted late.
P2.3	BACKEND	**The reads**	`GET /counters` and `GET /counters/{id}` with the running split by method.	`counter_receipts.list_totals`	The till shows what it holds, live.
P2.4	DB	`payments.counter_session_id`	A nullable deferred uuid — the shape `deposit_batch_id` already uses.	B11 F-3 · `models.py:93`	A payment names its till. **No ORM edge into another module.**
P2.5	DB	**An index on (tenant, counter_session_id)**	The till's own list reads it on every payment taken.	—	The running total stays cheap as a shift fills up.
P2.6	FRONT END	**The six Counter screens**	Counters, My counter, Start, Close, Force close, Declare — all from the kit.	§03 tab 3	A shift can be run from the product.
P2.7	FRONT END	`Start counter →` **in every page head**	**The control was missing from the whole product.**	your finding, 2026-09-21	**The counter screens ship reachable** — without this they would not be.
P2.8	FRONT END	**Source and Batch on 2·1**	Two columns and a filter chip for each.	phase 0's `payments.source`	The one list says where every payment came from.
P2.9	FRONT END	**2·3a gains the banner**	And **offers** `Start counter →` rather than only refusing when no till is open.	hard-won law 2's mirror	**A refusal with a way out.**
P2.10	OTHER	**Finance picks up the batch**	A closed counter batch **is** the deposit batch, so it appears on Deposits ready to match.	phase 0's re-key	**Where phase 0's constraint change pays off.**
P2.11	OTHER	**Notifications: the unclosed till**	A supervisor is told.	`rule_catalog.py`	Nobody has to notice a missing count by looking.
P2.12	TEST	**Open and its batch, one transaction**	And a second open for the same user refused.	—	The ruling holds in code.

P2.13	TEST	**The id attaches when taken**	**Not at close.**	PO ruling 2026-09-21	**The test is what stops this drifting back** — an earlier draft had it stamping at close.
P2.14	TEST	**Close with and without a difference**	Three states from three endpoints.	—	Each close kind is distinguishable afterwards.
P2.15	TEST	**The ledger pair, and the absence of one**	Balanced at take; **closing writes no journal line**.	hard rule 5	Counting is not a posting.
P2.16	TEST	`Start counter` → **asserted by address**	On every payments screen.	—	**The test that would have caught the control being missing at all.**
P2.17	E2E	**NEW** `J-PAY-counter-shift`	**The one that matters.** 08:55 open with a $200 float · seventeen payments in cash, cheque and card · 17:30 count · close.	`08-qa/01-e2e/journeys/`	**One journey, the whole phase** — and it ends on Finance → Deposits.
P2.18	E2E	**NEW** `J-PAY-forgotten-till`	Home without closing → auto-close at midnight → *Not counted* → declared next morning with a reason.	—	The forgotten-batch design is walkable.
P2.19	E2E	**Regression**	`J19-month-end-close`.	—	**Because the batching rule changed under it.**

Phase 3 A payment has an address 21 TASKS

Open one payment, and put it right.

Ships	2·2 Detail · 2·2r the row panel goes · 2·3c Correction · 2·3d Cancel · 2·3e the tier · 2·3h Resend
What a clerk can newly do	**Send a colleague a link to a payment. Cancel one taken by mistake** — the only genuinely missing logic in the module.
Task mix	4 backend · 4 db · 4 front end · 5 test · 2 other · 2 e2e
Done when	**A clerk opens PMT-2609-000418 at its own address, cancels a payment taken five minutes ago, and the history says who did what.**

TASK **Every row below is committed work**

Task	Kind	What	What it involves	Reference	Outcome
P3.1	BACKEND	`GET /payments/{id}`	There is no such read, which is **why a payment has no address**.	`PAY-H · paymentDetail.tsx:93`	A payment can be linked, bookmarked and returned to.
P3.2	BACKEND	`POST /payments/{id}/cancel`	The only genuinely missing logic in the module.	`reversal.py:496`	**A payment taken by mistake can be undone** without breaking immutability.
P3.3	BACKEND	`POST /payments/{id}/receipt`	Re-queues an existing document — templates, channels and queue all exist.	`receipts.py:684`	A lost receipt can be sent again.
P3.4	BACKEND	**An `audit_log` read**	For the history panel.	hard rule 7	**What makes Cancel honest** — a cancelled payment does not vanish.
P3.5	DB	**A sixth payment state**	⚠ `ck_payments_status` is written against the five, so `cancelled` needs a migration that **alters the CHECK**.	`models.py:496`	The state exists in the database, not only in the service.
P3.6	DB	**The immutability trigger, both ways**	`PAYMENT_FROZEN_STATUSES` is every status but `processing` — so the trigger **permits the move a cancel needs** and **freezes the row once it lands**.	`trg_payment_immutable · payments/0002`	**Prove both directions.** Neither half is safe to assume.
P3.7	DB	`payments.corrects_payment_id`	A self FK — the mirror of `reverses_payment`.	`models.py:112`	**An adjustment names what it corrects** instead of being related only in a clerk's memory.
P3.8	DB	**No change to the allow list**	`0014` is the only place it moves, and it moved once, for three columns.	`payments/0014 · PO 2026-09-02`	**Cancel writes a status and a new row; it does not widen immutability.**
P3.9	FRONT END	**2·2 as a page**	Its own address, the header strip, the history panel.	§03 tab 2	A payment has a home.
P3.10	FRONT END	**Delete the row panel in the same commit**	**Not before** — no way to see what a payment paid off. **Not later** — two answers to one question.	`paymentDetail.tsx`	**One commit: the page arrives, the panel goes, the 👁 appears.**
P3.11	FRONT	**Cancel, Correction,**	Three modals, with consequence boxes on all three money forms.	B14 rules 10, 11, 33	Three rule-11 violations in §06

	END	**Resend**			close.
P3.12	FRONT END	**The reversal commit moves tier**	It renders like Save today, while the row action that opens it is already danger-tiered.	`FormModal.tsx:1512` · `StateFrame.tsx:289` · B14 rule 6	**The warning is on the act, not just the door.**
P3.13	TEST	**Cancel by status**	Succeeds on `processing`; refused on `cleared` and the clerk is pointed at Reverse.	`BR-PAY-021`	The two words a clerk confuses stay two acts.
P3.14	TEST	**The trigger refuses a cancelled UPDATE**	And still permits the three amendable columns.	—	Both directions, as hard rule 4 requires.
P3.15	TEST	**A correction leaves the original byte-identical**	And `corrects_payment_id` points at it.	—	Immutability satisfied, not circumvented.
P3.16	TEST	**The reversal commit carries the danger class**	**B14's own row names a `FormModal` commit as the unchecked case.**	B14 rule 6 status column	**The gap the rule's own record names is closed.**
P3.17	TEST	**The swap cannot half-land**	The panel is gone **and** the 👁 is present, in one test.	—	No window with two detail views.
P3.18	OTHER	**⚠ Answer `OQ-1340` first**	A corrected reference's old value lives only in `audit_log`, which is pruned — and whether such a row is exempt was never put to the PO.	CLAUDE.md hard rule 4	**No screen may call this path a permanent record until it is answered.**
P3.19	OTHER	**Notifications: paper at the window**	The queue is asynchronous; a counter needs a receipt in hand now.	`receipts.py` · `payment_received`	A counter customer leaves with paper.
P3.20	E2E	**NEW `J-PAY-put-it-right`**	Four ways one payment goes wrong, in one walk: a mistyped cheque number · a wrong amount · taken in error · a bounce a week later.	—	**Four acts, and only the first writes in place.**
P3.21	E2E	**Regression**	`J12-nsf-life`.	—	The bounce path, which the tier change touches.

Phase 4 Debtors and money out 28 TASKS

The oldest screens, and the worst rule breaches in the module.

Ships	7·1 Debtors · 7·2 Debtor · 7·3a Ladder · 7·3b Write-offs · 6·1–6·3c Refunds
What a clerk can newly do	**Search, sort, filter and page the debtor list** — none of which works today. **Open one debtor.** And **choose a credit instead of a refund**, because the option is invisible from payments.
Task mix	6 backend · 2 db · 8 front end · 2 other · 7 test · 3 e2e
Done when	**A 40,000-account tenant can work its debtor list without the server materialising every open invoice, and a clerk closing an account in credit is offered both ways out.**

TASK Every row below is committed work

Task	Kind	What	What it involves	Reference	Outcome
P4.1	BACKEND	**Page and sort `/outstandings` in SQL**	The seam materialises every open invoice before filtering, and the sort runs in Python.	`api.py:674` · `outstandings.py:372` · NFR-3	**Correct at 3,000 connections; thirteen times short at 40,000.** Fixed.
P4.2	BACKEND	**One query for the ladder**	It runs one per debtor — an N+1, not an index problem.	`collections_read.py`	**The NFR-3 breach CLAUDE.md does not name.**
P4.3	BACKEND	**A debtor read**	Bills, ladder state and history for one account.	§03 screen 7·2	One call behind the screen the list never had.
P4.4	BACKEND	**`GET /refunds/{id}`**	No detail view exists — a raise modal and an act modal, nothing between.	`RefundsPage.tsx`	"Where is my money?" has an answer.
P4.5	DB	**Indexes for the debtor query**	A covering index on the balance read.	—	**Measure the current figure first**, or there is nothing to beat.
P4.6	DB	**No new table**	Refunds moving tab is a route change — model, service and API are already in `apps/payments`.	`paymentsShared.tsx:113`	Everything about refunds is payments except the URL.
P4.7	FRONT END	**7·1 redrawn**	Four KPI tiles · five ageing cards as filters · full toolbar · ⇅ headers · paging · row actions · selection toolbar.	B14 rules 12–16, 32	**Five rule breaches on one screen close.**
P4.8	FRONT END	**7·2 Debtor**	A screen the list has never had.	—	A clerk on the phone has one place to look.
P4.9	FRONT END	**7·3a and 7·3b**	The ladder gets the same treatment; the write-off panel gains a toolbar and paging.	—	The remaining rule-12/16 rows close.

P4.10	FRONT END	**Refunds moves, and Credit joins it**	To the Refunds tab, with a detail view, and **Credit beside Raise**.	supersedes CR-059 §10's placement	**The cheaper option becomes visible where the choice is made.**
P4.11	FRONT END	**Six buttons to two plus overflow**	Four of the six have never worked.	`OutstandingsTab.tsx:644`	The bar stops teaching clerks to ignore it.
P4.12	OTHER	**Billing: a door to the credit path**	**Not a copy of it.**	`corrections.execute_credit_only`	One credit path, reachable from two places.
P4.13	BACKEND	`POST /collections/notices`	⚠ **Not mounted today** — only `collections/run` and `collections/ladder` are. The vocabulary, the tone mapping and the five skip codes all already exist.	`api.py:1703` · `LADDER_RUNGS` · BR-PAY-017	**A clerk can send a notice by hand**, and it shares ONE vocabulary with the automatic ladder — which is what makes no-double-send possible.
P4.14	FRONT END	**7·3d Send notice**	Three steps: who, the picker with the letter each tone selects, and the per-row sent-or-skipped table.	`GET /notice-tones` · BR-NOT-008	**A clerk sees that *Firm* sends a regulated 10-day notice** before pressing, not after.
P4.15	TEST	**A tone selects the right letter**	And the day numbers come from tenant config, never constants — **and the tones still render when the settings cannot be read**.	BR-REP-016	The trap the route was built to disarm stays disarmed.
P4.16	TEST	**Every skip is recorded and shown**	Five reasons: below floor · medical hold · plan paused · partial payment · nothing overdue.	`ladder.py:168`	**No silent exclusion** — the one thing a collections trail may not contain.
P4.17	OTHER	⚠ **Re-check the site-visit refusal**	`RaiseFieldWork` is live on four screens and Work is the eleventh module. The refusal claims a missing *rule*, not a capability.	`RaiseFieldWork.tsx` · CR-070	**Nothing in this product re-checks a refusal.** This one gets checked.
P4.18	TEST	**The debtor list at 40,000**	One query, paged, sorted in SQL, and the plan does not materialise every open invoice.	NFR-3 · NFR-5	The ceiling the ICP actually has.
P4.19	TEST	**Five controls asserted on 7·1 and 7·3a**	Search, filter, sort, columns, paging.	—	Five assertions the screens fail today.
P4.20	TEST	**The selected total stays BigInt**	A selection whose sum a float would round.	hard rule 2	**No float on the money path, ever.**

P4.21	TEST	**The old refunds address**	**404s or redirects deliberately**, not by accident.	—	No silent dead link.
P4.22	E2E	**NEW** `J-PAY-chase-the-morning`	Filter to over sixty days · open a debtor · see what was tried · take a part payment.	—	**The ladder stops.**
P4.23	E2E	**NEW** `J-PAY-credit-or-cheque`	An account closes in credit and the clerk is offered both ways out.	`credit_issued`	**The customer is told** — a live event.
P4.24	E2E	**Regression**	The write-off two-person control.	`BR-PAY-020`	A requester cannot approve their own.
P4.25	BACKEND	**One read for refunds AND credits**	Payments cannot query billing's table. The list unions payments' refunds with billing's `CreditNote` through `apps/payments/billing_seam.py`.	`billing/models.py:1383` · hard rule on module boundaries	**One list, two number series** — refunds' own and **Credit memo #**, already one of the thirty configured.
P4.26	FRONT END	**A Kind column and filter on 6·1**	Refund or Credit, and a chip that shows one or both. **The columns must not flatten the two lifecycles:** a refund is raised, approved, issued, cleared; a credit note has **no state column at all**.	§03 screen 6·1 · PO 2026-09-22	**A clerk sees everything given back in one place** and can still tell a cheque from a credit.
P4.27	FRONT END	**7·3d is a wizard, not a modal**	Three steps on `WizardFrame`: who · which notice · **the rendered letter**. The step is URL state and clamps to the highest validated step.	`components/WizardFrame.tsx` · `BR-SHL-018`/**019**	**Payments mounts the shared frame for the first time.** Forty other flows already do.
P4.28	TEST	**The letter is shown before it is sent**	Assert the preview step renders the disconnect template when the tone is *Firm*.	`BR-NOT-008`	**The trap is disarmed on screen**, not in a helper line under a picker.

Phase 5 Settlements and the barcode 18 TASKS

Close the third source, and stop keying paper.

Ships	5·1 Settlements · 5·2 Settlement · the Scan source on 4·3a
What a clerk can newly do	**Tie a bank credit from a provider to the payments inside it**, which nothing does today. **Scan a bill instead of keying it.**
Task mix	3 backend · 3 db · 2 front end · 3 other · 4 test · 3 e2e
Done when	**A supervisor opens last night's payout and sees the forty-six payments inside it; a clerk scans a store's receipts instead of typing them.**

TASK Every row below is committed work

Task	Kind	What	What it involves	Reference	Outcome
P5.1	BACKEND	**The settlement ingest**	A payout arrives, its members are written.	`webhooks.py` · PO 2026-09-05	**Being paid out is a second message, and it is handled.**
P5.2	BACKEND	**The fee posting**	**A fee is an expense, not a smaller payment** — the customer paid the gross.	hard rule 5 · `OQ-264`	The trial balance ties out on card takings.
P5.3	BACKEND	**Idempotency on a repeat payout**	On (tenant, provider, settlement id).	hard rule 6	A redelivery writes nothing twice.
P5.4	DB	`settlements`	Tenant, code, provider, paid_out_at, gross, fees, net, bank_line_id, status. Money at `NUMERIC(18,6)`, currency `CHAR(3)`.	hard rule 2	The third source has a batch.
P5.5	DB	`settlement_members` **and its RLS**	Settlement, payment, gross, fee — a payment belongs to at most one. Plus its own RLS migration.	hard rule 1	Every tenant table has a policy.
P5.6	DB	`payments.settlement_id`	Nullable — **null is *Awaiting settlement***.	§03 screen 2·1 ⑥	**The third state the requirement never named, made real.**
P5.7	FRONT END	**The two Settlements screens**	A list and one payout.	§03 tab 5	A bank credit can be tied to its payments.
P5.8	FRONT END	**The Scan source, switched on**	A focused input behind the **Scan paper** card on Extract — **a scanner is a keyboard**. Everything after it is the runner.	§03 screen 4·3a · `SourceCards`	Paper stops being keyed, **without a screen of its own**.
P5.9	OTHER	**Integrations: a settlement feed**	Providers are integration manifests, so this is an adapter, not a change to payments.	`BR-INT-037` · `G77`	**Adding a provider stays one manifest and one adapter.**
P5.10	OTHER	**⚠ Billing: the remittance barcode**	On the three-to-five standard layouts, carrying account, invoice, amount and a check digit. **No schema change** — it is drawn from the bill's existing payload.	`documents.py:111` · `OQ-149`	**Without it a scanner reads nothing.** Which is why it ships in this phase.
P5.11	OTHER	**Integrations: a card terminal**	Tap-and-pay at a window is a device, not a webhook.	—	The counter's third tender works at the counter.
P5.12	TEST	**A payout clears the third state**	Every payment in it stops reading *Awaiting settlement*.	—	The gap the batch rule could not close alone.

P5.13	TEST	**The fee tie-out**	The balanced pair, with the gross intact.	hard rule 5	Finance can reconcile card takings.
P5.14	TEST	**A barcode check digit**	Rejects a mis-read, with the worked case in the test.	—	A mis-scan fails visibly.
P5.15	TEST	**A scan with no barcode**	**Fails visibly** rather than silently keying nothing.	—	The empty case is designed, not discovered.
P5.16	E2E	**NEW** `J-PAY-settle-the-night`	Last night's payout: forty-six payments, the fee, and one bank credit matched.	—	The digital source is closed end to end.
P5.17	E2E	**NEW** `J-PAY-scan-the-sack`	Fourteen bills scanned at a desk, each filling itself.	—	They land in the open counter batch.
P5.18	E2E	**Regression**	Every journey that ends in a bank match.	—	Settlements did not move anything under them.

Across all six

What will bite, and where

Risk	Why, and what to do about it
A green suite proves nothing here	**Seventeen of the eighteen rule breaches cannot be caught by any machine in this repository** — no gate reads rendered output. Every phase needs a walk against the drawing, in a browser, or it ships the rule instead of the code. **An earlier pass of this document drew the reversal button in the danger tier because the rule says so; the code renders it primary.**
SQLite hides the import failure	A Load writes hundreds of payments in one transaction, and **on PostgreSQL a single failed statement poisons the whole transaction** — so skipping a bad row needs a savepoint, not a try/except. **The suite runs on in-memory SQLite, which has no such state**, so the broken version passes every test on every machine and fails the first real refused row in production. **Phase 0's RLS probe and phase 1's import test are proved on PostgreSQL or they are not proved.**
A hold outlives its reason	Two controls on the bulk bar say “not built yet” over served endpoints; four on the debtor list say it over three real absences and one that needs re-checking. **Nothing in this product re-checks a refusal.** So when a phase writes a hold, it writes the test that fires when the hold expires — asserting the destination by address — **in the same commit**.
Two answers to one question	This audit found **three**: two write-off queues, two import pipelines, and a credit path in billing that payments cannot see. Each arose because the second author could not see the first. **Before any phase adds a component, check whether the kit already owns that job** — which is how Imports went from six new screens to one.
Immutability is not	Exactly one act in this plan writes a payment row in place — **Edit, three columns, through one named endpoint** — and that is the entire narrowing anyone has

a convention	granted. Cancel, Correction, Reverse, Move and Reapply all write new rows. **A phase that treats any other field as editable is breaking a hard rule, not a convention.**
A migration with no way back	Migration ordering is a 🔒 sensitive path. **Phase 0's deposit re-key and phase 3's CHECK alteration are the two that cannot be improvised**: one drops a unique key that currently prevents silent collisions, the other changes what the immutability trigger will freeze. Both need a reverse migration and both need deciding before they are written.

03

The seven tabs

Every tab is drawn in one order and only this order: ① the list · ② one row opened · ③ what you can do to it · ④ what each of those changes and where. **A screen appears once**, in the step that owns it. Where the product today and the agreed design differ, the drawing shows **the end state** with the changes marked, and the spec table under it says what is true now.

Tab	Name	Screens	New / Update / Remove	Where it stands
1	**Summary**	1	— / 1 / —	**Redrawn as a worklist.** Today it reports; it should say what needs you
2	**Payments**	12	6 / 5 / 1	Mostly built. **Cancel is the only missing logic in the module**
3	**Counter**	6	6 / — / —	**Nothing exists.** No session, no batch, no float, no count
4	**Imports**	3	— / 3 / —	**The ONE import runner, mounted** — its rail and all eleven of its panels. 4,000–6,000 receipts a month
5	**Settlements**	2	2 / — / —	Nothing exists. Being paid out is a message we do not handle
6	**Refunds**	5	1 / 4 / —	Built, and moved here from Finance on 2026-09-21
7	**Debtors**	6	2 / 3 / 1	Built, and carries the worst rule breaches in the module

> TODAY **What changed in this pass, and why the document got shorter**
>
> **Five screens used to be drawn twice** — once as built, again as a remedy — which is how a reader ends up unsure which picture is the product. They are merged: **one drawing each**, showing where we are going, with every difference from today marked on the drawing and listed beneath it.

And step ④ is new. No previous version said what an action *changes* — which ledger rows move, which other screen goes stale, what the customer receives. That is the step a build lane needs most and the one that was missing.

GAP **Where the figures come from**

The lists, the debtor screens and the summary are drawn in the state the running app rendered on the walked tenant — its zeros, its empty states and its em-dashes included. **The modals need a payment to exist** and the payments fixture did not seed, so those are drawn from their components' render trees: every field, in its real order, with its real label and helper text read out of the JSX. Every screen's spec table says which it was under **Walked**.

Nothing is drawn from the rule when it should be read from the code. An earlier pass drew the reversal commit in the danger tier *because B14 rule 6 says so* — and the code renders it primary. An as-built drawing that shows the rule instead of the code launders a violation into something that looks shipped.

TAB 1

Summary — what needs you this morning

Where the sidebar lands, and the one screen every clerk opens first. **Today it is a report**: four panels, each saying what happened, none saying what

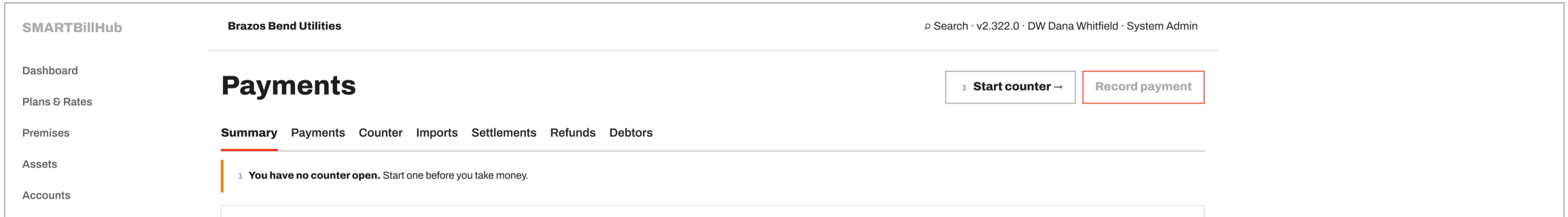
1 **The screen** Your counter, then what is waiting, then today

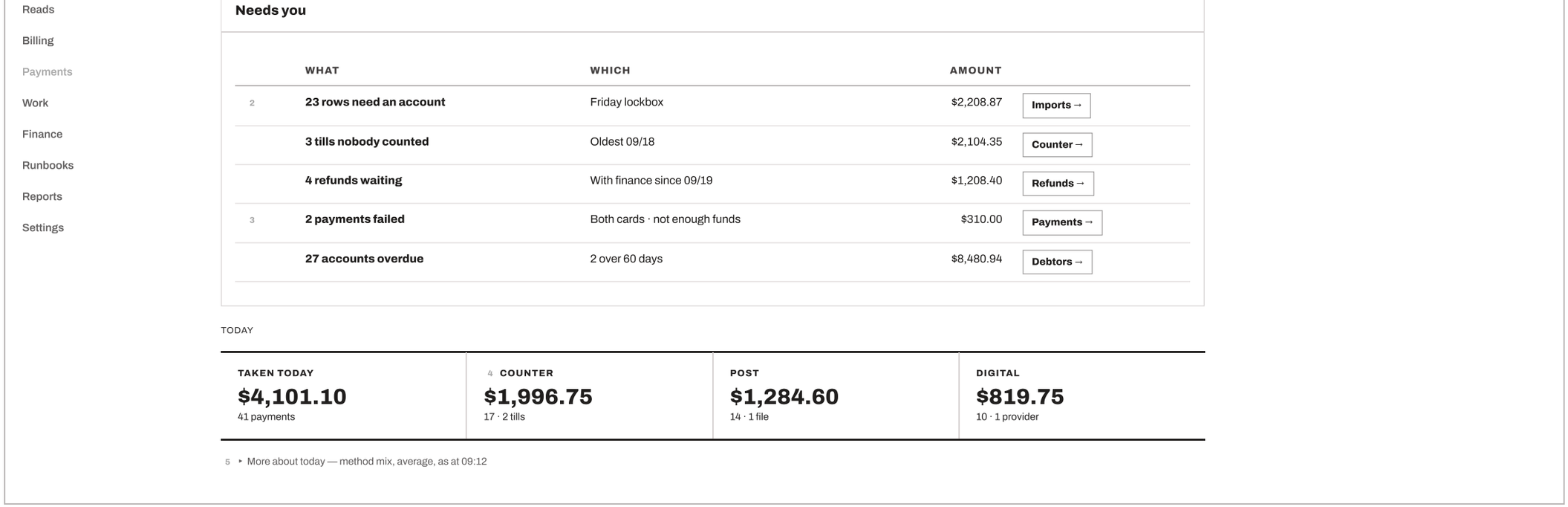
1·1 Summary UPDATE S32 · PAY-C · PAY-D

`/:tenant/payments → PaymentsPage → SummaryTab · second address /:tenant/payments/summary`

The question: "I have just sat down. What needs me?"

DESKTOP · 1440 · **REDRAWN AS A WORKLIST** · THE FIGURES ARE THE ONES THE RUNNING APP RENDERED





① **Your counter** · the first thing, because nothing else can happen until it is open · B14 **33** · NEW ② **Needs you** · a count, what it is, a figure and a door — **the page's new job** · NEW ③ **Failed payments** · folded in rather than given a panel of their own · MOVED ④ **By source** · counter · post · digital — with three ways in, where beats which tender · CHANGED ⑤ **Method mix** · a closed disclosure — the till shows it live on 3·2 · B14 **32** · MOVED

For	Every clerk, first thing, and the supervisor glancing at the day.
Doors	The sidebar. **1 door** — and it is the door, so this screen needs none named.
Primary action	**Record payment**, with **Start counter** → beside it. Both were absent from this screen.
Table controls	**Exempt.** *Needs you* is a fixed worklist of five rows, not a queryable list — the same exemption the method-mix table already carries. Rules 12–16 do not apply, and it is recorded rather than logged.
Write surfaces	**Detail** every row is a door · **Add** → 2·3a and 3·3a · **Edit** NOTHING · **Update** NOTHING. **This screen writes nothing**, which is why it carries no consequence box.
TODAY **Today**	**Four panels, and all four report.** Today · method mix · failed · collections. No counter, no worklist, no source split, and **no way to start a counter from the screen everyone lands on**.
TASK **To build**	One read that gathers the five waiting counts · the source split on the takings · the counter state · the method mix behind a disclosure.
Walked	**Yes.** The running app rendered $0.00 takings on the walked tenant, and its zeros are why the empty state below is drawn.

Walked	**Yes.** The running app rendered $0.00 takings on the walked tenant, and its zeros are why the empty state below is drawn.
Rules	B14 **2** · **4** · **12–16** exempt · **31** · **32** · **33** · **40**

TASK **Why the figures moved down the page**

The old page answered *what happened* four times over. **A clerk opening it at 08:50 cannot act on any of it** — the takings are a fact, the method mix is a fact, and the two lists at the bottom were the only things with a door. So the order inverts: **what needs you, then what happened. Nothing is deleted**; the method mix moves behind a disclosure because the till already shows it live, and the failed payments join the worklist because that is what they are.

Four tiles, and they now split by SOURCE rather than by tender. When there was one way money arrived, the tender was the interesting axis. With three — counter, post, provider — **where it came from is the question a supervisor actually asks**, and the tender split belongs on the till that is counting it.

GAP **The empty state is the one that matters here, and it is drawn from a real walk**

On the walked tenant this screen rendered **$0.00 taken, no failures, no collections** — and it was still four panels of zeros. **A worklist with nothing in it should say so in one line**, not draw five empty rows: *"Nothing needs you."* Three words, and the counter band above it still offers the one thing a clerk can do.

That is rule 17 doing real work rather than being satisfied on paper: **what is missing, and the action that fills it.**

TODAY **Steps ②–④ are empty here, and that is the honest shape**

A worklist has no row of its own to open — **every row is a door into another tab**, where the detail and the actions already live. And it writes nothing, so there is no impact to state. **Said rather than skipped**, so a reader can see the flow was applied and came out empty.

TAB 2

Payments — take money, find it again, put it right

The core of the module: one list holds every payment whatever its source, one payment opens to what it did, and eight things can be done to it. **Fou**

1 **The list** Every payment, whatever it came from

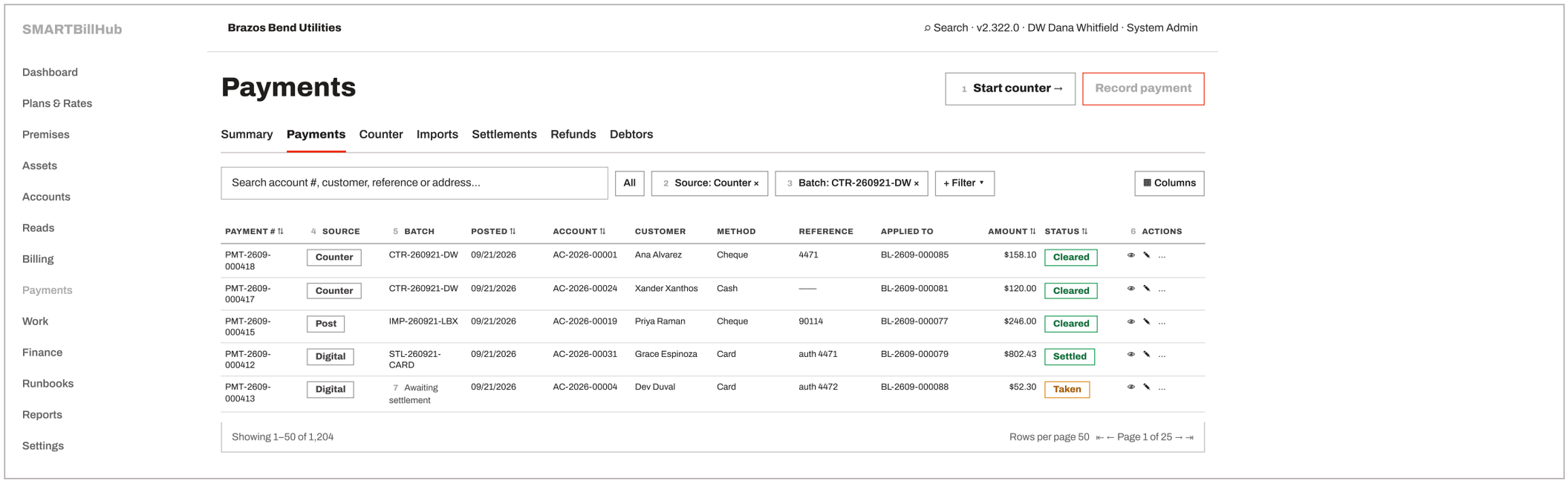
2·1 Payments

UPDATE | GAINS 2 COLUMNS · 2 FILTERS · 1 BUTTON | PAY-A · PAY-E · PAY-F · PAY-M

`/:tenant/payments/record → PaymentsPage → PaymentsTable (shared DataTable)`

The question: "Someone is at the counter with a cheque — and what have we already taken today?"

DESKTOP · 1440 · COLUMNS, HEADERS AND CONTROLS MEASURED IN THE RUNNING APP · ROWS ARE EXAMPLES · FOUR MARKED CHANGES



① **Start counter →** · opens your till **and its batch in one act** · B14 **4**'s **Verb →**, because it opens the screen in flow 2 · NEW — AND IT WAS MISSING ENTIRELY when your counter is already open this same slot reads **Your counter · CTR-260921-DW · open** and is the door back to it ② **Source** filter · Digital | Counter | Post · B14 **12** removable chip · NEW ③ **Batch** filter · one batch at a time · NEW ④ **Source** column · the three ways money reaches this list · NEW ⑤ **Batch** column · the id every payment carries · NEW ⑥ **Detail · Edit**, then **…** for Receipt and Cancel · B14 **15** caps visible icons at TWO ⑦ **Awaiting settlement** · a card taken but not yet paid out by the provider · NEW

For	The clerk who takes money, and the supervisor who checks what was taken.
How you get here	The tab strip on the summary and collections screens; the Summary tab's **Record payment**. **Not a rail door** — the nav lists `/payments/record` as a child but renders only parents as links, per the 2026-08-10 no-submenus ruling. **10 doors.**

Table controls	**All four work, and all four on the server.** Search posts `?q=` · six filters declared · column show/hide with five hidden by default, and the identity column can never be hidden · **sort and pagination both in SQL**. Only two handlers in the whole payments API build a paginator and this is one — the contrast with flow 4 is the point.
TODAY What is true today	Everything above **except the four marked changes**. There is no Source column, no Batch column, no filter for either, and **no way to start a counter from this page or any other**.
TASK What has to change	**One column on `payments`** recording where the money came from — the row has no such field, and a provider payment is currently only inferable from its callback key, a posted one from the import row that made it, and a counter one from nothing at all · **two filter declarations** · **one header control** · the batch column reads a value flow 2 creates.
Walked	**Yes** — signed in, tab opened, controls exercised. The tenant had no payments, so the rows are examples and the controls are not.
Rules	B14 **4** · **12–17** ✓ · **20** · **31** ✗ (the result panel prints a raw message key — §05) · NFR-3 ✓ · hard rules 2 and 4

GAP **⑦ is the finding this drawing produces, and no requirement names it**

"Every payment carries a batch id" and "the digital batch id arrives at settlement" cannot both be true at the moment a card is taken — settlement is later, and there is no settlement run in the product at all. **So there is a third state**, and a list that does not show it hands a clerk a blank cell and no explanation. Drawn as *Awaiting settlement*; the words are a placeholder for a ruling.

The counter and the post have no such gap, and for the same reason: their batch opens before the money is taken, so the id exists from the first payment. That is what makes ① and flow 2 one act rather than two.

2·1b Bulk

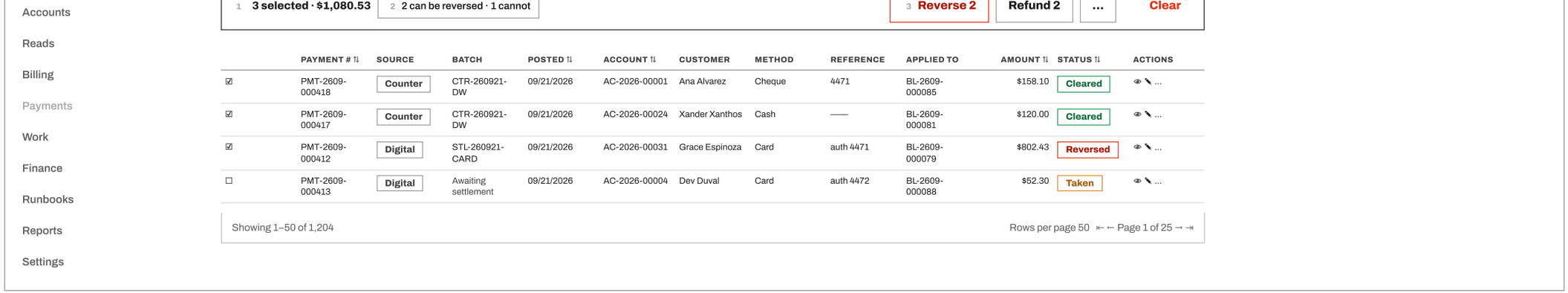
UPDATE · S33 · PAY-E

`the selection bar on the payments list — PaymentsPage.tsx, paymentsBulk.tsx`

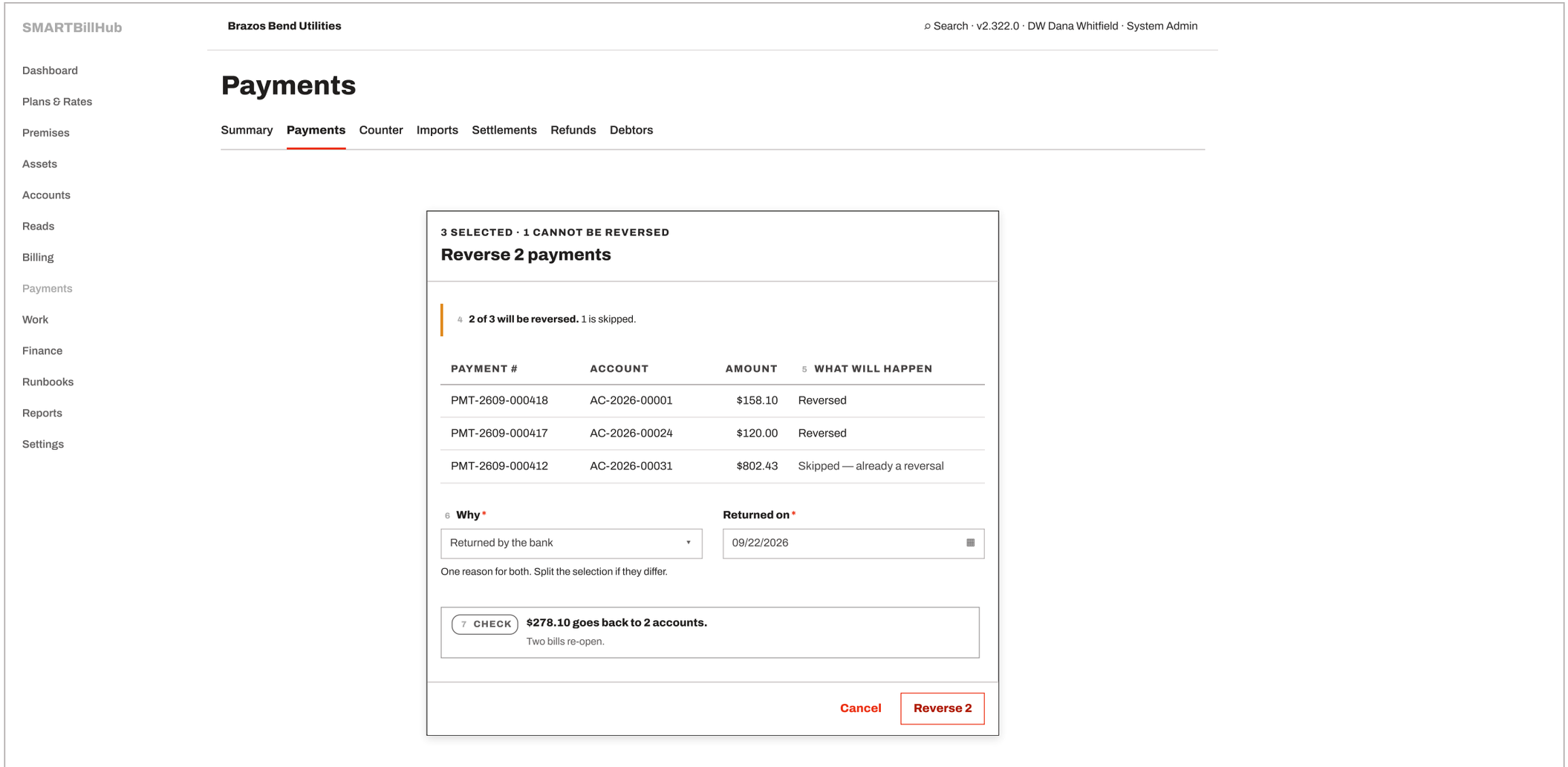
The question: "I need to reverse these three at once."

DESKTOP · 1440 · **THE TOOLBAR SWAPS IN PLACE** — THE SAME ROW, NOT A SECOND BAR · **THE SAME TWELVE COLUMNS AS 2·1**





THE CONFIRM IT OPENS — AND THE FIGURE IN THE BUTTON IS THE ONE THAT WILL RUN



① **The toolbar swaps in place** · search, chips and columns become the selection · **no second bar, no layout jump** · CHANGED ② **What can actually happen** · in the slot the filter chips vacated, because it is the same kind of fact · NEW ③ **Two visible plus an overflow** · B14 **15**'s grammar · **Reverse** is danger tier, **Refund** is not destructive · CHANGED ④ **Who is skipped, and why** · B14 **11** · NEW ⑤ **Per row** · a clerk reverses payments, not a number · NEW ⑥ **One reason for all of them** · said out loud, because the four reasons do different things · NEW ⑦ **What it will do** · in the customer's terms · B14 **11** · NEW Reapply and Send receipts are **off the bar entirely** — they return when 2·3g and 2·3h land

For	A clerk acting on several payments at once.
Doors	Ticking rows on 2·1. **1 door.**
Primary action	**Reverse**, danger tier, with **Refund** beside it. Nothing else on the bar.
Table controls	**The same twelve columns as 2·1**, plus the tick. This is that list with rows selected — not a second table.
TODAY **Today**	**Two of four controls are permanently disabled**, and the two that work tell a clerk less than the product already knows. See below.
TASK **To build**	The eligible count on the bar · the per-row confirm · a consequence box · **take the two dead controls off** until 2·3g and 2·3h land.
Rules	B14 **6** · **11** ✗ today · **20** · **33** · hard-won law 2's mirror

GAP **The bar goes live when it will mostly fail, and that is worse than the dead buttons**

`bulkActionRefusal` returns **"allowed" if ANY selected row can be acted on**. Select five payments where one is reversible and four are not, and the button is live, bright, and says nothing. The confirm that follows takes **only a count** — it asks *"Reverse 5 payments?"* when it is about to reverse one and silently refuse four. You find out afterwards, per row.

The product already knows which four. `bulkReverseRefusal(row)` computes a reason per row — not an admin, not cleared, already a reversal — and the bar throws that away for a single yes/no. **The fix is not new logic; it is showing what is already computed.** That is ② on the bar and ③④ on the confirm, and it makes the button honest: **Reverse 2**, never Reverse 3.

And it is a rule 11 breach. A bulk reversal writes money across several accounts and shows no "what this will do" box at all — the same gap as the three single-payment forms, one level up and multiplied by the selection.

GAP **The two dead controls come off the bar; they do not stay greyed**

Reapply says *"Reapplying is not built yet."* `POST /payments/{id}/reapply` is served, `reapplyPayment` is written in the API client, and **nothing calls it**. The refusal was true when written and stopped being true when the endpoint landed — and nothing re-checks a refusal. **Send receipts** is the same shape: bulk sending is not built, while a receipt queues automatically on every single payment.

A control that has never worked teaches a clerk to ignore the bar. So they come off, and return when 2·3g and 2·3h land — at which point the bar is four acts and takes rule 15's grammar: two visible, the rest under an overflow.

TASK **Three ways to put these controls on screen, and why this one**

A floating bar above the table — what this was — appears and disappears, pushing the table down at the exact moment you have just ticked three specific rows, and it is a *second* toolbar competing with the one B14 rule 12 already specifies.

A bar stuck to the bottom of the window survives scrolling a long selection, which matters at 1,204 rows — but it covers the last row and floats over content that is not a dialog.

The toolbar swapping in place, drawn here: the row already exists, so nothing moves; the controls appear where the eye already is; and rule 12's grammar is preserved rather than duplicated. **The eligible count lands in the slot the filter chips vacated**, because it is the same kind of fact — what this list is currently narrowed to.

If the selection scrolls out of view, the same row sticks to the top of the table rather than the bottom of the window — one behaviour, one place.

GAP ⑥ **One reason for every payment in the selection, and the four reasons are not alike**

A bulk reversal applies **one** reason to all of them, and the reasons behave differently: a bank return counts against the customer and sends a letter; a staff correction does neither. **Select a mixed batch and one of them is mislabelled**, quietly, in the record that decides whether a customer is chased.

The screen cannot solve that, so it says it: *"One reason for both. Split the selection if they differ."* **Six words, on the field where the choice is made.**

2 One payment opened What this money did, and what has happened to it

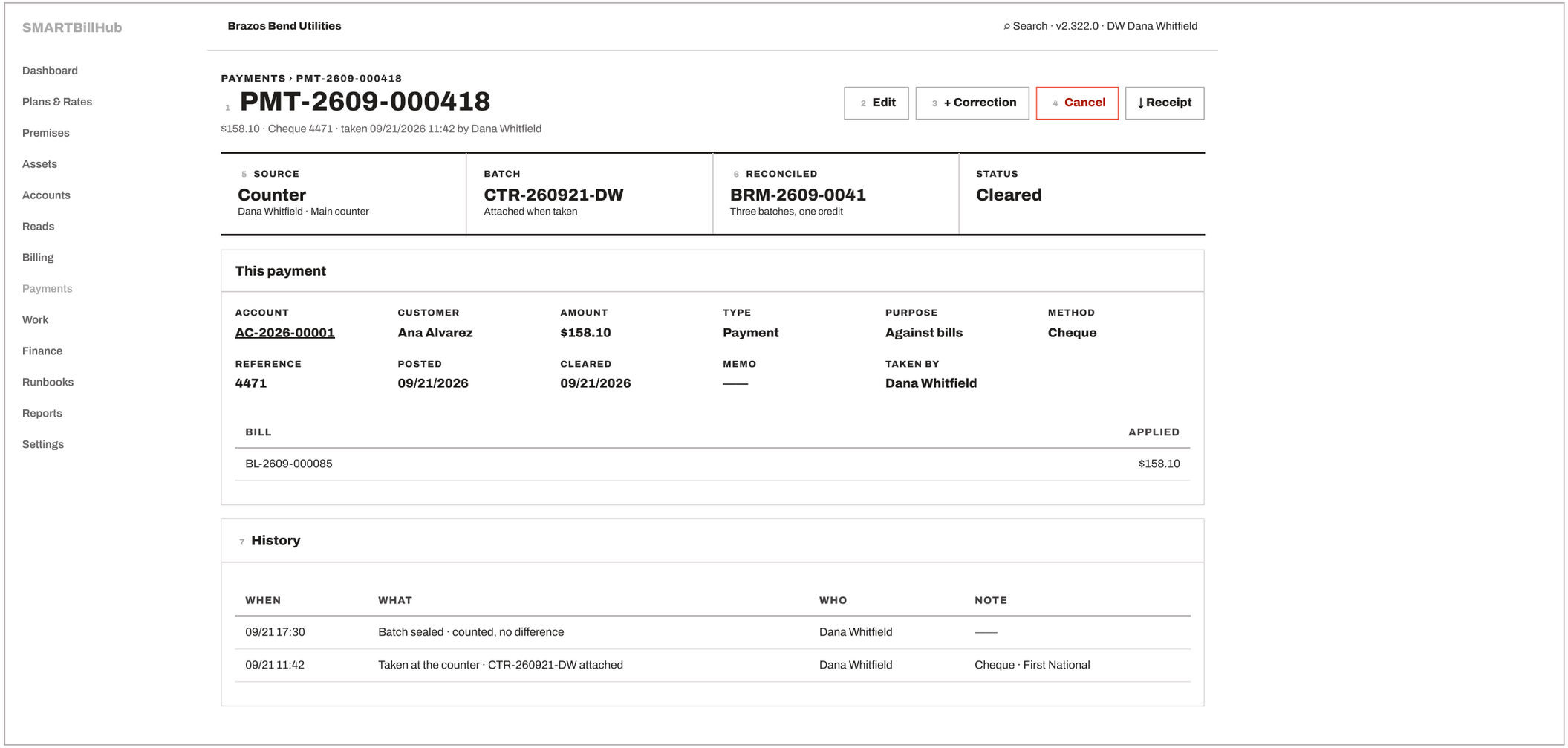
2·2 Payment detail NEW NO ADDRESS TODAY PAY-H · PAY-O

`today: no address — an expanded row · agreed: /:tenant/payments/PMT-2609-000418`

The question: "What did this $158.10 actually pay off — and what can I do about it?"

This screen has no address today. There is no `GET /payments/{id}` and no detail route, so a payment cannot be linked, bookmarked or returned to after a refresh — which is also why the row-action set has no **View** icon. The original ask was *"a clerk clicks PMT-2609-000412 and lands on a page for that payment showing what it paid, bill by bill"*. What was built is an in-place expansion, and the code says so honestly rather than claiming otherwise. **PAY-H.**

DESKTOP · 1440 · THE ELEVEN FACTS AND THE TWO-COLUMN BILL TABLE ARE READ FROM THE COMPONENT · THE HEADER STRIP, THE ACTIONS AND THE HISTORY ARE THE AGREED ADDITIONS



① **A payment has an address at last** · GET /payments/{id} · NEW READ — closes **PAY-H** ② **Edit** · writes IN PLACE, and only the three columns hard rule 4 allows · EXISTING — step ③a ③ **+ Correction** · cannot touch this payment · it **creates a row**, so B14 **4** gives it **+ Noun** · step ③c ④ **Cancel** · danger tier · two behaviours, chosen by status · NEW — NO SUCH PATH EXISTS ⑤ **Source** · Counter, Digital or Post, and who or which provider · NEW ⑥ **Where the bank matched it** · the many-to-one match set · NEW — replaces the deposit batch ⑦ **History** · read from audit_log · NEW READ — and it is what makes ④ honest

For	Answering "what did this money pay off, and what can I do about it" in one place.
How you get here	Today: expanding a row. **1 door, and no address.** Agreed: the 👁 row action, and a link anyone can send.
Fields	**Eleven facts, in this order, read from the JSX:** Account (a link) · Customer · Amount · Status · Type · Purpose · Method · Reference · Posted · Cleared · Memo.

	Then a table with **exactly two columns**: Bill and Applied. **All eleven stay.**
TODAY **What is true today**	The facts and the bill table. **Not** the address, the header strip, the four actions or the history.
States	**Empty** a payment that met no bill draws **a named absence**, not an empty table — "a carried credit, or money that has not cleared" is a real outcome, said in words **Domain** a null clearing date renders *"Not cleared yet"*, **not** the dash a genuinely absent field draws — `processing` is a state, not a missing value **Domain** a released allocation appends *released by correction* or *released by reversal*
Words	Absences are distinguished rather than merged: *No contact recorded* · *No instrument* · *Not cleared yet* · *No account number* — four different dashes for four different facts.
Walked	**No** — needs a payment row. Drawn from the render tree.
Rules	B14 **4** · **6** (④ is danger tier) · **31** · **33** · hard rule 4

> TODAY **② and ③ look like one thing and are two, and hard rule 4 is what splits them**
>
> **Edit** writes in place, and only ever to `confirmation`, `memo` and `method_id` — the entire narrowing granted on 2026-09-02 (*"(a) Allow those fields to change"*). **+ Correction** cannot touch the payment at all: an amount that is wrong is fixed by a **new row**. **Two controls, because they are two rules**, and a clerk shown one control for both will reach for the wrong one.

2·2r Row panel REMOVE S33 · PAY-H

`renderExpanded → PaymentDetailPanel, paymentDetail.tsx:93 — no address`

The question: "What did this payment pay off?" — answered in place, and only in place.

> TASK **This is the one thing being removed, and it needs saying out loud**
>
> Today a payment opens as an **expansion inside the list row**. It has no address, so it cannot be linked, bookmarked or returned to after a refresh — which is also why the row actions have no **View** icon. It goes when **2·2** lands.
>
> **Not before.** Removing it first would take away the only way to see what a payment paid off. And leaving both would give the product two answers to one question, which is how a figure starts to drift. **One commit: the page arrives, the panel goes, the 👁 icon appears.**
>
> Its eleven facts and its two-column bill table are **kept exactly** and move to 2·2 — the drawing there is the same content with an address, a header strip and a history panel around it.

3 What you can do to it

Eight acts. Hard rule 4 decides which may write in place — exactly one

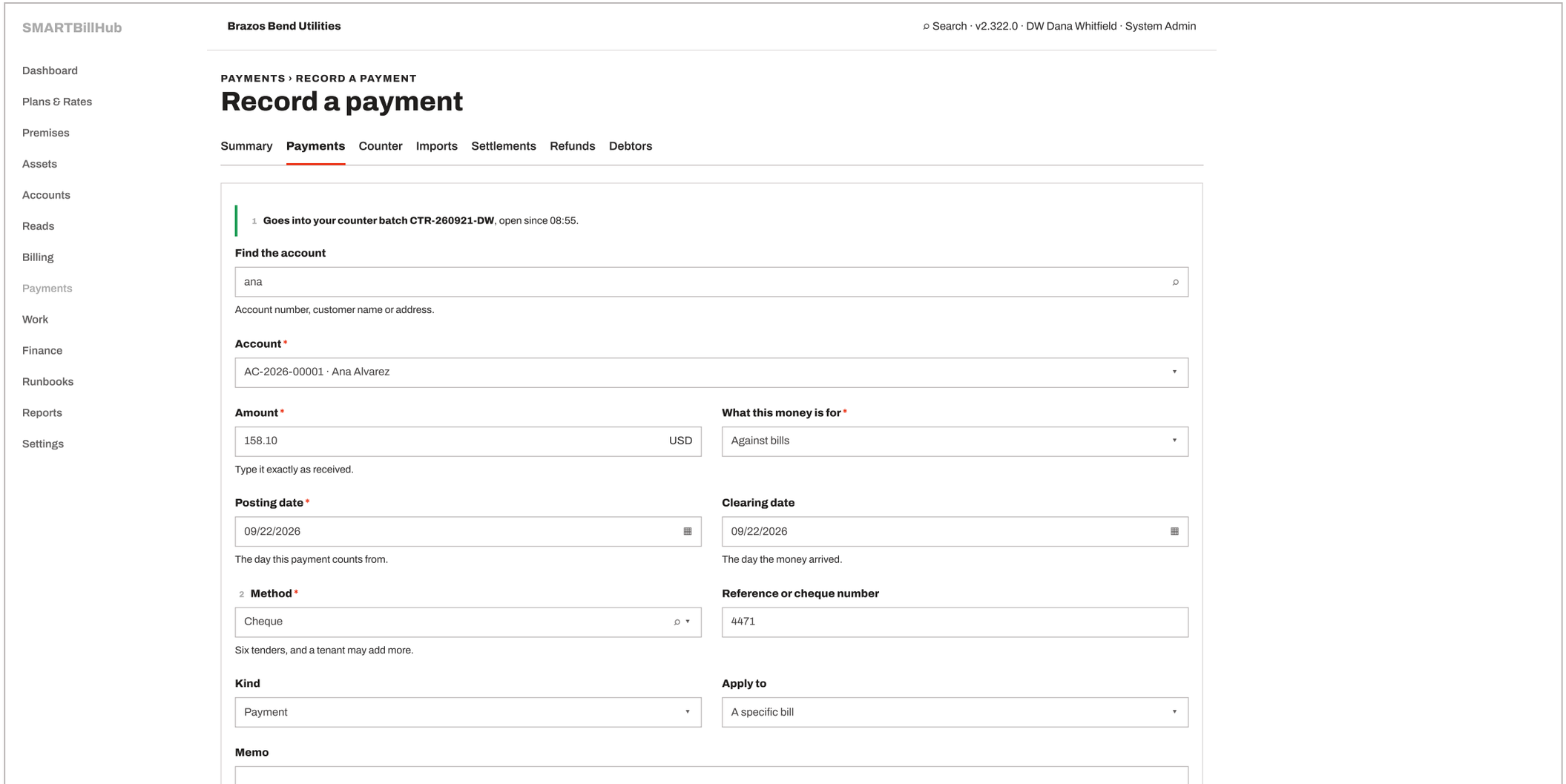
2·3a Record a payment

UPDATE · GAINS THE COUNTER BANNER · PAY-M · PAY-N

`today: a modal · agreed: /:tenant/payments/record/new — a page, not a modal`

The question: "They are at the window with a cheque."

DESKTOP · 1440 · **A PAGE, FULL WIDTH** · NINE FIELDS IN THE ORDER THE JSX EMITS THEM · TWO MARKED CHANGES





① **Which batch this goes into** · a clerk must never wonder · NEW — refuses the form when no batch is open ② **Cheque · Cash · Tap and pay** as shortcuts, with **More ▾** for the rest · CHANGED (was a plain select) drawn as the kit's SegmentedField — one bordered group, not four buttons — value and options both served from GET /payments methods[] — never a literal array ③ **What this will do**, before the commit · the kit's ConsequenceBox · B14 **11** · FIXES A VIOLATION asterisks · the four the schema marks required — account, amount, purpose, posting date · B14 **10**, unchanged from §02 screen 03 everything else · nine fields, their order, labels, helpers and the open-bill picker · UNCHANGED from §02 screen 03

② **keeps the served list.** Promoting three tenders to buttons must not hard-code them: the method vocabulary comes from the same response that renders the list, which is what stops the form offering a method the ledger has retired (BR-PAY-026). **The three shortcuts are a display choice over a served list**, and *More ▾* holds whatever else the tenant has configured.

The posting date still defaults to the UTC day — PAY-G is unchanged by this architecture and is if anything more urgent under it, because a counter batch is a *shift*, and a shift that ends after 17:00 local would stamp its payments into tomorrow.

③ **is not a new idea — it is the kit's own component, on the one module that never called it.** B14 rule 11 opens on the word *money*: *"any form writing money, service state or legal notices shows a 'what this will do' box before its commit button."* `ConsequenceBox` is that box, and it is rendered at **52 non-test call sites across seventeen areas** — assets, metering, customers, work, plans, and `portal/payments/PayNow.tsx`, the screen where a *customer* pays. In `features/payments/`, where a clerk writes to the ledger, it is rendered **zero** times. **So the customer-facing payment screen obeys rule 11 and the staff one does not**, on the same money. Filed against §02 screens 03, 05 and 06 in §05.

TASK **Why this one is a page and the other seven are modals**

It is **nine fields, an open-bill picker and an allocation table** — the largest write surface in the module, and the one a counter clerk uses dozens of times a shift. A modal cannot be linked, bookmarked or returned to after a refresh, and it has no page-head, so B14 rule 2's kicker, title and right-aligned actions have nowhere to go.

The split is by size, not by taste: the one long form is a page; Edit, Correction, Cancel, Reverse, Move, Reapply and Resend are short and confirmatory, so they stay modals over the list you came from. **It needs a route** — there is none today, because the form has only ever been a modal.

GAP **② was a segmented control until 2026-09-22, and that was a rule 8 breach wearing a nicer coat**

Promoting the three counter tenders to buttons read well and broke the rule underneath: **a list of five or more options is searchable**, and a tenant is seeded **six** methods and may add more. A segmented control cannot type-filter and grows sideways with every tender a utility configures.

The served list is what matters either way. The method vocabulary comes from the same response that renders the list, which is what stops the form offering a tender the ledger has retired — and what stops a hardcoded three from hiding the other three.

2·3b Edit the reference

UPDATE · THE ONLY IN-PLACE WRITE HARD RULE 4 ALLOWS

`no address — CorrectReferenceModal, PaymentsPage.tsx:2048 → POST /:tenant/payments/{id}/correct-reference`

Staff. Drawn from the component's render tree.

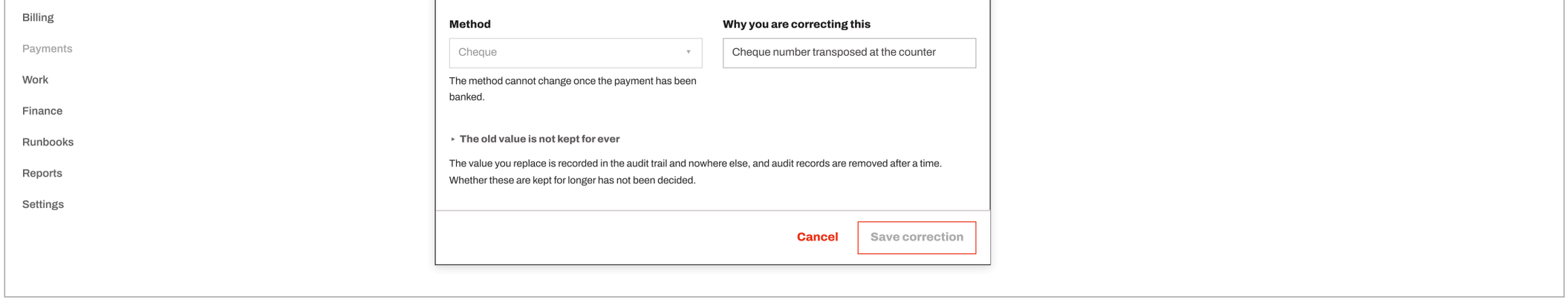
The question: "I typed the cheque number wrong. Can I fix it without undoing the payment?"

RULED **This is the best-built thing in the module, and it is worth saying so**

CLAUDE.md hard rule 4 freezes payments against UPDATE, narrowed by PO ruling of 2026-09-02 for exactly three columns. **The implementation matches the ruling exactly, at both layers**, and nothing here is a convention a caller could step around.

DESKTOP · 1440 · **FOUR FIELDS IN THE ORDER THE JSX EMITS THEM, PAYMENTSPAGE.TSX:2096-2140**





For	Fixing a mistyped cheque number, a memo, or the tender — and nothing else, ever.
How you get here	The ✎ row action on screen 02. **1 door.**
Primary action	**Save correction.** Not danger tier — it moves no money by construction.
Fields	**Four, in this order:** Reference · Memo · Method · Why you are correcting this. Then the retention disclosure. **Exactly the three writable columns plus the required note** — the form cannot express anything the allow list does not permit.
Write surfaces	**Edit / update** → `POST /payments/{id}/correct-reference`, `amendments_api.py:106`, mounted from `config/urls.py:359`. Permission `AnyTenantRole` — it mirrors who records the payment. Contrast `POST /payments/{id}/reverse`, which is `SystemAdminOnly`, correctly, because that one moves money.
The allow list	**Exactly three columns, at both layers.** **Service:** `amendments.py:389-397` loops over exactly `confirmation`, `memo`, `method_id`, saved with a named `update_fields` (`:402`) so a column added later cannot be written without being named. **Database:** the `payments_payment_immutable()` trigger in `migrations/0014` extends two arrays by exactly those three names (`:116-117`, `:122-123`) and subtracts them from a whole-row jsonb comparison — **so any future column is frozen by default.** DELETE still raises in every status (`:127-130`).
The banked refusal	**Implemented twice, deliberately.** Python at `amendments.py:288-322`, invoked at `:384` — *before* the tender is resolved and before the audit shape is composed. Database at `migrations/0014:141-146`, checked in every status. The migration says why both: *"a Python check is not a guarantee when a repair script can open a cursor"*. "Banked" means `deposit_batch_id IS NOT NULL` at both layers — a cleared-but-unbatched payment's tender **is** still correctable.
Walked	**No** — needs a payment row. Drawn from the render tree.
States	**Domain** the Method field renders disabled once the payment is banked, with its reason beneath it **Failed** the server's own `message_key` against the field **Refused** a correction with no note is refused (`amendments.py:367-373`); a money field is refused **by name** (`:251-273`) before anything is read; an unknown field is refused (`:274-285`), so the route cannot quietly grow a fourth

Words	The refusal names the batch **code** (`DEP-260921-CHK`), never the uuid — B14 rule 31, correctly applied. The two layers' messages are worded differently on purpose: the trigger's is for whoever bypassed the service, the service's names a batch a clerk can act on.
Rules	**Hard rule 4**, narrowed · B14 **11** *X* (no consequence preview — §05) · **31** · **33** · hard rule 7 (audit)

OPEN **Compliance pass · the open question is surfaced honestly**

CLAUDE.md states the replaced value then lives **only** in `audit_log`, which is pruned; whether such a row is exempt is **OQ-1340, still open**, and until it is answered *no document, screen or docstring may describe this path as a permanent record of the old value.*

The screen obeys it. `CorrectionRetentionNote` (`PaymentsPage.tsx:2209`) renders, in all eleven languages: *"The value you replace is recorded in the audit trail and nowhere else, and audit records are removed after a time. Whether these are kept for longer has not been decided."* The server carries the same fact on every successful response as `unavailable[]` (`amendments.py:417`) rather than leaving it in a docstring. The audit row is composed **before** the update, so `before` holds the genuine old value, and written in the same transaction as the save — **a corrected row without its audit row is unreachable. No commitment is needed here. This is what compliance looks like.**

OPEN **Finding · the file that implements this tells a reader it is stubbed**

`amendments_api.py` contradicts itself on a money path. Its **module** docstring (`:37-48`) says *"⚠ THE WRITE IS LIVE SINCE 2026-09-02 … OQ-1322 CLOSED … SO THE SCREEN LANE MAY NOW ENABLE THE CONTROL."* Its **handler** docstring (`:122-125`) says *"⚠ A REQUEST THAT PASSES EVERY REFUSAL MEETS THE STUB and raises … nothing corrects a reference today."* The code at `amendments.py:399-403` is a real `payment.save()` inside a transaction. **There is no stub.**

The handler docstring cites *"this module's docstring"* as its authority, and that docstring now says the opposite. Worse, the lifecycle block at `:159-161` instructs the screen lane to *render the control DISABLED with its reason while the write is stubbed*. **Had a lane obeyed it, the product would have shipped a dead control over a working endpoint** — hard-won law 2's mirror, the exact failure that left the customer portal with two working doors out of eight. The front end did not obey it, so this was luck rather than correctness. **PAY-E**.

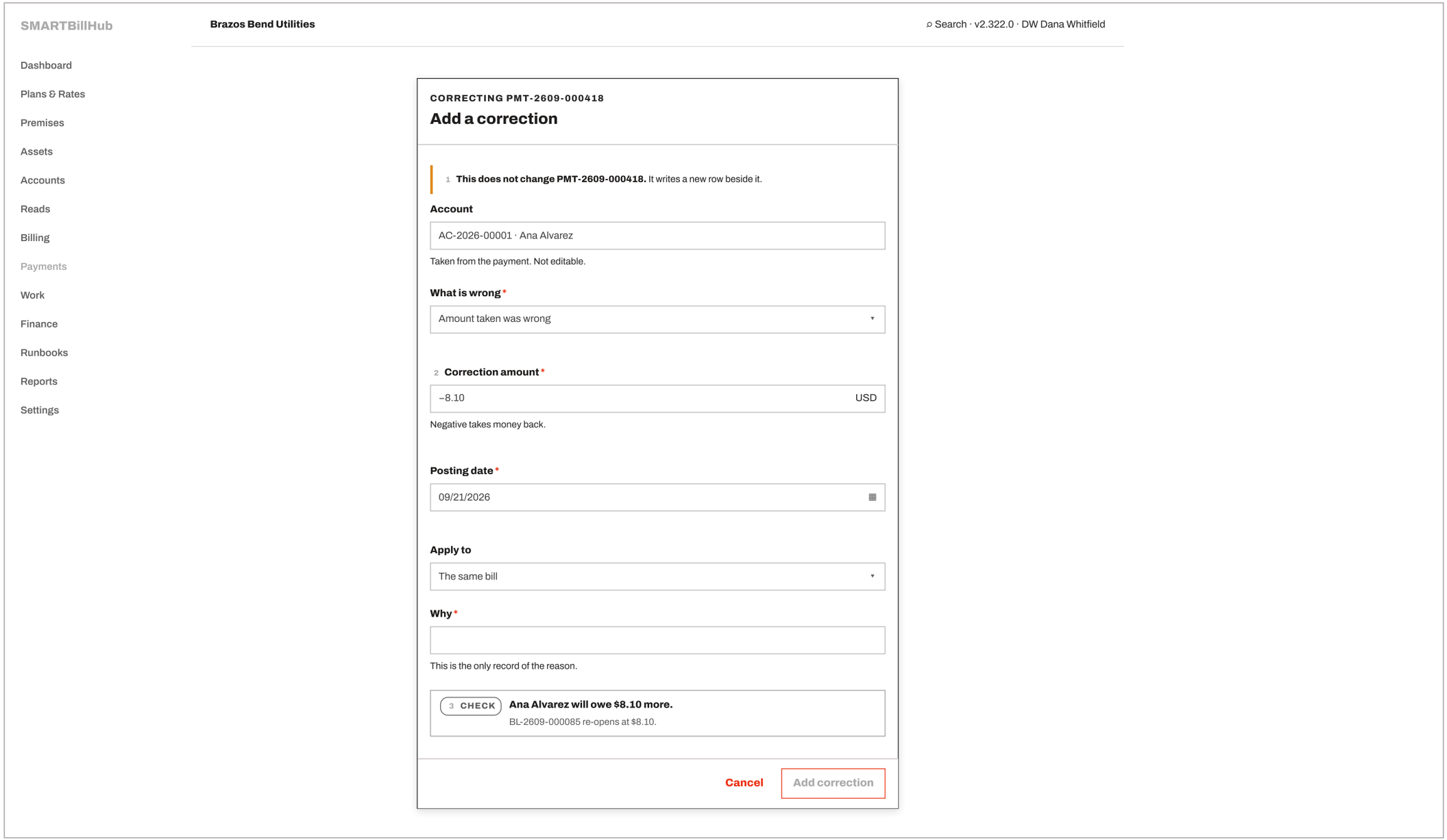
2·3c Add a correction NEW PAY-P

`proposed — POST /:tenant/accounts/{id}/payments with type=adjustment`

The question: "The amount is wrong and I cannot edit it. Now what?"

Half of this exists and it is the half nobody can find. An adjustment is already a real thing: a payment whose *Kind* is `adjustment`, written down the same path as any other payment. So the engine, the ledger posting and the audit row are all there. **What is missing is a way in from the payment being corrected** — today a clerk must go to the record form, find the account again, and know to change a field called Kind.

DESKTOP · 1440 · PROPOSAL — THE RECORD FORM, PRE-FILLED FROM THE PAYMENT IT CORRECTS



① **Said before anything else** · the single most misunderstood rule in the module · B14 **33** consequence · NEW ② **Signed** · a correction can go either way, and the sign is the whole meaning · NEW ③ **What this will do**, in the customer's terms · B14 **11** · NEW the account, the original and the link between them are all filled in — a clerk correcting a payment should never have to find it again

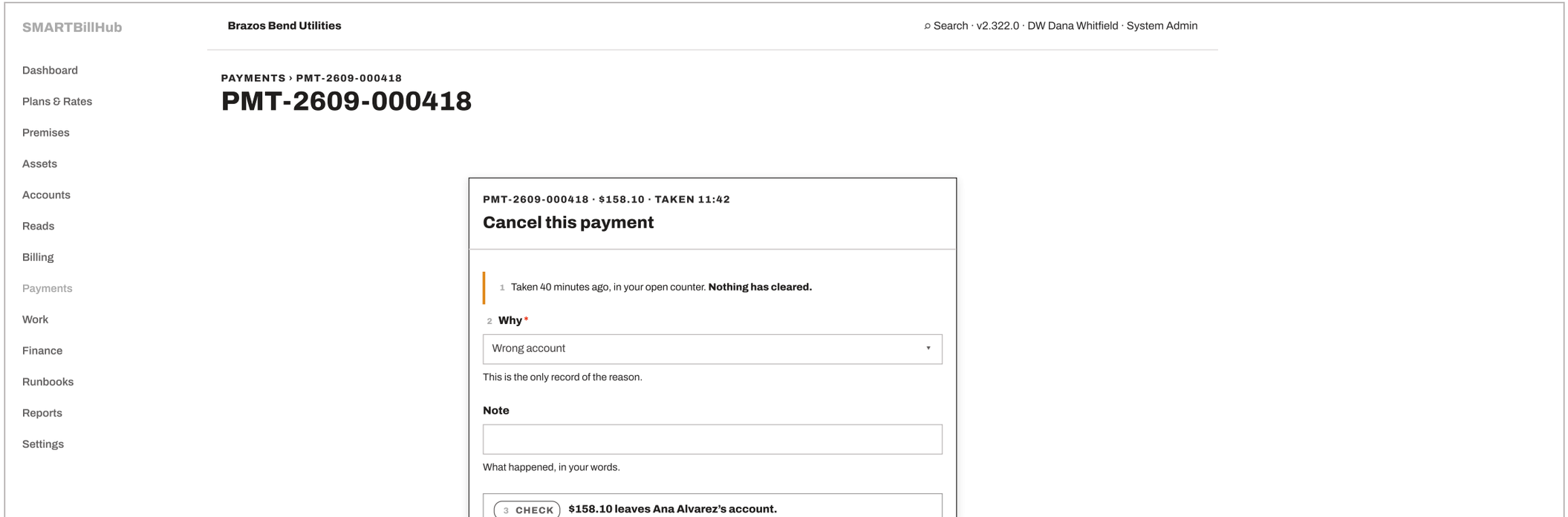
For	Fixing an amount, a date or an allocation that a reference edit cannot reach.
How you get here	**+ Correction** on the payment detail. **1 door — and it does not exist today.**
Write surfaces	**Add** → the existing record path with `type=adjustment`. **No new endpoint is needed**, which is what makes this the cheapest of the three missing actions.
TASK **What has to change**	A control on the detail · a modal that pre-fills from the payment · a link column so a correction knows which payment it corrects — `payments` has `reverses_payment` for a reversal and **no equivalent for an adjustment**, so today the two rows are related only in a clerk's memory.
Rules	B14 **4** (**+ Noun**, it creates) · **10** · **11** · **33** · hard rule 4 (satisfied, not circumvented) · hard rules 5 and 7

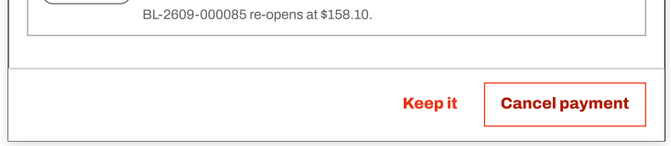
2·3d Cancel NEW S35 · PAY-P

`proposed POST /:tenant/payments/{id}/cancel`

The question: "I just took that from the wrong person. Undo it."

DESKTOP · 1440 · PROPOSAL · THE ONLY GENUINELY ABSENT LOGIC IN THE MODULE





① **Why this is cancellable** · state and age, said first — the control behaves differently by status · B14 **33** · NEW ② **Why** · required · B14 **10** · NEW ③ **What it will do** · in the customer's terms, not the ledger's · B14 **11** · NEW

For	The clerk who has just made a mistake.
Doors	**Cancel** on 2·2, and the row overflow on 2·1. **2 doors.**
Primary action	**Cancel payment**, danger tier.
TODAY **Today**	**Nothing.** The five payment states are processing, cleared, failed, reversed, posted — **there is no cancelled**. The reversal code refuses an uncleared payment with the words *"a payment that has not cleared is cancelled instead"*, pointing at a state that does not exist and an endpoint nobody wrote.
TASK **To build**	A sixth state · `POST /payments/{id}/cancel` · the rule for when it is allowed · the reversing pair.
Rules	B14 **6** · **10** · **11** · **33** · hard rules 4, 5, 7

> TODAY **Cancel and Reverse are the same word to a clerk and two different acts to the ledger**
>
> **Cancel** is for money that never cleared — the mistake is minutes old, in an open batch, and nothing has reached the bank. **Reverse** is for money that arrived and went back: a bounce, a chargeback. Hard rule 4 means neither deletes anything; both write new rows.
>
> **So the clerk should see one control and the product should choose.** Drawn that way on 2·2: one **Cancel** in the danger tier, behaving by status, with the history panel making whichever happened legible afterwards.

2·3e Reverse this payment

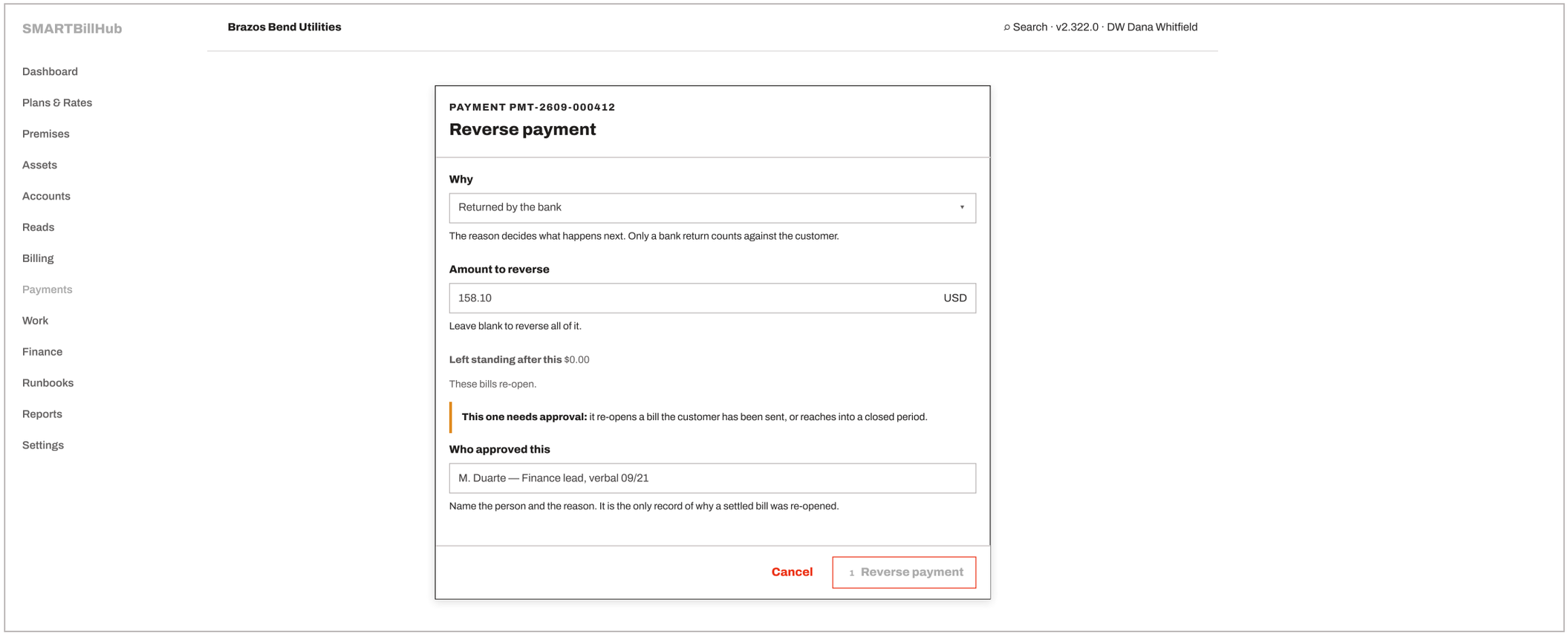
UPDATE · COMMITS THROUGH THE WRONG TIER

`no address — ReversePaymentModal, PaymentsPage.tsx:1574 → POST /:tenant/payments/{id}/reverse`

Staff, and narrower than the rest — `SystemAdminOnly`.

The question: "The cheque bounced. Undo it — but leave the record straight."

DESKTOP · 1440 · **FIELDS AND CONDITIONAL NOTES READ FROM PAYMENTSPAGE.TSX:1674-1800**



For	Undoing money that did not arrive, without breaking financial immutability.
How you get here	The ↺ row action on screen 02, which carries a **per-row refusal reason** — an already-reversed payment shows the icon disabled with why. **1 door.**
Primary action	**Reverse payment** ① — and it renders in the **filled primary**, not the danger tier. **B14 rule 6 names *reverse* by example** and this is the commit it names. See the finding below; §05 logs it.
Fields	**Two always, plus conditionals, in this order:** Why (select of four reasons — *Returned by the bank · Chargeback · Customer dispute · Recorded wrongly*) · Amount to reverse. Conditional blocks render between and after them: a staff-correction note, an outside-the-window warning, a payment-plan summary, a credit-goes-negative warning, and — when the reversal re-opens an issued bill — an **approval** field. **There is no date field and no free note field**; an earlier draft of this document drew both, wrongly.
Write surfaces	**Update** → `POST /payments/{id}/reverse`, `reversal_api.py:409`, `SystemAdminOnly`. **It writes a new row and does not touch the original** — hard rule 4 satisfied rather than circumvented.

Walked	**No** — needs a payment row. Drawn from the render tree.
States	**Loading** *"Working out what is left."* while the remaining balance resolves **Domain** partial reversal is supported and the amount helper says so **Domain** *"This account will owe you money once this is done"* when the credit behind it has already been spent **Result** the panel uses `t(…, {defaultValue})` — **the safe pattern screen 02's record panel is missing**
Words	The reason list is written for a clerk, not for the ledger: *Returned by the bank* on this screen where the ledger's own vocabulary says *Not enough funds*. Two registers for two audiences, deliberately kept apart.
Rules	B14 **6** (**broken**, ①) · **11** (broken — no consequence preview) · **33** · hard rules 4, 5, 7

GAP **① The row action is danger-tiered. The button that actually reverses the money is not.**

Three files decide it and none of them is wrong on its own. `WorkingButton` takes an optional `variant`, and **absent means `primary`** — its own docstring says so, and says why (so the fourteen existing call sites did not move when the prop was added). `FormModal` passes `variant={props.submitVariant}` straight through. **And all three payments modals pass no `submitVariant` at all** — the string does not occur in `PaymentsPage.tsx`. Across the whole front end it occurs at **three** call sites: `MoveUnitModal`, `serviceRequestDecisions` and `MarkAsCutDialog`. None is in payments.

So the ↺ that OPENS this dialog is drawn in the danger tier — `PaymentsPage.tsx:513` passes `danger` to the row action — **and the button that commits the reversal is styled exactly like Save.** The warning is on the door and not on the act.

B14 predicted this hole in its own status column. Rule 6 is enforced on rendered output for a `WizardFrame` commit and for a `ConfirmDialog` commit, and the row ends: *"⚠ Still unchecked: a commit button that composes NEITHER shared component."* A `FormModal` composes neither. **This is not a rule nobody got to — it is the gap the rule's own record names**, and payments is the module sitting in it.

An earlier draft of this document drew this button in the danger tier, because the rule says it should be. That is the failure this section exists to prevent: an as-built drawing that shows the rule instead of the code launders a violation into something that looks shipped.

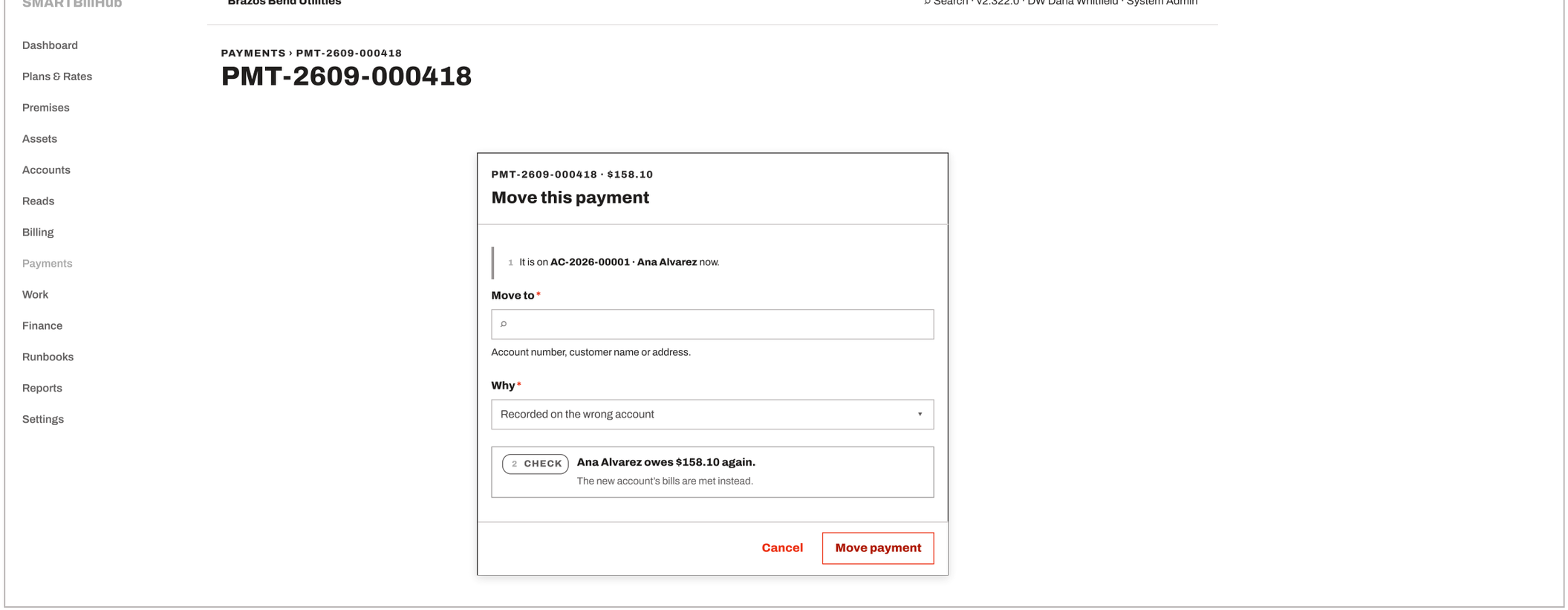
2·3f Move NEW S35 · PAY-P

`POST /:tenant/payments/{id}/move — served, and called by nothing`

The question: "This went on the wrong account."

DESKTOP · 1440 · THE ENGINE EXISTS; THIS IS THE SCREEN IT NEVER GOT





① **Where it is now** · because the whole act is a comparison · NEW ② **Both sides** · a move changes two accounts and the screen says both · B14 **11** · NEW

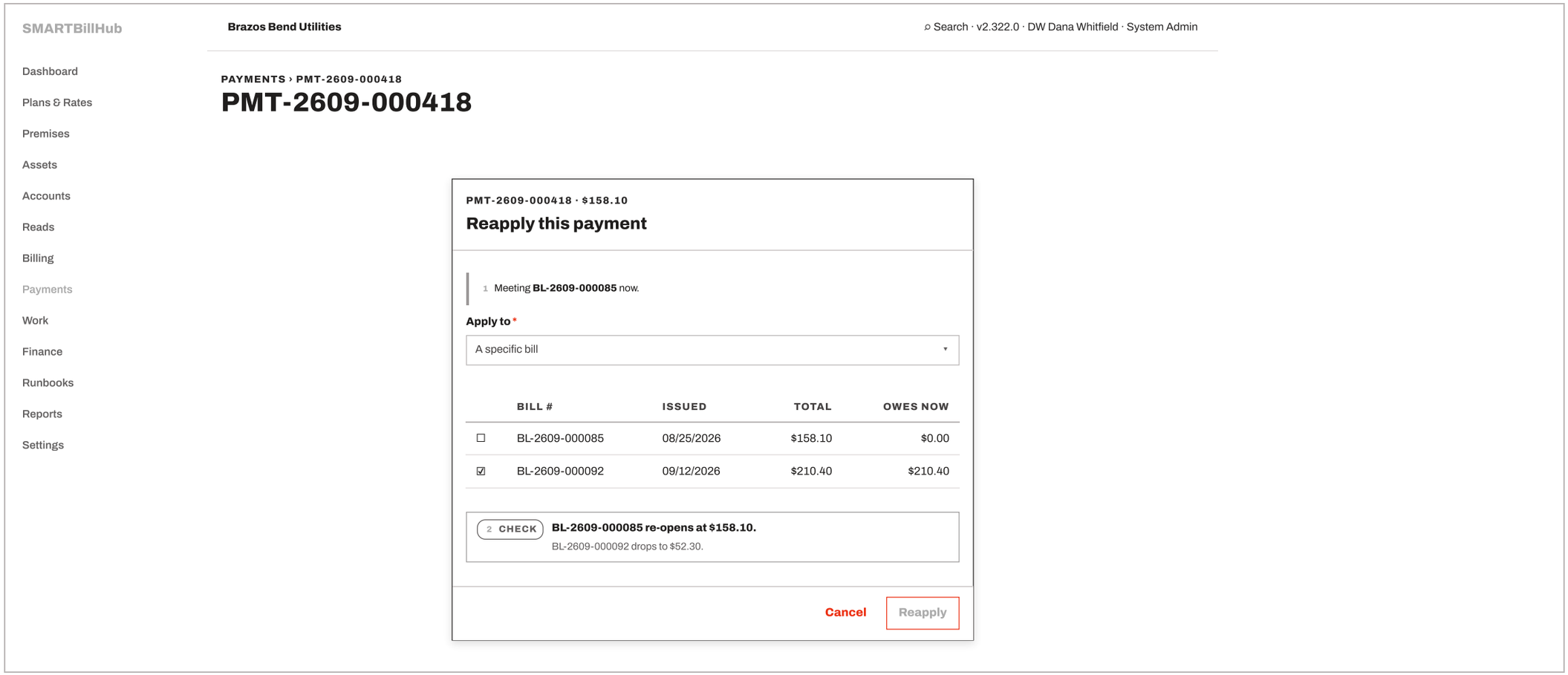
For	A clerk correcting an account, not an amount.
Doors	The row overflow on 2·1, and 2·2. **2 doors — there are none today.**
Write surfaces	**Update** → `POST /payments/{id}/move`, with `/move-preview` behind the consequence box. **Both served.**
TODAY **Today**	**Served, and unreachable.** `movePayment` and `movePreview` are written in the API client and called by nothing.
TASK **To build**	The screen only.
Rules	B14 **6** · **10** · **11** · hard rule 4 — it is a reversal and a new payment, never an edit

2·3g **Reapply** NEW S35 · PAY-E

`POST /:tenant/payments/{id}/reapply — served, and called by nothing`

The question: "It paid the wrong bill."

DESKTOP · 1440 · THE ENGINE EXISTS; THE BULK BUTTON SAYS IT DOES NOT



① **What it meets now** · the before, because a reapply is a swap · NEW ② **Both bills move** · stated as two facts, six words each · B14 **11 · 33** · NEW

For	A clerk moving money between a customer's own bills.
Doors	The row overflow on 2·1, and 2·2. **2 doors — there are none today.**
Write surfaces	**Update** → `POST /payments/{id}/reapply`, with `/reapply-plan` behind the preview. **Both served.**

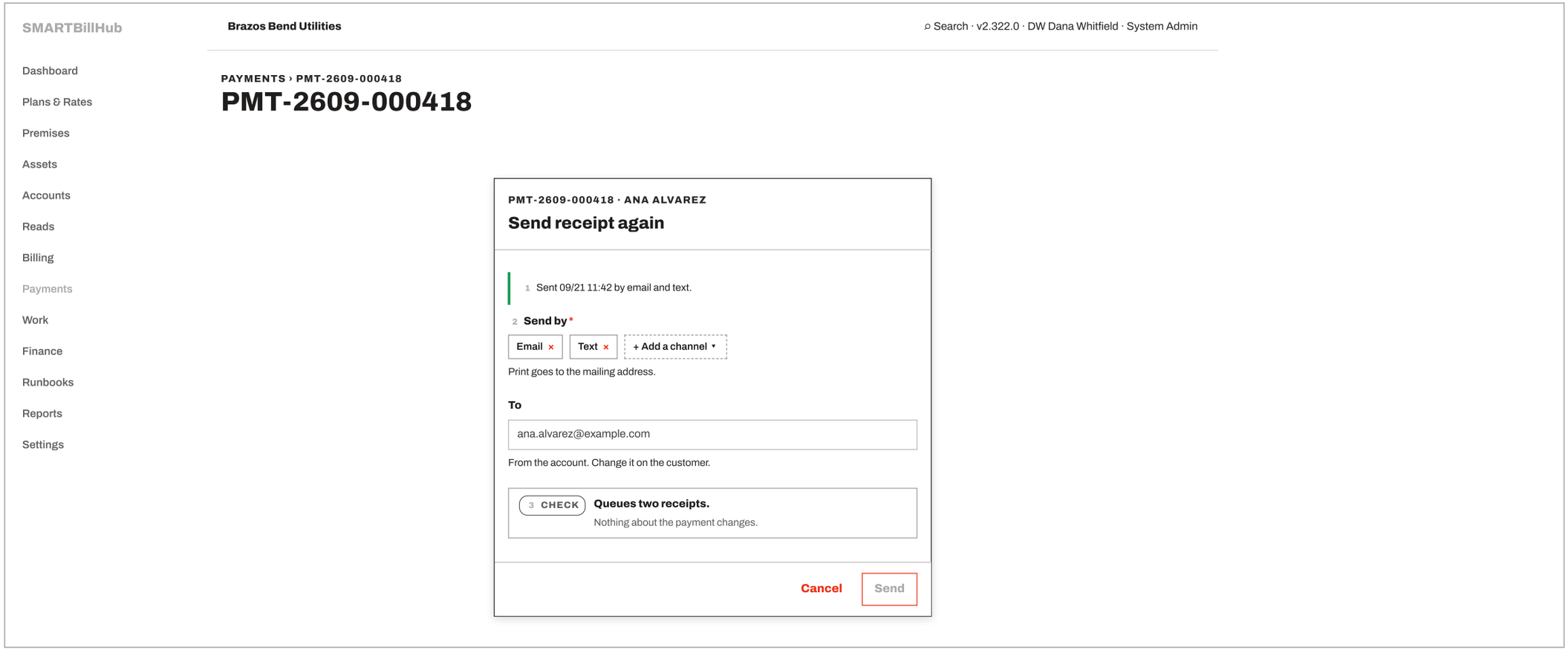
TODAY Today	**Served, unreachable — and the bulk bar tells a clerk it does not exist.**
TASK **To build**	The screen, and delete the refusal on 2·1b.
Rules	B14 **11** · **33** · hard rules 4 and 5

2·3h Resend NEW S35 · PAY-E

`proposed POST /:tenant/payments/{id}/receipt — re-queues an existing document`

The question: "They lost the receipt and want another."

DESKTOP · 1440 · SMALL, BECAUSE THE WHOLE MACHINE ALREADY EXISTS · A RECEIPT MAY GO BY MORE THAN ONE CHANNEL AT ONCE



① **It already went** — so the clerk knows this is a repeat, not a first send. NEW ② **Three channels** — the ones the platform actually has for this event. EXISTS ③ **It**

① **It already went** · so the clerk knows this is a repeat, not a first send · NEW ② **Three channels** · the ones the platform actually has for this event · EXISTS ③ **It touches no money** · the reassurance this screen exists to give · B14 **11** · NEW

For	A clerk answering “can you send that again?”
Doors	↓ **Receipt** on 2·2, and the row overflow. **2 doors.**
TODAY **Today**	**The receipt itself is live** — every recorded payment queues one on email and text, with a printed floor so a utility that switches every electronic channel off still gets paper. **What is missing is re-sending.**
TASK **To build**	One endpoint that re-queues the existing document, and this screen.
Rules	B14 **11** · **33** · hard rule 7

4 What it changes

Every act, and where it lands

Act	What it writes	The ledger	What changes elsewhere
Record	A payment, its allocations, its batch id	A balanced pair the same minute	Bills close · the collection ladder stops · a receipt queues on email, text and paper · the counter total moves
Edit	**In place**, three columns only: cheque number, memo, tender	Nothing	**The old value survives only in the audit log, which is pruned.** Whether such a row is exempt from pruning is unanswered — so no screen may call this a permanent record
Correction	A new payment row of type adjustment	A balanced pair	The bill re-opens or closes further · the customer’s balance moves
Cancel	A reversing row; the original stands	A reversing pair	The bill re-opens · the ladder may restart · the counter total drops
Reverse	A reversing row, all or part, with an approval when it re-opens a sent bill	A reversing pair, plus any bank fee	A bounce counts against the customer and sends a letter; a staff correction does neither
Move	A reversal on one account and a new payment on the other	Two balanced pairs	**Two customers’ balances move**
Reapply	Allocations released and rewritten	No new pair — the money did not move	One bill re-opens, another closes
Resend	A queue row	Nothing	Nothing about the payment

GAP **The row worth reading twice**

Edit is the only act on this page that writes in place, and it is the only narrowing of financial immutability anyone has granted — three columns, through one named endpoint, and the tender is refused once the payment is banked because deposit batches group by day and tender. Everything else here writes a new row. **A build lane that treats any other field as editable is breaking a hard rule, not a convention.**

TAB 3

Counter — open a till, take a shift's money, close and count it

Nothing in this tab exists. There is no session, no float, no owner and no count anywhere in the product. All six screens sit behind one new table,

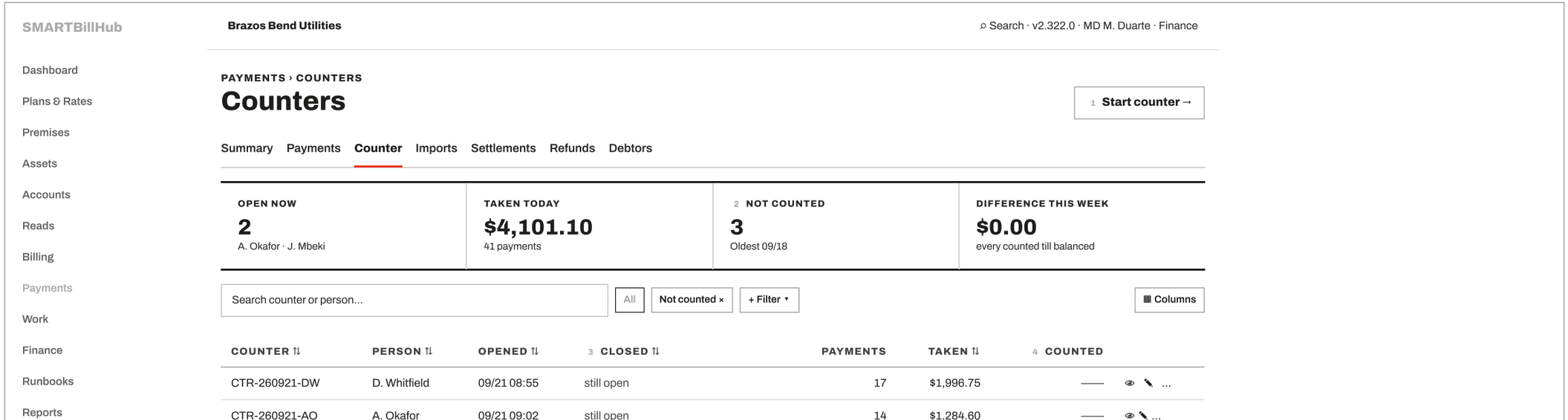
1 The list Every till, open and closed

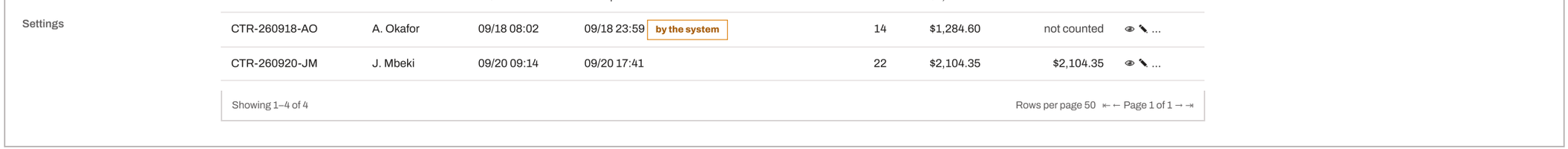
3·1 Counters NEW S-NEW · PAY-N

proposed /:tenant/payments/counter — the supervisor's list

The question: "Whose till is open, and whose did not get counted?"

DESKTOP · 1440 · PROPOSAL · EVERY FIGURE IS AN EXAMPLE





① **Start counter →** · the missing control, on every payments screen · B14 **4** · NEW · opens 3·3a ② **Not counted** · a till the system closed and nobody counted · NEW ③ **by the system** · an auto-close, marked so it can never read as a real close · NEW ④ **Counted** · empty is the finding — the payments are safe, the drawer figure is missing · NEW

For	A supervisor or finance lead checking the day's tills.
Doors	The Counter tab, and the **Start counter** control on every payments screen. **2 doors.**
Primary action	**Start counter** → — it navigates, so B14 rule 4 gives it the arrow.
Table controls	All four, on the server, exactly as the payments list already does them.
Write surfaces	**Detail** → 3·2 · **Add** → 3·3a · **Edit** NOTHING · **Update** → 3·3c, 3·3d
TODAY **Today**	**Nothing.** No counter table, no endpoint, no screen.
TASK **To build**	`counter_sessions` — owner, centre, float, opened_at, closed_at, closed_by, counted totals · `GET /counters`
Rules	B14 **4** · **12–17** · **31** · **33** · **40**

2 One till opened

What this shift has taken so far

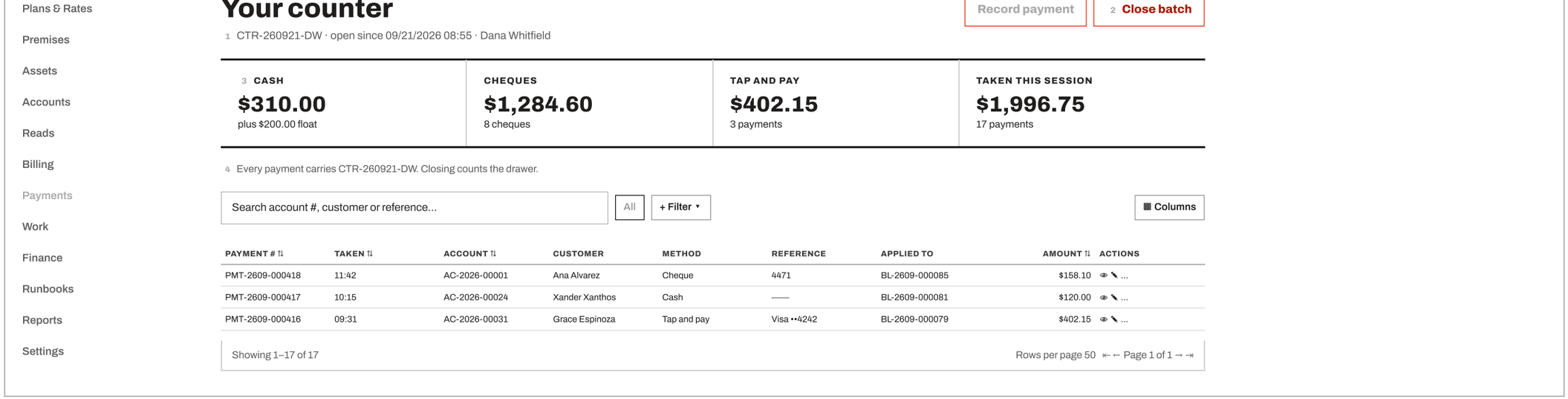
3·2 My counter NEW S-NEW · PAY-N

`proposed /:tenant/payments/counter/CTR-260921-DW`

The question: "What have I taken this shift, and does the drawer match?"

DESKTOP · 1440 · PROPOSAL · THE OPEN STATE





① **Batch id, date and time** · the three the requirement names, and they exist from the first payment · NEW ② **Close batch** · danger tier — it seals the session and its count · B14 **6** · NEW · opens 3·3b ③ **Running split by method** · what a till reconciliation actually needs · NEW READ ④ **When the id attaches** · said plainly, because a clerk must never wonder · B14 **33** · NEW

For	The clerk on the till, during their shift.
Doors	A row on 3·1, and the open-counter control on every payments screen. **2 doors.**
Primary action	**Record payment** — the same form as 2·3a, with the batch already known.
Write surfaces	**Detail** → 2·2 · **Add** → 2·3a · **Update** → 3·3b
TODAY **Today**	**Nothing.**
TASK **To build**	`GET /counters/{id}` and its running totals. The payments list underneath is the existing one, filtered by batch.
Rules	B14 **2** · **6** · **12–17** · **33** · **40**

GAP **This is the one screen with nothing behind it at all**

`CounterReceipt` records *who took a payment and when* — but only for application fees on service requests, one source type, and **no screen opens it**. There is no session object: nothing opens, nothing closes, nothing holds a float. `counter_sessions`, `POST /counters` and `POST /counters/{id}/close` are new work, and they are the gate this whole tab sits behind. **PAY-N.**

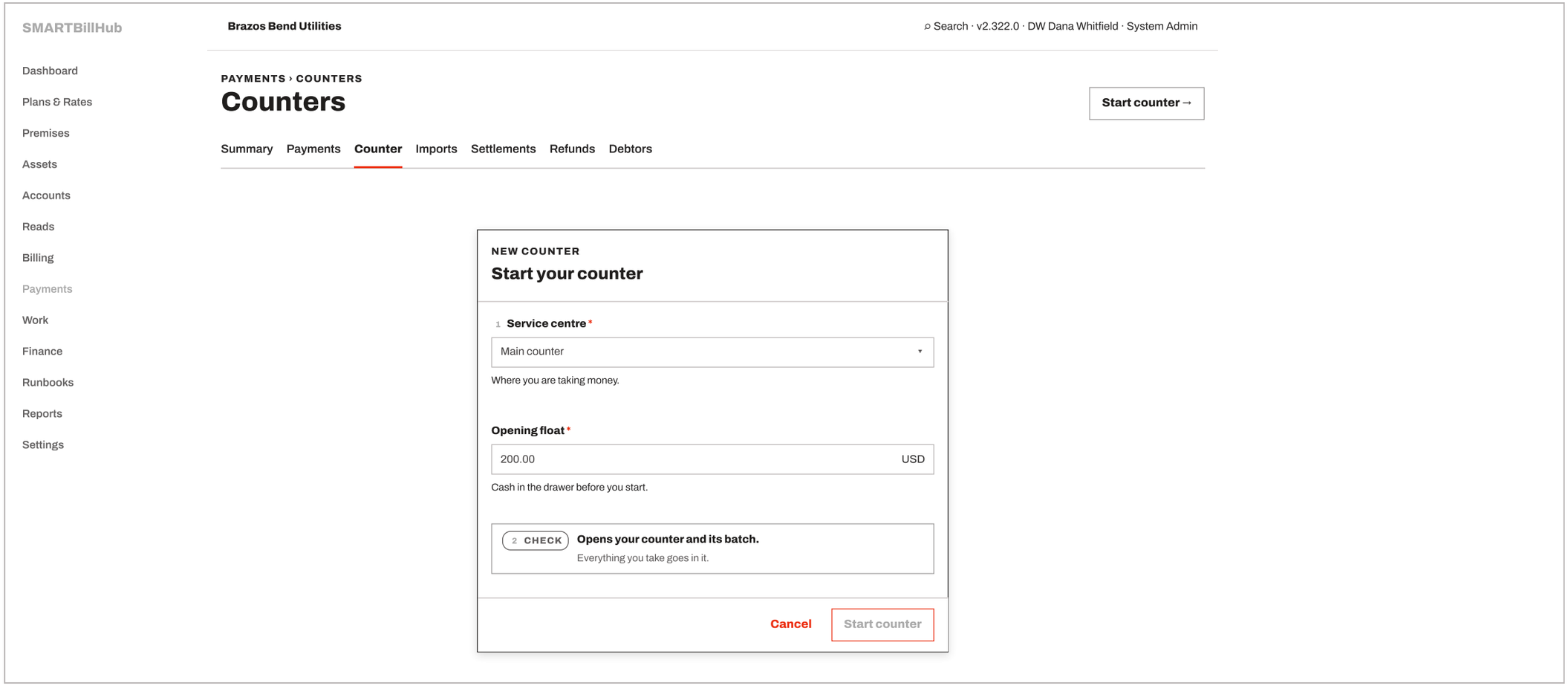
3 What you can do to it — Four acts, and only the first two belong to the clerk

3·3a Start NEW S-NEW · PAY-N

`proposed POST /:tenant/counters — opens the session AND its batch, in one act`

The question: "I am opening the counter for my shift."

DESKTOP · 1440 · PROPOSAL · A MODAL, LIKE EVERY OTHER ACT IN THIS TAB



① **Service centre** · the EXISTING reference-data list — portal-published, already carries which methods a site takes · REUSED · settings/reference-data/service-centres

② **One act, not two** · the batch is created by this button — there is no separate 'start batch' · B14 **11** · RULED 2026-09-21 the commit is POST /counters — **NOT SERVED**

For	A clerk at the start of a shift.
Doors	**Start counter** → on every payments screen. **1 door — and it did not exist.**
Fields	Two. Service centre (required) · Opening float (required).
Write surfaces	**Add** → `POST /counters`. **NOT SERVED.**
`TODAY` **Today**	**Nothing** — and the control to reach it was missing from the whole product.
`TASK` **To build**	The session row, the batch it opens, and a refusal when this user already has one open.
Rules	B14 **4** · **10** · **11** · **33** · **39**

`RULED` **The ruling this screen carries**

2026-09-21: *"When a counter is started a batch should be automatically started with it."* An earlier draft of this document had the batch id attaching at **close**. It does not: the batch exists from the moment the till opens, so **every payment carries its id from the first one**, and closing only seals it and records the count. That removes a window in which a counter payment would have had no batch — the same gap card payments still have, drawn on 2·1 as *Awaiting settlement*.

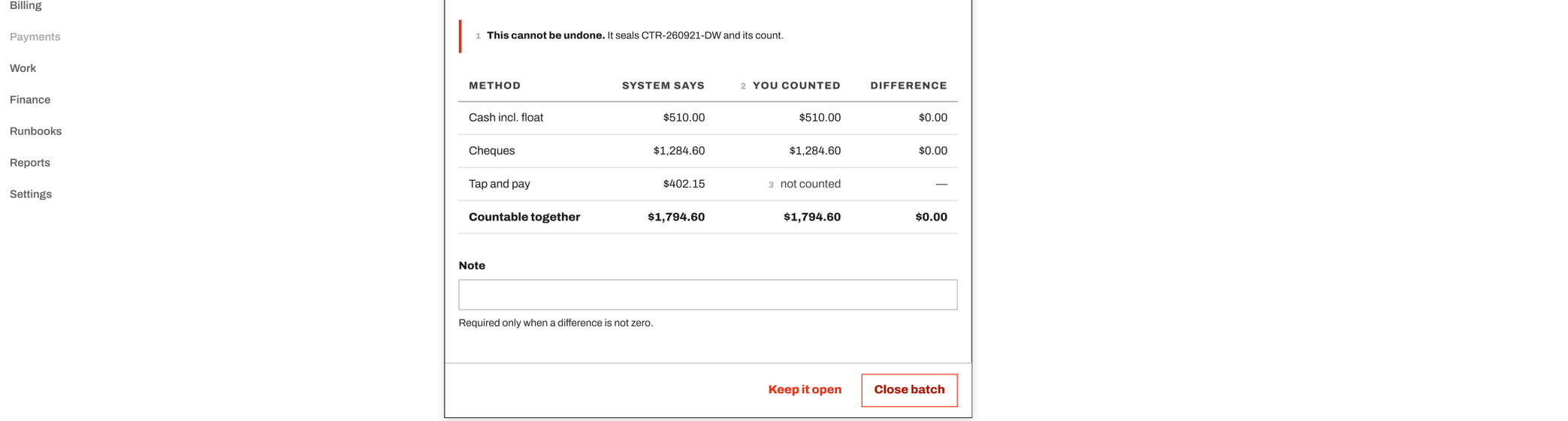
3·3b Close `NEW` `S-NEW · PAY-N`

`proposed POST /:tenant/counters/{id}/close`

The question: "My shift is over — does the drawer match?"

DESKTOP · 1440 · PROPOSAL · A COMPARISON, NEVER A READ-BACK





① **Consequence before an irreversible act** · two sentences, five words each · B14 **33** · NEW ② **You counted** · a till close is a COMPARISON — the clerk types what is in the drawer · NEW ③ **not counted** · you cannot count a card total in a drawer, so the screen does not pretend · NEW

For	The clerk closing their own till.
Doors	**Close batch** on 3·2. **1 door.**
Primary action	**Close batch**, danger tier — B14 rule 6, and it is irreversible.
Fields	A counted figure per countable method, and a note that is required only when a difference is not zero.
Write surfaces	**Update** → `POST /counters/{id}/close`. **NOT SERVED.**
Rules	B14 **6** · **11** · **33** · hard rules 4 and 7

GAP **③ is drawn deliberately, and the float is drawn deliberately too**

A close screen that demands a counted card total trains clerks to type the system figure back at it, which makes the whole reconciliation worthless. Only cash and cheques are countable, and the total row says *Countable together* rather than pretending otherwise.

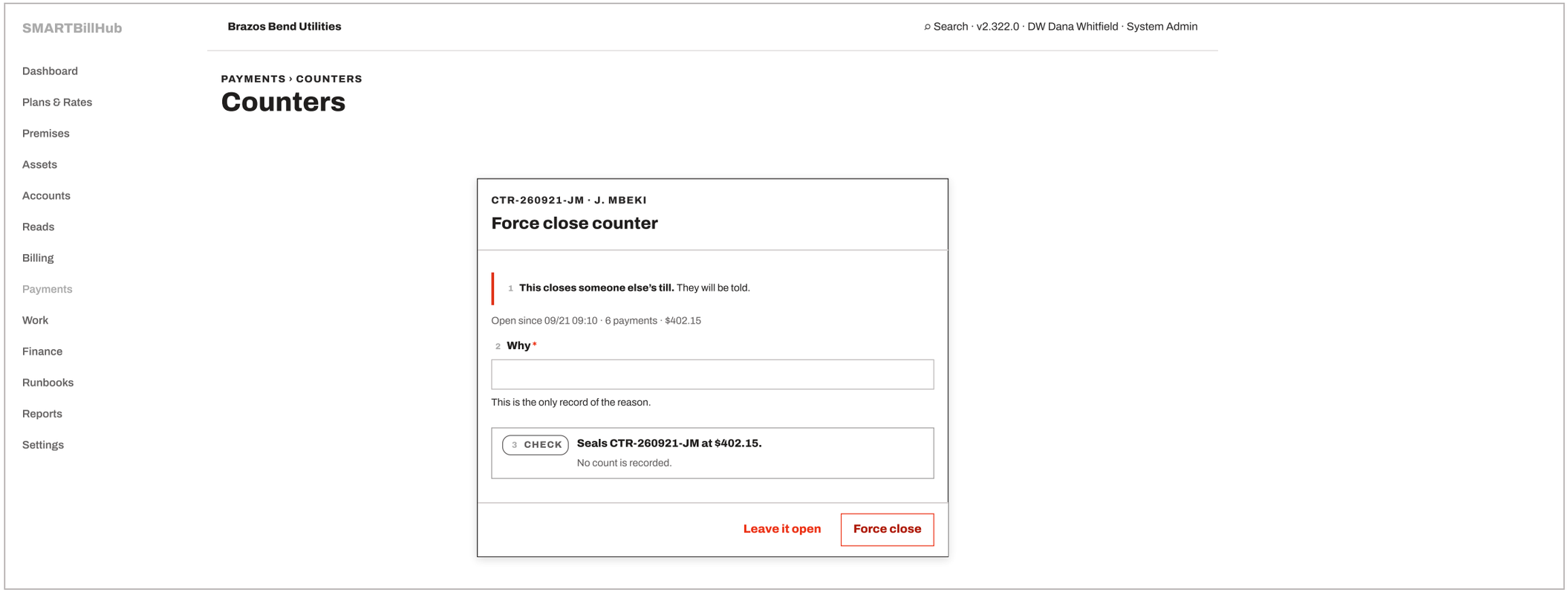
The float sits inside the cash line rather than being netted out. Which is right is a finance question, not a UI one, and it is in the decisions list.

3·3c Force close NEW S-NEW · PAY-N

`proposed POST /:tenant/counters/{id}/force-close — supervisor only`

The question: "Someone went home without closing their till."

DESKTOP · 1440 · PROPOSAL



① **Whose till it is** · said first, because this is an act on another person's work · B14 **33** · NEW ② **Why** · required — a force close with no reason is indistinguishable from a mistake · NEW ③ **Sealed, not counted** · the two are different and the screen keeps them apart · B14 **11** · NEW

For	A supervisor or finance lead.
Doors	A row action on 3·1. **1 door.**

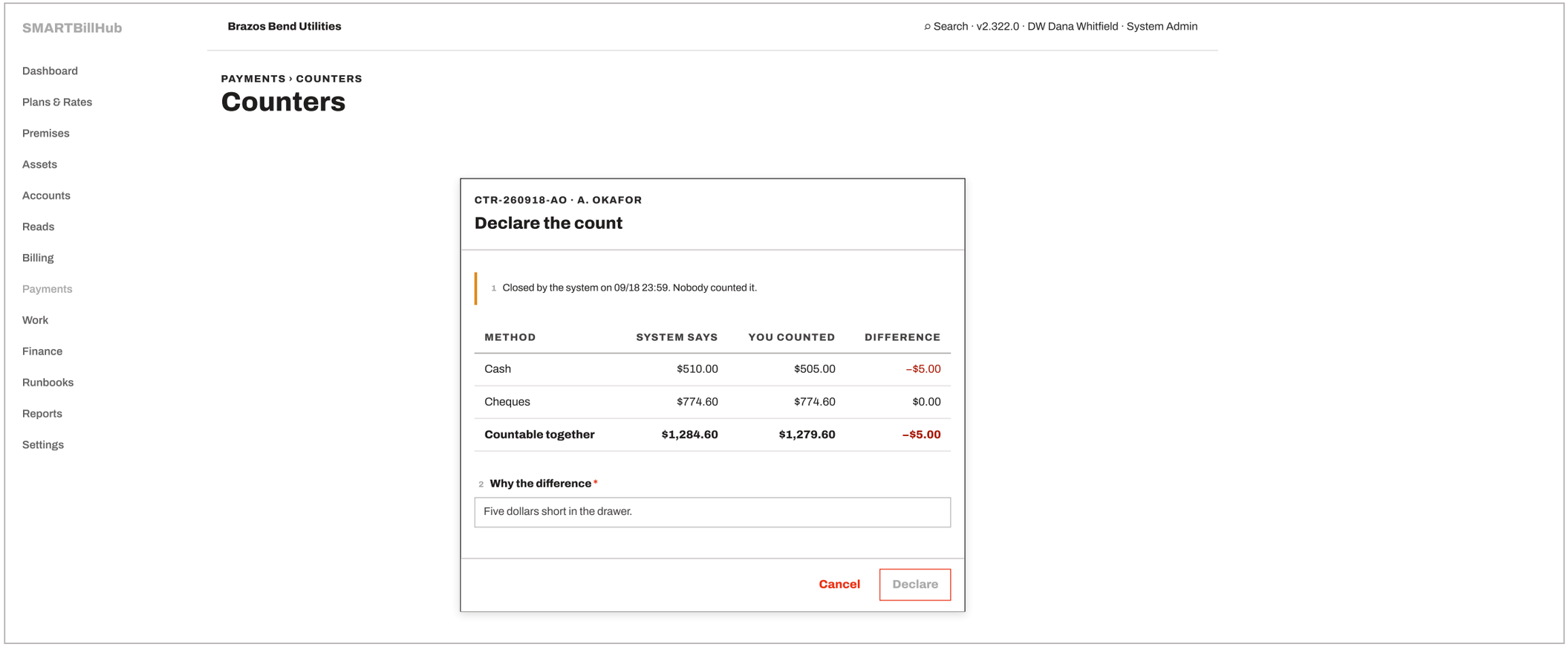
Primary action	**Force close**, danger tier.
Write surfaces	**Update** → `POST /counters/{id}/force-close`. **NOT SERVED.**
Rules	B14 **6** · **11** · **31** · **33**

3·3d Declare NEW S-NEW · PAY-N

`proposed POST /:tenant/counters/{id}/declare`

The question: "The system closed my till on Friday. Here is what was actually in it."

DESKTOP · 1440 · PROPOSAL · THE SAME COMPARISON, AFTER THE FACT



① **Why this is open at all** · an auto-close is a fact, said plainly · B14 **33** · NEW ② **A reason is required** · because a declared difference is the only number nobody watched

happen · NEW

For	The person whose till it was, or a supervisor.
Doors	A row action on 3·1, on an uncounted row only. **1 door.**
Primary action	**Declare** — it records a count, it does not move money, so it is the standard tier.
Write surfaces	**Update** → `POST /counters/{id}/declare`. **NOT SERVED.**
Rules	B14 **10** · **11** · **33**

4 What it changes

Where each act lands, and what goes stale

Act	What it writes	The ledger	What changes elsewhere
Start	A counter session and its batch, both open	Nothing yet	The **Start counter** control on every payments screen becomes the open-counter door
Take money	The payment carries the batch id from the first one	A balanced pair the same minute — undeposited funds against customer credit	The payment appears on 2·1 with Source **Counter** and its batch
Close	Closed time, closed by, the counted figure per method, any difference	Nothing — counting is not a posting	The batch becomes bankable. **It is the deposit batch**, so it appears on Finance → Deposits ready to be matched
Force close	The same, with no count and a required reason	Nothing	The row lands on 3·1 filtered as **Not counted**, and the owner is told
Declare	The count, late, with a reason	Nothing	The row leaves **Not counted**

TODAY **The one that reaches another module**

Closing a counter produces the deposit batch. Today deposit batches group **day × method** and the database enforces it with a unique key. Under the ruling of 2026-09-21 a batch is one person's session, so that constraint is replaced — which is a ledger and finance change, not a payments one, and it is in §04.

Bank reconciliation survives it. The matching already supports one bank credit paying for several of our deposits, so three tills closing on one day and landing as one credit is a shape it already handles.

TAB 4

Imports — lockbox files and store-collected cheques

A 10,000-connection utility takes 4,000 to 6,000 of these a month. **This tab is the ONE import runner the product already has** — the same five stages

---

This tab was redrawn on 2026-09-22, and the reason is a signed rule payments never cited

`BR-MET-096`**, PO ruling 2026-08-21:** *"ONE RUNNER, FIVE STAGES... the runner takes the target as a parameter; the field map and the rule set are looked up from it."* And explicitly: ***"This forbids building two importers."*** It has been exercised **seven times** through one component.

Payments built its own anyway. `payment_import.py` is a second staged pipeline with six endpoints, written under `BR-PAY-023` — *"batch → per-row validation → preview → commit"* — and it **never mentions BR-MET-096**. Two confirmed rules describing one act, and the second author could not see the first.

They are compatible, and payments' version is missing the stage that matters most here. BR-PAY-023's four steps map onto the runner's five almost exactly — except there is **no Transform**, so no column mapping. That is precisely the defect that forced accounts onto the shared runner: *"a utility whose CSV did not already use our header names could not import accounts at all."* **With every bank and every store sending a different layout, payments has that defect today.**

The decision that is not ours: under BR-MET-096 payments' pipeline either retires — as assets' staged path did, its two write verbs now refusing — or the rule is amended. Assets has the precedent and it went the first way. **Put to the PO; drawn here as the reuse.**

And the first redraw named the runner without drawing it — corrected 2026-09-22

These screens carried a five-stage rail **I drew myself out of boxes**, over panels that were also mine: a KPI strip and a bespoke reject table. **So the drawing said "this is the runner" and showed something else**, which is the exact failure this document exists to catch, one level below where anyone was looking.

They are now drawn from `07-frontend/src/components/workflow/` — `StageBar`, `SourceCards`, `DropZone`, `WriteModeField`, `MappingTable`, `RowHandlingOptions`, `ReconcileStrip`, `RuleResultsTable`, `RejectList`, `DryRunPanel`, `Receipt`, `RollbackNotice` — **in the kit's own words**, taken from `en.json`. `ImportWorkflowPage.tsx` is the reference: it mounts the kit and carries **zero stylesheet classes**.

The kit's own header says why this matters: *"ONE IMPORT PATH FOR THE WHOLE KIT, so a screen that needs the runner cannot reach half of it and hand-roll the other half — which is how `ImportAccountsWizard.tsx` ended up drawing a customers screen out of metering's stylesheet."* **A wireframe can make that mistake before a line of code does.**

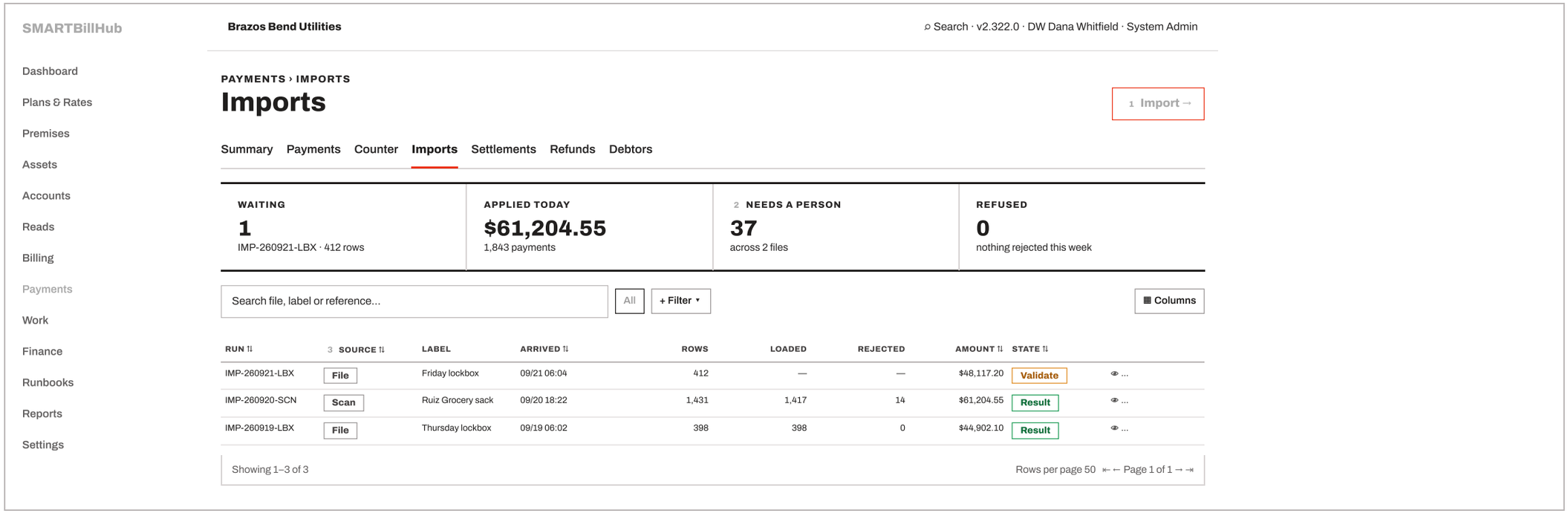
1 The list What has come in, and what state it is in

4·1 Batches UPDATE S35 · PAY-F

proposed /:tenant/payments/imports — **the same imports-tab list** premises, assets and accounts already render

The question: "What came in from the lockbox this morning?"

DESKTOP · 1440 · **AN EXISTING COMPONENT**, FILTERED TO THE PAYMENTS TARGET



① **Import →** · **one door, not two** — a file and a sack of cheques are the same run with a different SOURCE, and the source is chosen inside Extract · B14 **4** · **REUSED** · /payments/imports/new?target=payments ② **Needs a person** · rejected rows a guess must never commit on its own · **EXISTS** ③ **Source** · which card was chosen at Extract — File, Scan, and later SFTP · **EXISTS**

For	The clerk who handles the post.
Doors	The Imports tab. **1 door.**
Primary action	**Import** →, which enters the runner at Extract.
Table controls	Whatever the existing imports tab already does — this is that component with a different target filter.

Today	**Three copies of this list exist** — premises, assets and accounts — each querying the same `import-runs` read. Payments is a fourth caller, not a fourth implementation.
To build	A tab that mounts the existing list with `target=payments`.
Rules	B14 **4** · **12–17** · **36** · **43** · `BR-MET-096`

> **A second button was here until 2026-09-22, and taking it off is the change**
>
> This head carried **Scan** beside **Import** →, opening a screen of its own. **It is one door now.** Scanning a sack of store receipts is not a different pipeline — it is a different **source card** on Extract, and everything after it is the runner: mapping, rules, rejects, dry run, receipt, all of it already built. **That removes a screen rather than adding one.**
>
> **Scanning at the window is a different act and stays where it is** — a customer standing there needs the payment recorded now, which is 2·3a with the barcode filling the account, the bill and the amount. A sack of paper on a desk is a batch. **Same scanner, two acts, and only the batch belongs here.**

2 One run opened

Stages three, four and five — the checks, the commit, and the receipt

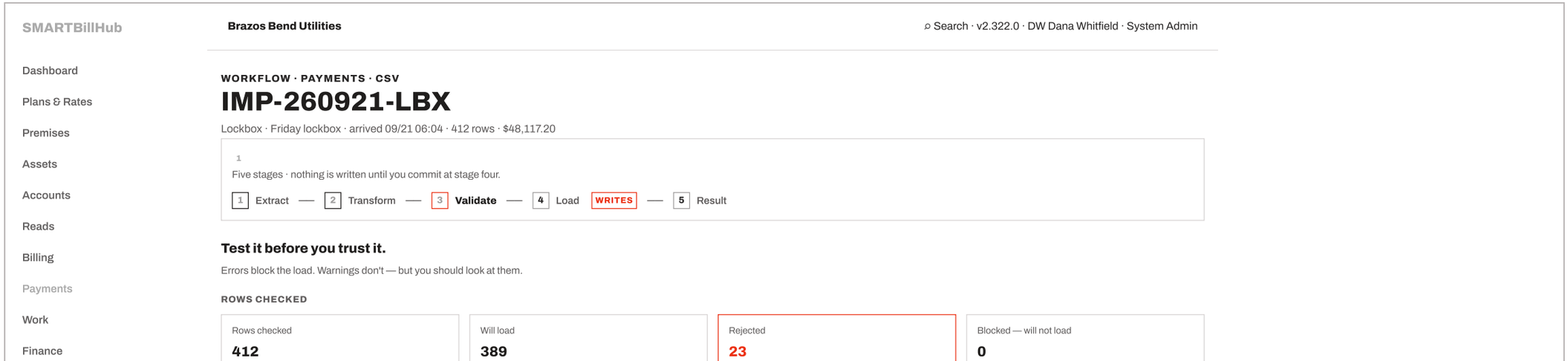
4·2 Run

UPDATE · S35 · PAY-F

`proposed /:tenant/payments/imports/IMP-260921-LBX — the runner's own run view, mounted with target=payments`

The question: "What is in this file, and which rows need me?"

DESKTOP · 1440 · **TWO STAGES OF ONE SCREEN** — VALIDATE, WHERE A PERSON WORKS, AND LOAD, WHICH IS THE ONLY STAGE THAT WRITES



Runbooks

Reports

Settings

2 Rules that ran

RULE	SEVERITY	HITS	ACTION
Account reference exists	Error	23	Reject row
Amount is a positive money value	Error	0	Reject row
Confirmation number not already on file	Error	0	Reject row
Paid-on date inside the statement period	Warning	6	Flag only
Payer name matches the account	Warning	31	Flag only

3 Fix the rejects here — no need to re-upload

ROW	KEY	PROBLEM	4 SUGGESTED FIX	FIX HERE
114	90114	No account matches this reference	Ann Ross · AC-2026-00019	Fix here…
208	90208	Two accounts match this reference	— no suggestion —	Fix here…
311	(blank)	Reference is empty	— no suggestion —	Fix here…

Saving re-runs the mapping and the checks over the corrected row — nothing commits over a stale verdict.

← Back

389 payments will post Load →

SMARTBillHub

Brazos Bend Utilities

Search · v2.322.0 · DW Dana Whitfield · System Admin

Dashboard

Plans & Rates

Premises

Assets

Accounts

Reads

Billing

Payments

Work

Finance

Runbooks

Reports

Settings

WORKFLOW · PAYMENTS · CSV

IMP-260921-LBX

Lockbox · Friday lockbox · arrived 09/21 06:04 · 412 rows · $48,117.20

Five stages · nothing is written until you commit at stage four.

1 Extract — 2 Transform — 3 Validate — 4 Load WRITES — 5 Result

Dry run first. Then commit.

This is the only stage that writes. Everything before it was rehearsal.

DRY RUN · WHAT WILL BE WRITTEN

Will create	Rows rejected	Rows rejected — already on file	Numbers we will issue
389	23	0	389

Before writing

5 Load options

☐ Create a rollback point before writing — unavailable for payments

☑ Notify me when the load finishes



① **StageBar** · the rail, its promise line, and **WRITES** on the one stage that writes — the runner marks it, r10 never did · B14 **3** · **EXISTS** · components/workflow/StageBar ② **RuleResultsTable** · Rule · Severity · Hits · Action, straight from the kit · B14 **20** · **EXISTS** · the rule set comes from the target ③ **RejectList** · the kit's inline editor, and its own sentence about stale verdicts · **EXISTS** · BR-MET-096 ④ **Suggested fix** · offered, never applied — **this column is the safety** · **EXISTS** ⑤ **No rollback point** · a posted payment cannot be unwritten — hard rule **4**. Reads is the other target that refuses it · B14 **32** · **NEW COPY** · the target supplies the sentence ⑥ **The commit names its scope** · **Commit 389 rows ✓** — the count is in the label because the load is PARTIAL · B14 **4** · **EXISTS** · advanceLabel

For	The clerk working through the morning's file.
Doors	A row on 4·1. **1 door.**
Primary action	**Load** → at Validate; **Commit 389 rows ✓** at Load, danger tier. Every stage's footer names its destination (B14 rule 4).
The footer fact	**389 payments will post** sits to the LEFT of the primary, not inside its label — a label is ≤ 3 words and carries no sentence (B14 rule 33). The kit has a slot for exactly this.
Write surfaces	**Edit** the rejects inline · **Update** at Load · **Delete** → discard the batch. **All the runner's.**
Today	**The runner does all of this for seven targets.** Payments has a parallel pipeline that does four of the five — it has no Transform, and none of these panels.
To build	**Nothing on this screen.** The target registration and its seven catalogue keys are the work.
Rules	B14 **3** · **4** · **6** · **11** · **20** · **32** · **33** · **36** · **43** · hard rule **4**

The rule this screen protects, and it is already built

The engine refuses to commit a row it cannot match with certainty, and its own code argues the case at length: *"Ann Ross · $292.87 matched exactly one account, just post it"* is the reasonable-sounding shortcut, and when it is wrong **the money lands on a different customer's account, pays down their bill and stops their collection ladder — while the customer who actually paid is chased for arrears they settled.** Two customers wrong, both silently, and neither can be edited away.

So the 23 rows are the feature, and the kit's `RejectList` is exactly where a person resolves them — which is why payments needs no screen of its own for it.

⑤ Payments is the second target that cannot be rolled back, and the sentence has to be written

Every target supplies its own rollback sentence — premises deactivates what it created, meters return to Inventory, parts are switched off rather than deleted. **Reads refuses outright:** *"A posted read batch cannot be rolled back. Every reading it committed is permanent, and a wrong one is corrected by uploading the right one — which supersedes it and keeps both on the record."*

Payments is the same shape, and for a stronger reason: hard rule 4. No UPDATE and no DELETE on a payment — corrections are new rows. So the payments target's sentence is a refusal, the rollback checkbox is unavailable with that reason, and **Discard this batch** before the commit is the only way to throw a run away. **Writing that sentence is part of the target, not an afterthought** — a target registered without it inherits nothing and shows an empty promise.

Stage 5 · what the receipt holds — no chrome, because nothing on it is a control

Loaded. Here's the receipt. `Receipt` records, per run: **Source** · **Mapping used** · **File hash (SHA-256) · verified** against the file you uploaded · **Run by** · **Session · IP** · **Rollback point**, which for payments reads *Not recorded*. Then `RollbackNotice` carries the target's sentence, and for payments it carries no **Roll back this run** button at all.

The hash line is the one worth naming. It reads *verified* or *not verified · could not be checked against the file you uploaded* — a distinction that matters the day a bank disputes what it sent, and one payments' own pipeline has nowhere to put.

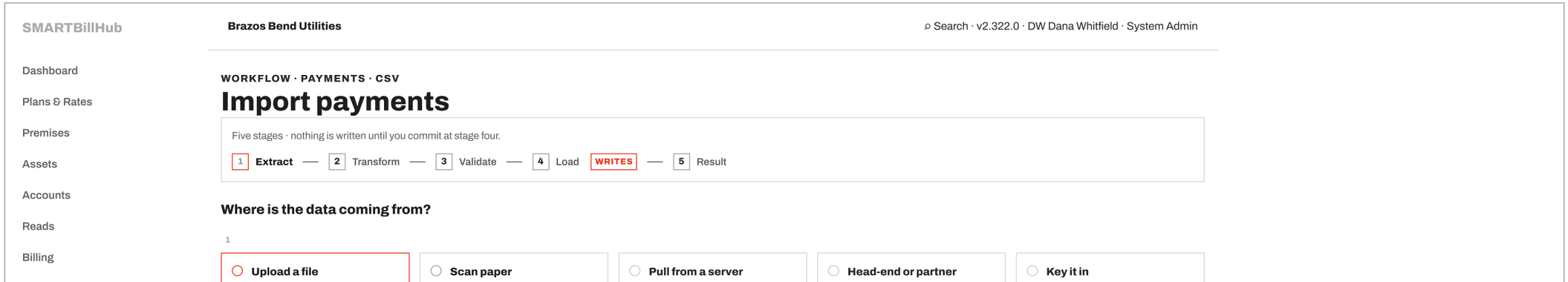
3 What you can do to it — Starting a run — the two stages before anything is checked

4·3a Import — UPDATE — S35 · PAY-F

`the ONE runner — /:tenant/payments/imports/new?target=payments`

The question: "This morning's lockbox file has arrived."

DESKTOP · 1440 · **EXTRACT AND TRANSFORM**, DRAWN FROM THE KIT — FIVE SOURCE CARDS, THE DROP ZONE, THE WRITE MODE, THE MAPPING TABLE



Payments
Work
Finance
Runbooks
Reports
Settings

CSV, fixed-width, BAI2, XLSX

Cheques and receipts, no file

Bank, lockbox, vendor drop
Not switched on yet. Upload a file instead.

AMI, payment processor
Not switched on yet. Upload a file instead.

Small batches, corrections
Not switched on yet. Upload a file instead.

2
↓ Drop the file here, or browse
lockbox-260921.txt · 412 rows · 96 KB
File hash (SHA-256) · 4f2a…8c1d

Download a sample CSV

3 **What does this file mean?**

◯ **Insert new only**
Every row becomes a new payment.

◯ **Insert & update**
Refused for payments — a payment on file is never updated (hard rule 4).
Not offered on this target.

Asked here, before anything is read, because it decides what the checks at the next two steps are checking FOR. You can change it until you commit.

Batch label
Friday lockbox
What you will call this run.

4 **Mapping template**
First National lockbox ⌕ ▾
How their column names map to ours.

Cancel

Transform →

SMARTBillHub

Brazos Bend Utilities

⌕ Search · v2.322.0 · DW Dana Whitfield · System Admin

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

WORKFLOW · PAYMENTS · CSV

Import payments

Five stages · nothing is written until you commit at stage four.

1 Extract — 2 **Transform** — 3 Validate — 4 Load **WRITES** — 5 Result

Map their shape onto ours.

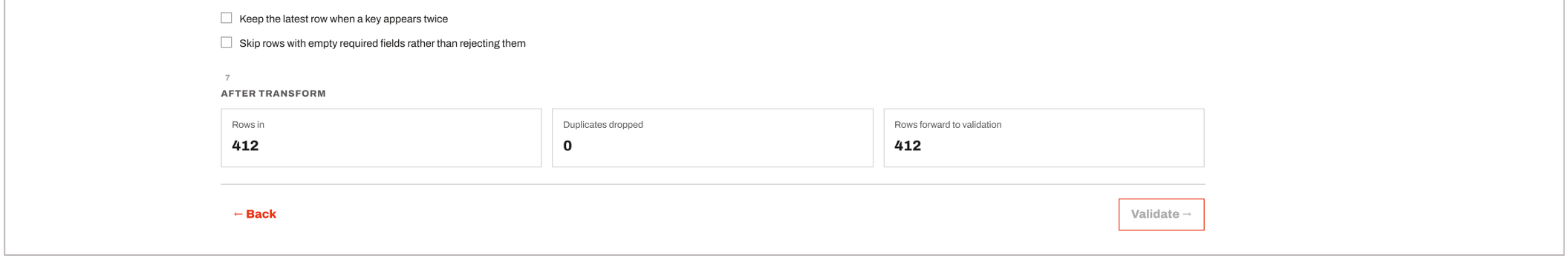
Set it once and save it as a mapping template — every future run of this import reuses it.

Save mapping as template

5 SOURCE COLUMN	SAMPLE VALUE	MAPS TO	TRANSFORM	STATUS
CUST_REF	90114	Account reference ▾	——	Matched
AMT	292.87	Amount ▾	Money · 4 dp	Matched
PAY_DT	09/21/2026	Paid on ▾	Date · MM/DD/YYYY	Matched
CHK_NO	4471	Confirmation ▾	——	Matched
PAYER	Ann Ross	— ignore — ▾	——	Ignored
BATCH	0914	— pick one —	——	Not mapped

6 **Row handling**

☑ Drop exact duplicate rows



① **SourceCards** · and **Scan paper is the fifth card** — a sack of cheques is a source, never a second screen · B14 **36** · **1 NEW CARD** · components/workflow/SourceCards ② **DropZone, and the sample CSV** · the file, its rows, its size and its SHA-256 — a repeat is already refused. **The sample is the SERVER's**, its header row derived from the target's real columns · **EXISTS** · CR-025, PO 2026-08-21 · BR-MET-088 ③ **WriteModeField** · **one control for every import in the product** — CR-042 / BR-MET-210. My first drawing left it off entirely · B14 **9** · **1 OPTION** · options is a prop; payments passes one ④ **Mapping template** · **the stage payments' own pipeline does not have** — and the reason this tab moves to the runner · B14 **8** · **EXISTS** ⑤ **MappingTable** · Source column · Sample value · Maps to · Transform · Status · B14 **20** · **EXISTS** · the field map comes from the target ⑥ **RowHandlingOptions** · three choices, the kit's own words · **EXISTS** ⑦ **ReconcileStrip** · the arithmetic of the stage — rows in, dropped, forward · **EXISTS**

For	The clerk starting a file, or working a sack of paper.
Doors	**Import** → on 4·1. **1 door.**
Primary action	**Transform** → at Extract, **Validate** → at Transform — every stage's footer names its destination.
Fields	Source (required, cards) · File · **What does this file mean?** (required) · Batch label · Mapping template.
Write surfaces	**Add** → the runner's own extract. **Nothing is written until Load.**
Today	**The screen exists and works for seven targets.** Payments is not one of them.
To build	One dict entry: column set, aliases, default mapping template, dedupe key, rule set — plus **seven catalogue keys** (target, kicker, blurb, next, exit, return, **rollback**) and a **Scan paper** source. **The LOAD calls payments' own `import_writer` through the seam registry**, exactly as `parts` calls work's — so metering writes nothing of payments'.
Rules	B14 **3** · **4** · **8** · **9** · **10** · **11** · **20** · **36** · **43** · `BR-MET-096` · `BR-MET-210` · hard rule **4**

③ The write mode was missing from my first drawing, and that is a defect rather than a style note

`WriteModeField` is **one control for every import in the product** — `CR-042`, `BR-MET-210` — and it is asked at step 1 **before anything is read**, because the mode *"decides what the file MEANS"* and the checks at the next two stages report against it. The PO found it the hard way: their screenshot of the premises importer showed step 3 saying *"the write mode*

decides what happens to it" on a screen that still refused to show the write mode.

Payments offers one of the two. `options` is a prop, so a target chooses what it passes — and **Insert & update** would mean overwriting a payment already on file, which hard rule 4 forbids outright. So it is drawn present and refused, with its reason, rather than silently absent: **a clerk who has used the premises importer will look for it.**

② The scanner is the easy half. There is nothing to scan.

A barcode scanner is a keyboard — it types what it reads and presses Enter. No driver, no integration, no pairing. So **Scan paper** is a source card and a focused input, and everything after it is the runner: the same mapping, the same rules, the same rejects, the same dry run, the same receipt.

But our printed bill carries no remittance barcode, and the product says so itself: the document code declares *"no logo, no remittance barcode, no colour"* on every result, and the mail file's readme prints *"No Intelligent Mail barcode has been applied."* Custom bill design was ruled out for V1 — three to five standard layouts only — **so the barcode goes into those layouts**, carrying account, invoice, amount and a check digit. **Per the ruling of 2026-09-21 the symbology is not a tenant setting** — one format, shipped.

It pays off twice: a sack of store receipts scanned into a run here, and a customer's own bill scanned at the window, which fills 2·3a in one movement.

What the payments target adds that no other target has

Every existing target **creates** rows — a premises, a meter, an account. Payments **matches** a row to an account that already exists. **That is not a second pipeline; it is a rule set at Validate**, and `RejectList` is already where a person resolves what the rules could not. The rule that a name-and-amount guess never commits on its own stays exactly as written.

One card already describes our own case and is switched off: *Pull from a server*'s own subtitle is *"Bank, lockbox, vendor drop"*. A nightly lockbox drop is precisely what a utility wants, and the kit already has the card for it — held with *"Not switched on yet. Upload a file instead."* **That is a door worth opening later, and it needs no new screen when it is.**

4 What it changes

Where a loaded run lands

Stage or act	What it writes	The ledger	What changes elsewhere
Extract	The run, its file stamp and its staged rows. **No payment.** A scanned sack stages exactly as a file does — one row per scan	Nothing	Nothing. A clerk who uploaded the wrong file has moved no balance
Transform	The column mapping for this run, and the template if it is saved	Nothing	Nothing — and **this is the stage payments has never had**
Validate	Per-row verdicts, and the account a person picks for a reject	Nothing	The row leaves **Rejected** and the checks re-run over it

Load	One payment per ready row, through **payments' own writer** via the seam — metering writes nothing of ours. **Partial by ruling:** what is correct posts, what is not is rejected	A balanced pair per payment	Every one appears on 2·1 with Source **Post** and its run · bills close · ladders stop · receipts queue
Result	The receipt — source, mapping, verified hash, who ran it, session and IP. **No rollback point**	Nothing	The run reads **Result** on 4·1. **A wrong payment is reversed, never unwritten**

The failure this tab has to survive, and the runner already does

A Load writes hundreds of payments in one transaction. **On PostgreSQL a single failed statement poisons the whole transaction** — every later query raises and the work is lost at commit — so skipping a bad row needs a savepoint, not a try/except. **The test suite runs on SQLite, which has no such state**, so the wrong version passes every test on every machine and fails the first time a real row is refused in production. **The partial commit above makes this certain rather than likely:** a run that rejects 23 of 412 rows is a run that takes a refusal mid-transaction. Any change here is proved on PostgreSQL or it is not proved.

TAB 5

Settlements — what each provider actually paid out

Taking a card payment and being paid for it are two different messages, and the product only handles the first. **There is no settlement run anywhere**

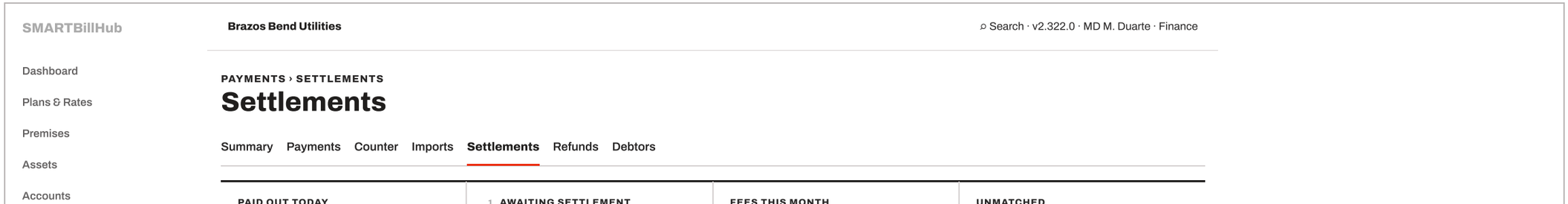
1 **The list** One row per provider payout

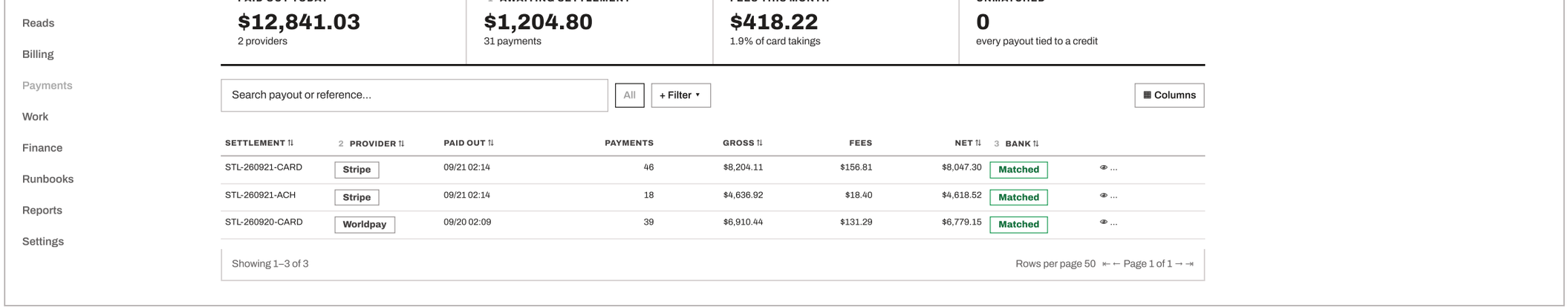
5·1 Settlements NEW S-NEW · PAY-P

proposed /:tenant/payments/settlements

The question: "The bank shows one credit from Stripe. Which payments is it?"

DESKTOP · 1440 · PROPOSAL · NOTHING BEHIND THIS EXISTS





① **Awaiting settlement** · the third state no requirement names — taken, not yet paid out · NEW ② **Provider** · one, two or three per utility, ruled 2026-09-05 · NEW ③ **Bank** · a payout is one bank line, which is why the batch exists · NEW

For	Finance, matching the bank.
Doors	The Settlements tab. **1 door.**
Primary action	None — this is a read.
TODAY **Today**	**Nothing.** Provider payments arrive and are recorded; **nothing models being paid for them.** No settlement table, no feed, no screen.
TASK **To build**	`settlements` and its members · a provider settlement feed in the integration framework (§04) · **one new number series** · `GET /settlements`
Rules	B14 **12–17** · **31** · **33** · **40**

GAP **Why this tab exists at all**

Your rule is that **every payment carries a batch number**. A counter payment gets one when the till opens; an imported one gets its file's. **A card payment has nothing to belong to until the provider pays out** — and the payout is the thing the bank statement shows as a single line. So the settlement *is* the card payment's batch, and without it the rule cannot hold for a third of the money.

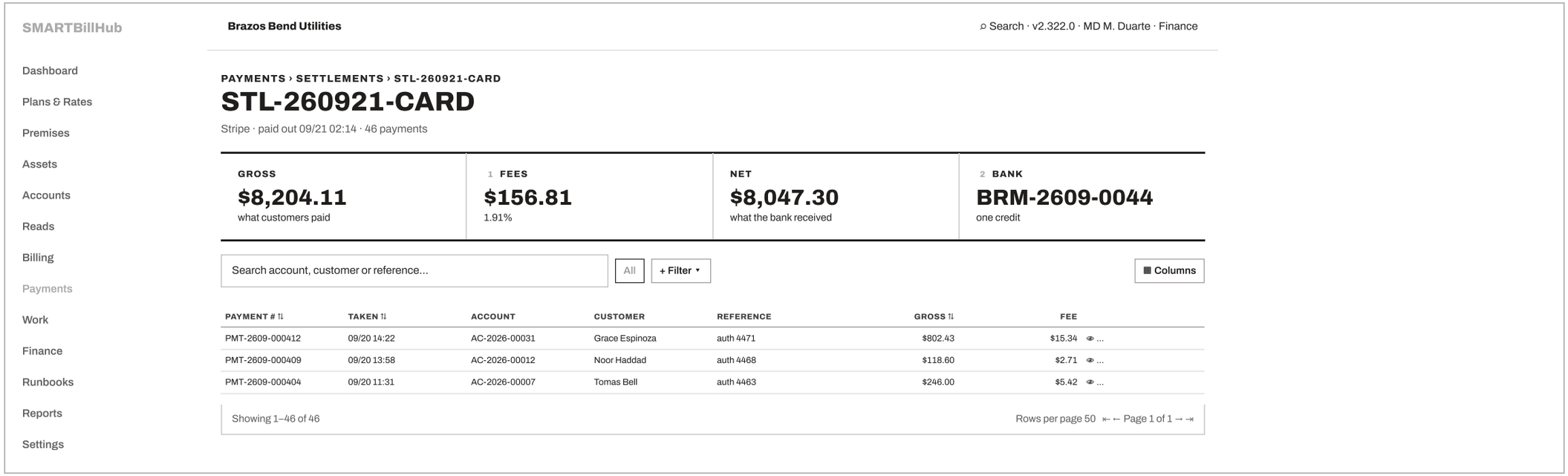
2 **One payout opened** The payments inside it, and the fee

5·2 Settlement NEW S-NEW · PAY-P

proposed /:tenant/payments/settlements/STL-260921-CARD

The question: "Which 46 payments made up this $8,047.30?"

DESKTOP · 1440 · PROPOSAL



① **Fees** · the difference between what a customer paid and what we banked · NEW ② **One credit** · which is the whole reason a batch number is needed · NEW

For	Finance tying a bank credit to its payments.
Doors	A row on 5·1. **1 door.**
TODAY **Today**	**Nothing.**
TASK **To build**	The read, and the fee posting — a fee is an expense, not a smaller payment, and the customer paid the gross.
Rules	B14 **12–17** · **31** · **33** · hard rules 2 and 5

3 What you can do to it

Nothing — and that is the design

TODAY **A settlement has no actions, on purpose**

It is a **statement of fact from a provider**. There is nothing to approve, nothing to edit and nothing to cancel — if a payout is wrong, the provider issues another one. Every action a clerk might want here belongs to the **payment** inside it (tab 2) or to the **bank match** (Finance).

Drawn as a step rather than left out, because **a step with nothing in it is a decision**, and a reader should be able to see it was made.

4 What it changes Where a payout lands

Act	What it writes	The ledger	What changes elsewhere
A payout arrives	A settlement and its members	The fee posts as an expense; the gross was already posted when each payment was taken	**Every payment in it stops reading *Awaiting settlement* on 2·1** and shows its batch
The bank confirms	A match between the payout and one bank line	Undeposited funds clears	Finance → Bank reconciliation shows it matched

TAB 6

Refunds — give it back, or leave it on the account

Moved here from Finance on 2026-09-21. The model, the service, the API and the screen component are all in payments already; **only the URL said Fina**

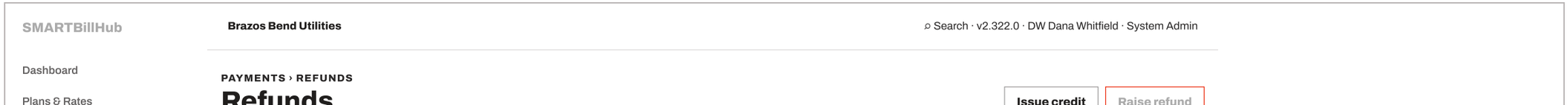
1 The list What is waiting, what has gone

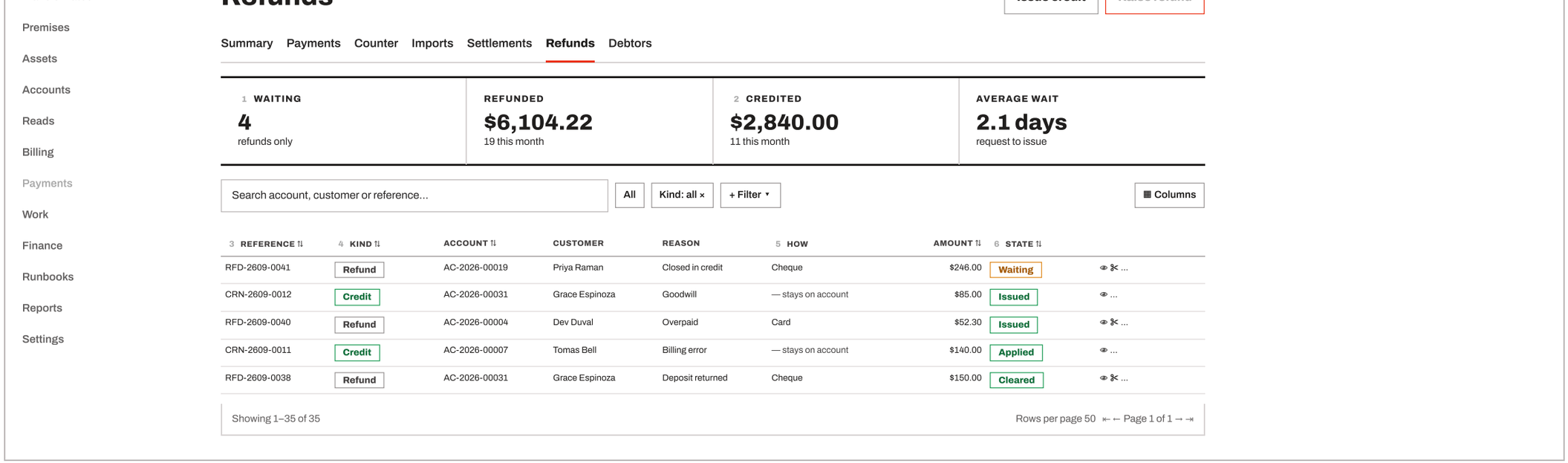
6·1 Refunds UPDATE S58 · PAY-P

today /:tenant/finance/refunds — **proposed /:tenant/payments/refunds,** holding refunds AND credits

The question: "What money have we given back, and what have we left on accounts?"

DESKTOP · 1440 · **ONE LIST, TWO KINDS** · THE REFUND HALF EXISTS WITH THREE TABS; THE CREDIT HALF IS THE CHANGE





① **Waiting** · refunds only — **a credit has nothing to wait for** · EXISTS ② **Credited** · what we left on accounts this month · NEW ③ **Reference** · RFD- or CRN- — two series, one column · NEW ④ **Kind** · Refund or Credit, as a column **and** a filter · B14 **12** · NEW ⑤ **How** · cheque or card for a refund; **a credit is not paid out at all** and says so · NEW ⑥ **State** · **the two lifecycles are different and the column does not pretend otherwise** · NEW ✂ appears only on a refund row — a credit has nothing to void

For	A clerk answering “did we give that back?”, and finance approving what goes out.
Doors	Today: the Finance tab strip only. Proposed: the Refunds tab, plus a door from a payment. **1 door today, 3 proposed.**
Primary action	**Raise refund**, with **Issue credit** beside it — so the cheaper option is visible at the moment the choice is made.
Table controls	Three tabs today — approval, issued, all. **The toolbar, the Kind filter and paging are the change.**
TODAY **Today**	**Refunds only, and in the wrong place.** Everything about refunds is payments except the URL, which is declared in the payments module’s own file. **A credit issued today appears nowhere in payments at all** — it lives in billing and is visible only from the account.
TASK **To build**	Move the route · the toolbar and paging · **a Kind column and filter** · **and a read that unions both** — see below.
Rules	B14 **12–17** · **20** · **31** · **33** · **34**

RULED Why it moved, and what it supersedes

The code states the old reason plainly: *"money leaving the utility is approved and reported in Finance, while the record belongs to payments."* That is a real rule, and the split it

produced is exactly what orphaned the write-off queue — same reasoning, same Finance tab strip, and today **nothing anywhere links to that screen**.

Role separation does not need a separate page: issuing and clearing already carry their own role checks server-side, which work wherever the button sits. **This supersedes CR-059 §10's placement, not the rule behind it** — Finance keeps reporting money out, which is a read.

GAP

The two have different shapes, and the columns say so rather than flattening them

A refund has five states and a two-person control — requested, approved, issued, cleared, voided — because money leaves the utility and a bank has to confirm it. **A credit note has one field that matters and no lifecycle at all**: it is issued, and it auto-applies at the next bill. There is no `state` column on `CreditNote` to read.

So the shared columns tell the truth instead of inventing a state for a credit: **How** reads — *stays on account* rather than a blank, and the **✂ void** action appears only on a refund row. **A single "status" that meant something different depending on the row would be the drift this document keeps finding.**

GAP

Payments cannot read billing's table, and that is the whole of the backend task

`CreditNote` **is billing's** — account, original invoice, credit note number, amount, reason, issued at — and cross-module reads go through a seam, never a direct query. `apps/payments/billing_seam.py` already exists for exactly this. **So the list is one read that unions payments' own refunds with billing's credit notes through that seam**, and filtering by Kind decides which half is asked for at all.

Two number series, one column. Refunds carry their own; credits carry **Credit memo #**, which is already one of the thirty series a tenant configures. **Neither is invented here.**

TODAY

One thing this change makes visible, and it is worth saying to the Product Owner

With credits in the list, **the tab is called *Refunds* and holds four things that are not refunds**. That is the same complaint that renamed 6·3c on 2026-09-22 — a name that describes half of what is behind it.

The obvious answer is *Money out*, two words, within the naming rule. **It is not done here**, because renaming a tab moves the strip on all twelve drawings and is a product decision rather than a drawing one. **Say the word and it changes.**

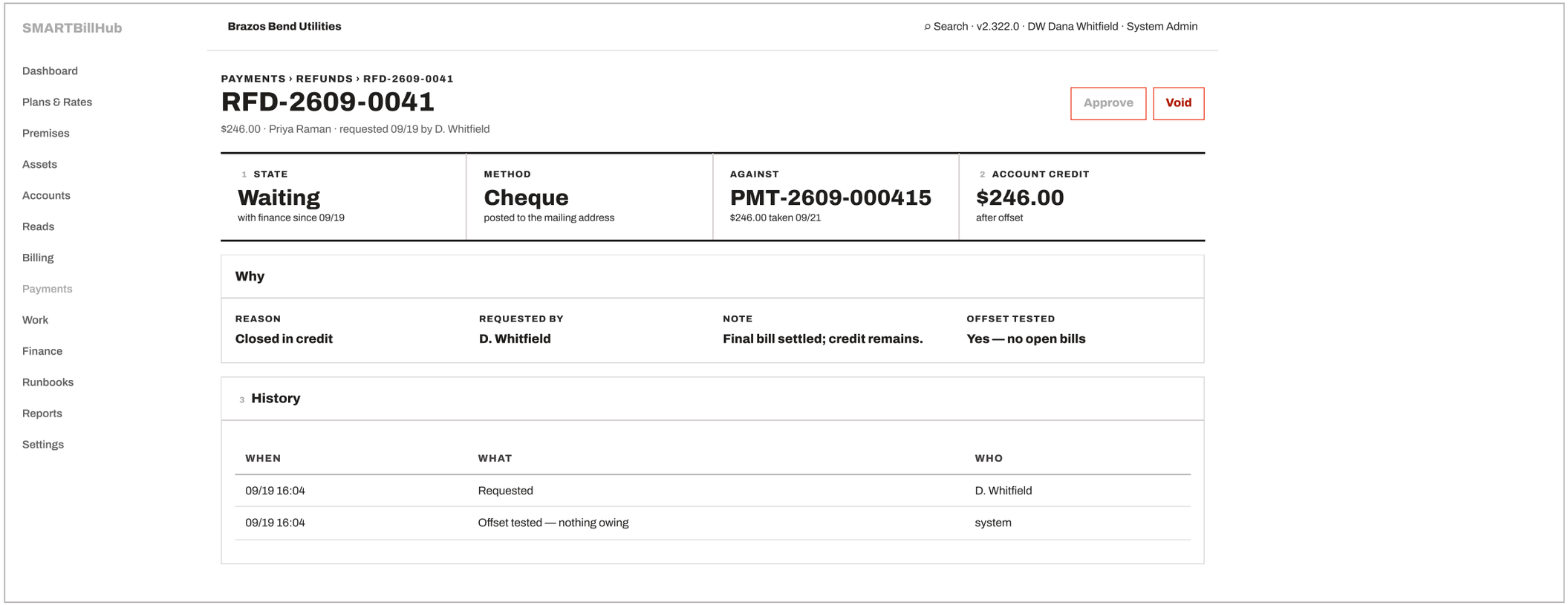
2 **One refund opened** Its whole story, which today you cannot see

6·2 Refund NEW S-NEW · PAY-P

proposed /:tenant/payments/refunds/RFD-2609-0041

The question: "Where has this refund got to, and why is it taking so long?"

DESKTOP · 1440 · PROPOSAL — THE SAME GAP A PAYMENT HAS



① **State** · the question the list can show but not explain · NEW ② **Offset first** · a refund may not go out while the account owes — tested at request and again at issue · EXISTS ③ **History** · who did what, when · B14 **31** · NEW READ

For	Anyone asked “where is my money?”
Doors	A row on 6·1. **1 door — there is none today.**
TODAY **Today**	**No detail view exists.** The list has a raise modal and an act modal, and nothing in between.
TASK **To build**	`GET /refunds/{id}` and this screen.

Rules	B14 **6** · **31** · **33** · **40**

3 What you can do to it

Raise it, move it along, or leave the money where it is

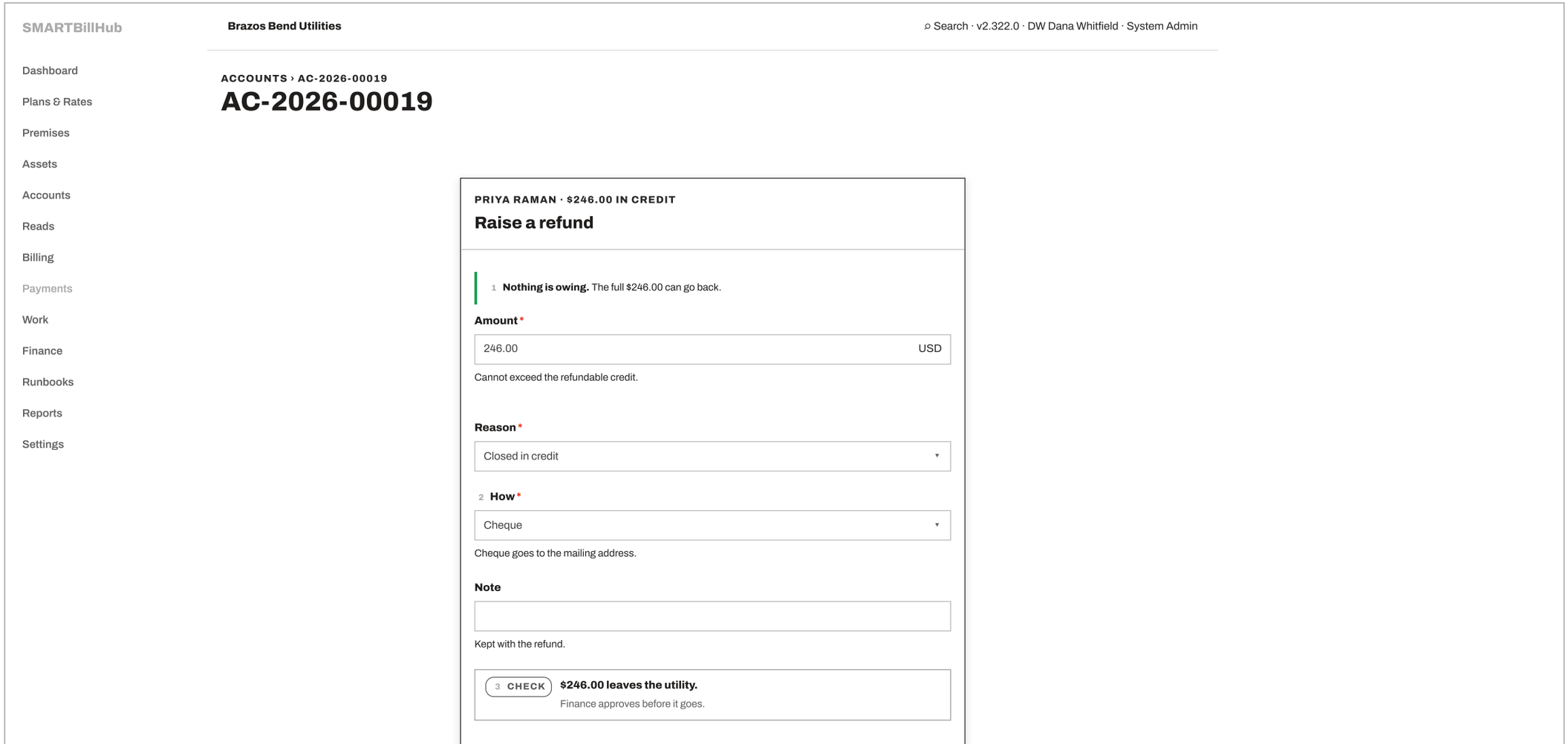
6·3a Raise

UPDATE · S58 · PAY-P

`POST /:tenant/refunds — served and built`

The question: "They closed the account and we owe them $246.00."

DESKTOP · 1440 · EXISTS; THE CONSEQUENCE BOX IS THE CHANGE





① **Offset first** · the rule that stops a refund going out to someone who owes us · EXISTS ② **How** · cheque or back to the card · EXISTS ③ **What it will do** · and that approval stands between · B14 **11** · NEW

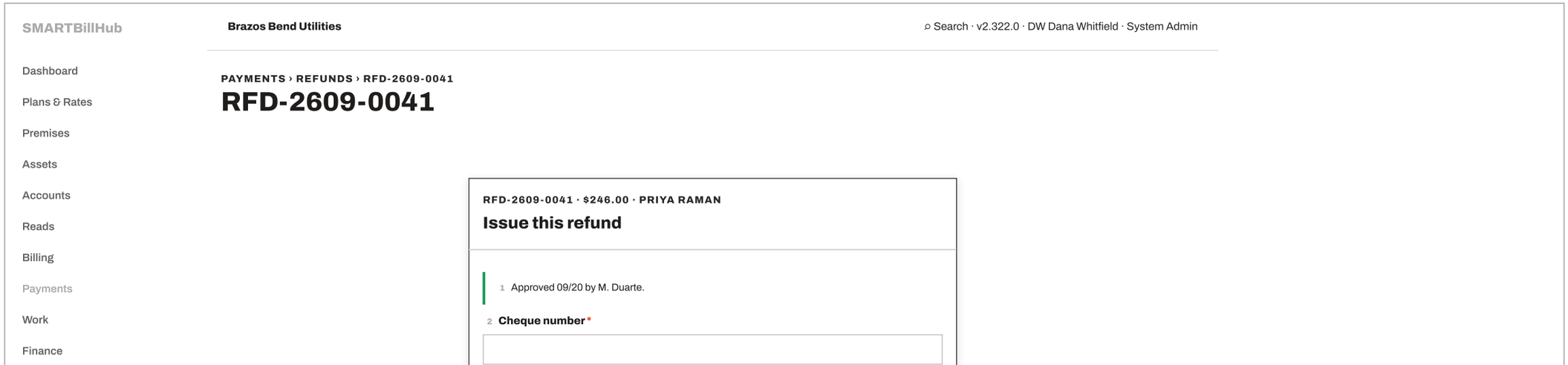
For	A clerk closing an account in credit.
Doors	**Raise refund** on 6·1, and the account. **2 doors.**
Write surfaces	**Add** → `POST /refunds`. **Served.**
TODAY **Today**	Built. Missing only the consequence box.
Rules	B14 **10** · **11** · **33**

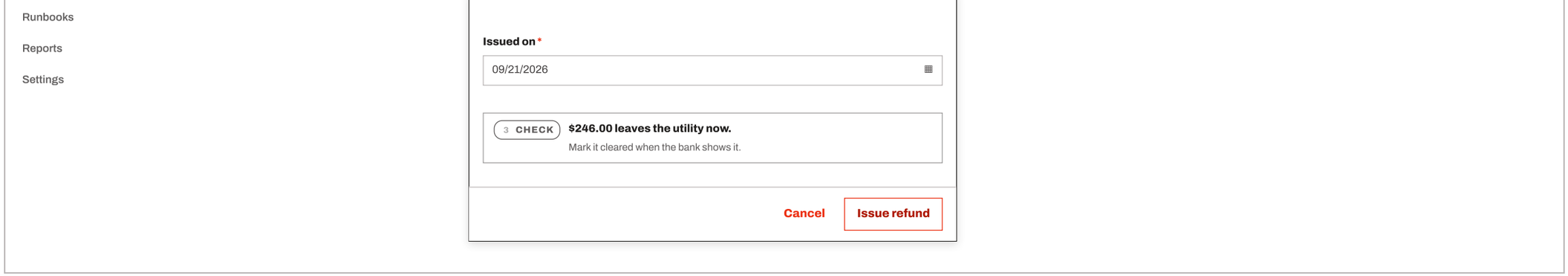
6·3b Issue UPDATE S58 · PAY-P

`POST /:tenant/refunds/{id}/approve · /issue · /clear · /void — all served`

The question: "Finance has approved it. Send the cheque."

DESKTOP · 1440 · EXISTS; THE DANGER TIER AND THE CONSEQUENCE BOX ARE THE CHANGE





① **Who approved** · because issuing is a different person's act · EXISTS ② **Cheque number** · the only record of which cheque this was · EXISTS ③ **Money out** · danger tier — it is irreversible in the way that matters · B14 **6 · 11** · CHANGED

For	Finance, at the point money leaves.
Doors	A row on 6·1 and the header on 6·2. **2 doors.**
Write surfaces	Four endpoints, **all served**, with real role separation: approve and issue carry a service role; *mark cleared* is *"a bank fact, not an approval"*.
TODAY **Today**	Built. **The commit renders in the standard tier**, like Save — same defect as the reversal on 2·3e.
Rules	B14 **6** ✗ today · **10** · **11** · hard rules 4, 5, 7

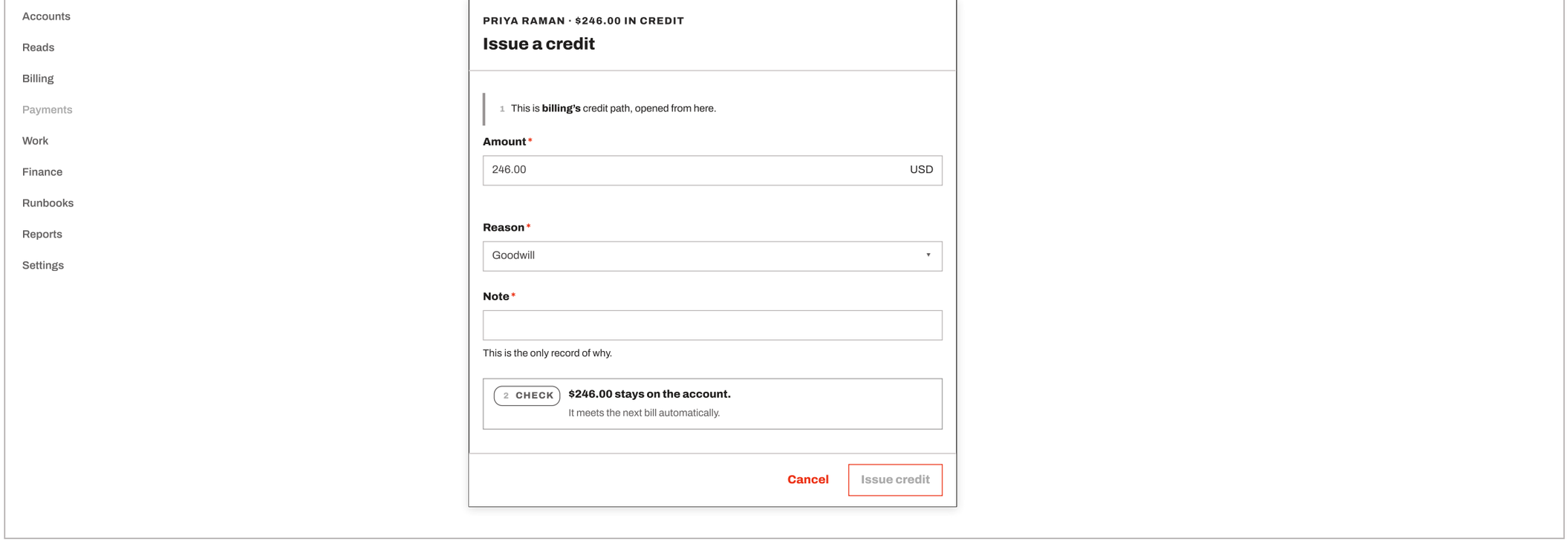
6·3c Credit UPDATE S22 · PAY-P

billing's credit-only correction — reached from the account today

The question: "Don't send a cheque — leave it on their account."

DESKTOP · 1440 · EXISTS IN BILLING; WHAT IS MISSING IS A DOOR FROM PAYMENTS





① **Whose path this is** · named, because a clerk should not have to know — but a build lane must · EXISTS ② **What happens next** · auto-apply at the next issue, which is already built · B14 **11** · NEW

RULED

Renamed 2026-09-22, and it was a rule breach as well as a puzzle

The headline read *"Leave it on the account"* and the PO's objection was simply that **a user cannot understand it**. It is now **Issue a credit**, which is also the words S22's own spec has always used — *"Record payment, Adjust, Statement, Print bill, **Issue credit**..."*.

And the old one broke B14 rule 33: a heading is **three words** and never a sentence, and that was five. **Auditing the rest found seven more over the cap** — modal titles and panel headings alike — and all eight are cut. Where a heading was carrying a fact, the fact moved to a note under it rather than being deleted: *Rejected rows* keeps *"Fix them here. No re-upload needed."*, and *Write-offs waiting* keeps *"A second person decides."*

For	A clerk who would otherwise raise a refund.
Doors	Today: the account only. Proposed: **beside Raise on 6·1**, so the choice is visible at the moment it is made. **1 door today, 2 proposed.**

Write surfaces	billing's credit-only correction. **Served and built.**
TODAY Today	**Built, in another module, and invisible from here** — so a clerk reaches for a refund because it is the only option they can see.
TASK To build	A door, and the consequence box. **No new engine.**
Rules	B14 **10** · **11** · **33**

4 What it changes

Money out, and money left where it is

Act	What it writes	The ledger	What changes elsewhere
Raise	A refund, requested	Nothing yet	It appears in finance's approval queue
Approve	Who approved, when	Nothing	It becomes issuable
Issue	The cheque or card reference, and the date	A balanced pair — the credit clears, cash goes out	**The customer is told** — `refund_issued` is a live event with its own template
Clear	The bank date	Undeposited funds clears	Bank reconciliation matches it
Void	A reversing row; the original stands	A reversing pair	The credit returns to the account
Credit	A credit-only correction in **billing**	A balanced pair	**The customer is told** — `credit_issued` is live · it auto-applies at the next issue

> TASK **The choice this tab exists to make visible**
>
> A refund costs a cheque, a posting, a bank line and a reconciliation. **A credit costs nothing and the customer often prefers it.** Today a clerk sees only the refund button, so that is what they press. Putting **Credit** beside **Raise** is the whole change — no new engine, one door.

TAB 7

Debtors — who owes, chasing it, writing it off

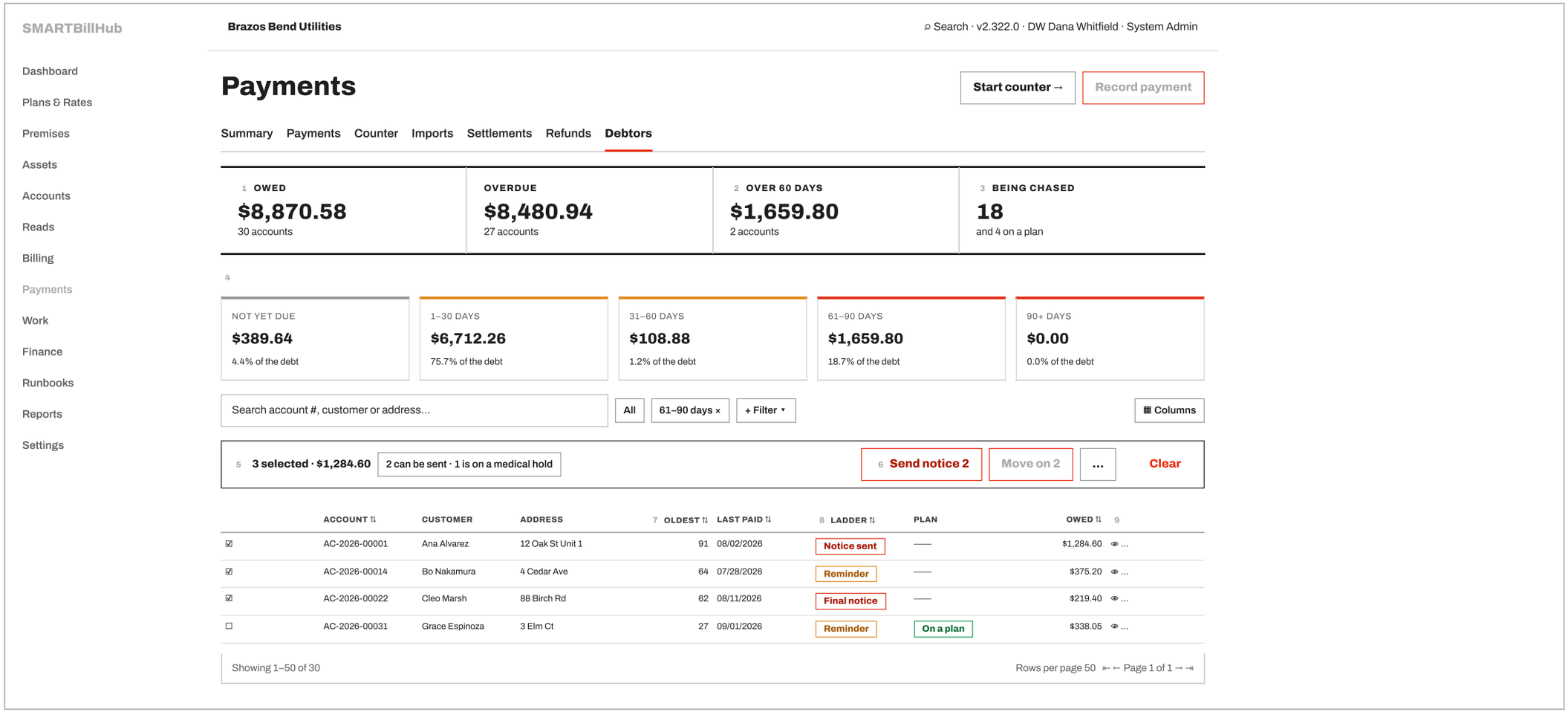
The oldest screens in the module and the worst rule breaches in it: **four of the eighteen sit on one screen,** and both unpaginated debtor reads are h

7·1 Debtors UPDATE S34 · PAY-J

`/:tenant/payments/outstandings → OutstandingsTab`

The question: "Who owes us money, and who do I chase first?"

DESKTOP · 1440 · **DRAWN TO THE RULES** · THE FIGURES ARE THE ONES THE RUNNING APP RENDERED



① **Four tiles, not nine** · owed · overdue · over 60 · being chased · B14 **32** · CHANGED ② **Over 60 days** · the tail that decides escalation, not the total · NEW ③ **Being chased** · what is already in hand, so you can see what is left · NEW ④ **Five cards, each a filter** · the ageing bands — lighter than the tiles above, and they narrow the

list · CHANGED ⑤ **The toolbar swaps in place** · the same row, its contents replaced — **no second bar, no layout jump** · B14 **12** · CHANGED ⑥ **Send notice** · danger tier — real postage, a legal instrument, and it cannot be recalled · **Write off** moves under **…**, B14 **15**'s two-plus-overflow · B14 **11** · NEW ⑦ **Oldest** · days, not a balance — the ladder runs on age · B14 **13** · NEW ⑧ **Ladder** · which rung, as a pill rather than a sentence · B14 **33** · NEW ⑨ **Row actions** · 👁 Detail then overflow — **there are none at all today** · B14 **15** · NEW

For	A clerk deciding who to chase this morning.
Doors	The Debtors tab. **1 door.**
Primary action	**Move on** — advance the ladder. **Write off** beside it, danger tier.
Table controls	**All four, and none of them exists today.** Search, the `All` chip and filter chips, `+ Filter ▾`, `▦ Columns`, ⇅ headers, a pagination footer.
Write surfaces	**Detail** → 7·2 · **Add** NOTHING · **Edit** NOTHING · **Update** → the bulk acts
TODAY **Today**	**Four rule breaches on one screen** — no toolbar (the word "search" appears zero times in 923 lines), a static "Sorted by balance" label where a control belongs, no column chooser, no paging. **Plus NFR-3**: the seam materialises every open invoice before filtering, so a filtered request costs what an unfiltered one costs, and the sort runs in Python.
TASK **To build**	The toolbar · ⇅ headers · columns · paging · **sort and page in SQL** · the four KPI tiles · the ageing bar as filters · the bulk bar's eligible count · row actions.
Walked	**Yes.** Every figure above is what the running app rendered.
Rules	B14 **11** ✗ · **12** ✗ · **13** ✗ · **14** ✗ · **15** ✗ · **16** ✗ · **32** · **33** · NFR-3 ✗

TASK **Nine tiles became four, and nothing was lost**

The page carried two KPI blocks — accounts owing, total owed, a *Sort* label, five ageing bands and a days-sales-outstanding tile. **The five bands are now one row**, sized by share, and **each segment is a filter**: the information became a control instead of taking four times the height to say the same thing.

Two tiles left the KPI area on rules, not taste. *Sort · Largest balance first* is a static label standing in for a control — it belongs in the table as a ⇅ header (rule 13). *Days sales outstanding · Reported on the Finance page* is a **stated absence**, and rule 32 makes a stated absence a closed disclosure, never a tile.

GAP **Six bulk buttons, and four of them have never worked**

The code says so in its own words: *"FOUR of the prototype's six bulk actions render DISABLED WITH THEIR REASON."* **Send a reminder** and **Send a disconnection notice** both need `POST /collections/notices` — **which is not mounted**; only `collections/run` and `collections/ladder` are. **Offer a payment plan** is another story's wizard. **Book a site visit** says *"no agreed rule behind it"*.

So the bar drops to two plus an overflow, and the four return when their endpoints land. And like the payments bulk bar, it now states **what can actually happen** before you press — the eligible count, not just the selection.

One of the four is worth re-checking rather than accepting. `RaiseFieldWork` is a live component on four screens and Work became the eleventh module on 2026-09-03. The refusal claims a missing *rule*, not a missing capability — and a collections visit may genuinely still have none — but nothing re-checks a refusal, which is how the other one on this page rotted.

TASK **The selection toolbar is the product's ONE pattern for this now**

Both lists that carry bulk acts — this one and 2·1b — draw the **same** control: the toolbar's own row, its contents swapped. Not a floating bar above the table, which pushes the table down at the moment you have just ticked specific rows; not a bar stuck to the window, which covers the last row. **One pattern, in the kit, so the third list that needs it inherits rather than invents.**

And each act carries its count — *Move on 2*, not *Move on* — because the selection and the eligible set are different numbers and only one of them is going to happen.

TODAY **One thing here is right and must not be "optimised"**

The selected total is summed in the browser, exactly, on the money grid with BigInt — because no endpoint totals a selection and a float would be the money defect hard rule 2 exists to prevent. **And the missing CSV export is correct, not missing:** `POST /exports` registers no outstandings resource, and an undeclared capability stays unrendered.

2 One debtor opened

Everything they owe, and everything we have tried

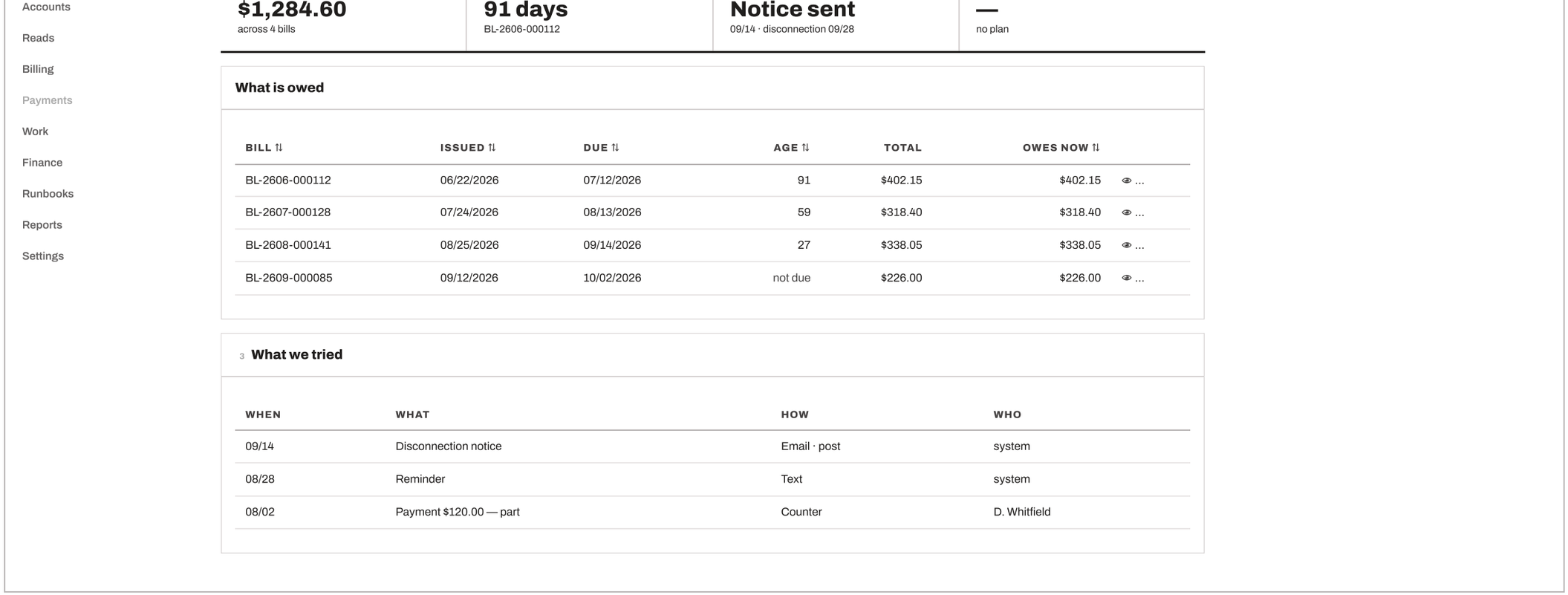
7·2 Debtor NEW S-NEW · PAY-J

`proposed /:tenant/payments/debtors/AC-2026-00001`

The question: "Why does this account owe $1,284.60, and what have we already tried?"

DESKTOP · 1440 · PROPOSAL — THE SCREEN THE LIST HAS NEVER HAD





① **Oldest** · the number that decides the ladder rung, not the total · NEW ② **Where they are on the ladder** · and what happens next, with its date · NEW READ ③ **What we have already tried** · the question a clerk on the phone actually has · B14 **31** · NEW READ

For	A clerk with the customer on the phone.
Doors	A row on 7·1. **1 door — there is none today.**
Primary action	**Record payment**, which is 2·3a with the account known.
TODAY **Today**	**No such screen.** The debtor list is a report you cannot drill into.
TASK **To build**	One read that gathers bills, ladder state and history for an account.
Rules	B14 **4** · **12–17** · **31** · **33** · **40**

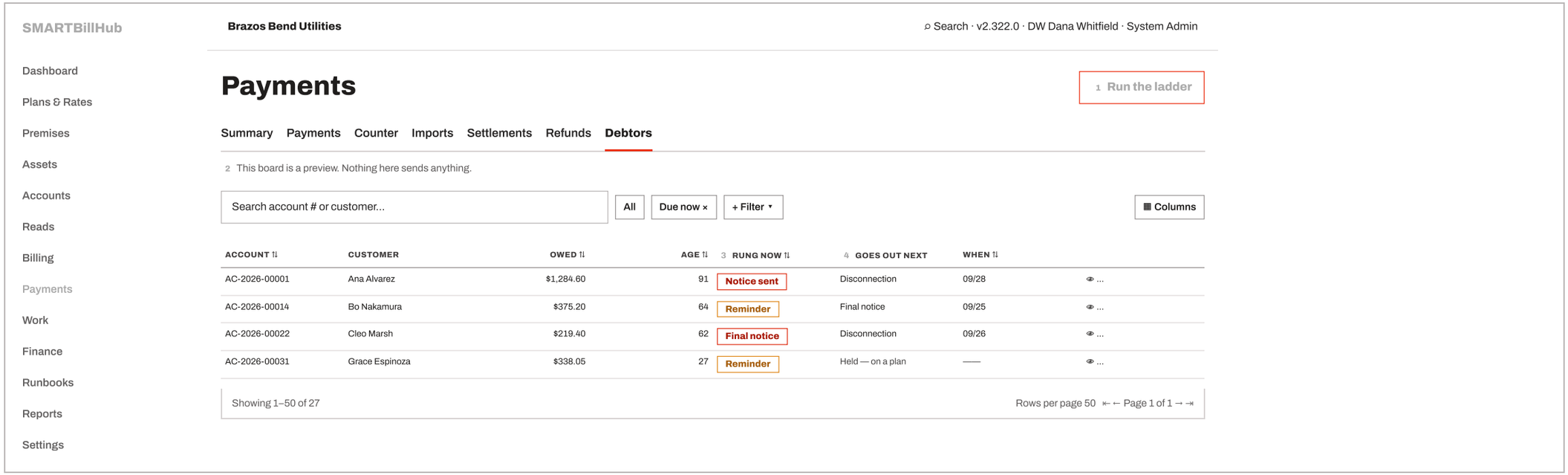
3 What you can do to it Chase it, spread it, or give it up

7·3a Ladder UPDATE S34 · PAY-B

`/:tenant/payments/collections → CollectionsPage (a separate route, not the :tab handler)`

The question: "Who is at which step of the chase, and what goes out next?"

DESKTOP · 1440 · **DRAWN TO THE RULES** · WALKED IN THE RUNNING APP



① **Run the ladder** · the one commit — and it belongs here, not on 7·1 · B14 **4** · MOVED ② **A preview, said plainly** · the board shows what WOULD go out · B14 **33** · NEW ③ **Rung now** · as a pill, not a sentence · B14 **33** · NEW ④ **Goes out next** · and when — the question the board exists to answer · NEW

For	Seeing the chase as a board, and advancing it deliberately.
Doors	**Exactly one** — the Debtors tab. The route constant is imported by nothing but the router: **the door is built from the tab id**, so a constant-only search would have reported this screen unreachable. It is not.
Routing	The static route outranks `/payments/:tab`, so this tab leaves `PaymentsPage` entirely — and renders the same strip with the tab active, so the way back exists. **A dead branch remains**: a comment says the tab resolver answers “one of exactly three values” and it now answers more.
Table controls	**All four absent today.** One filter chip and a date field, on a plain table.
TODAY **Today**	**The second NEB-2 breach, and the one CLAUDE.md does not name** — unpaginated, with one query per debtor behind it.

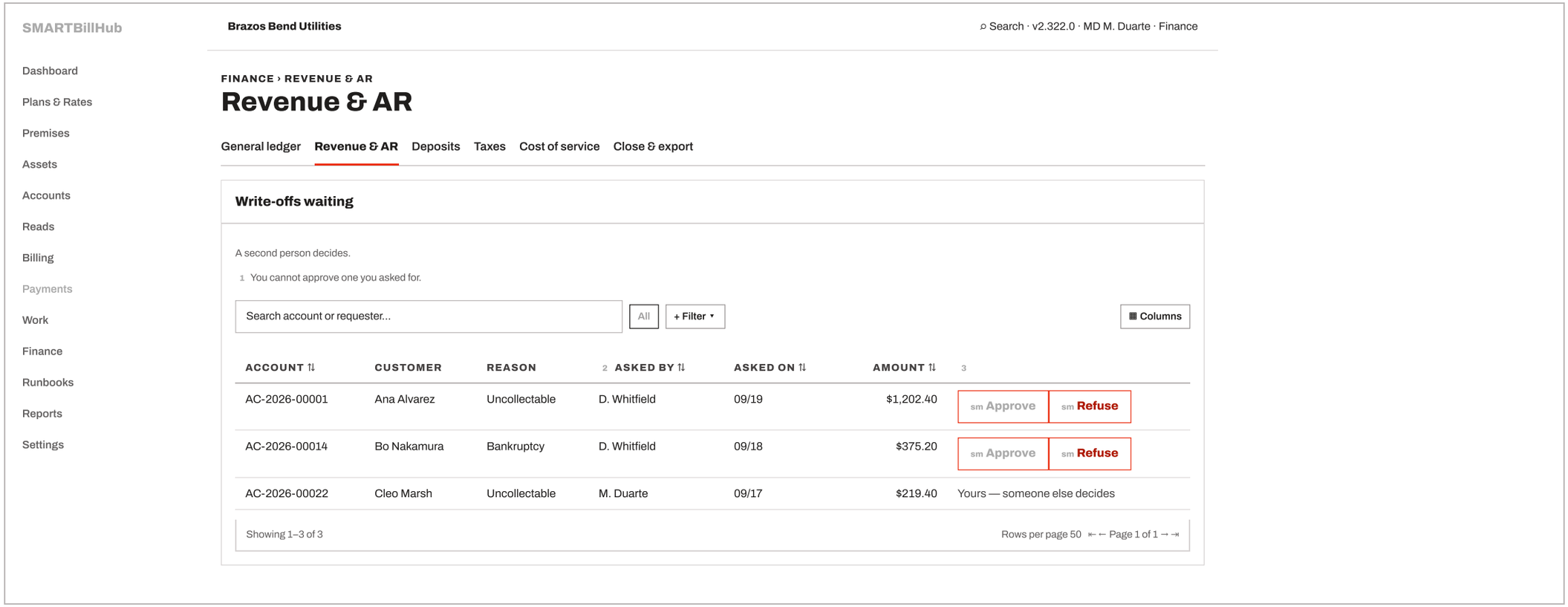
TODAY Today	**The second NFR-3 breach, and the one CLAUDE.md does not name** — unpaginated, with one query per debtor behind it.
TASK **To build**	The toolbar · ⇅ headers · columns · paging · one query, not one per debtor.
Walked	**Yes** — and walking it is what confirmed there is no commit control on the board today.
Rules	B14 **12** ✗ · **13** ✗ · **14** ✗ · **16** ✗ · **32** · **33** · NFR-3 ✗

7·3b Write-offs UPDATE S36 · PAY-K

`Finance › Revenue & AR — the panel, which is the ruled home`

The question: "Someone asked to write off $1,202.40. Do I agree?"

DESKTOP · 1440 · **THE PANEL THAT ALREADY EXISTS**, GAINING A TOOLBAR AND PAGING



① **The two-person control** · the requester asks from 7·1; a different person decides here · EXISTS ② **Asked by** · which is what makes the control visible · EXISTS ③

Decision · named buttons, not icons — an approval queue is not a row-actions column · B14 **15** · EXISTS

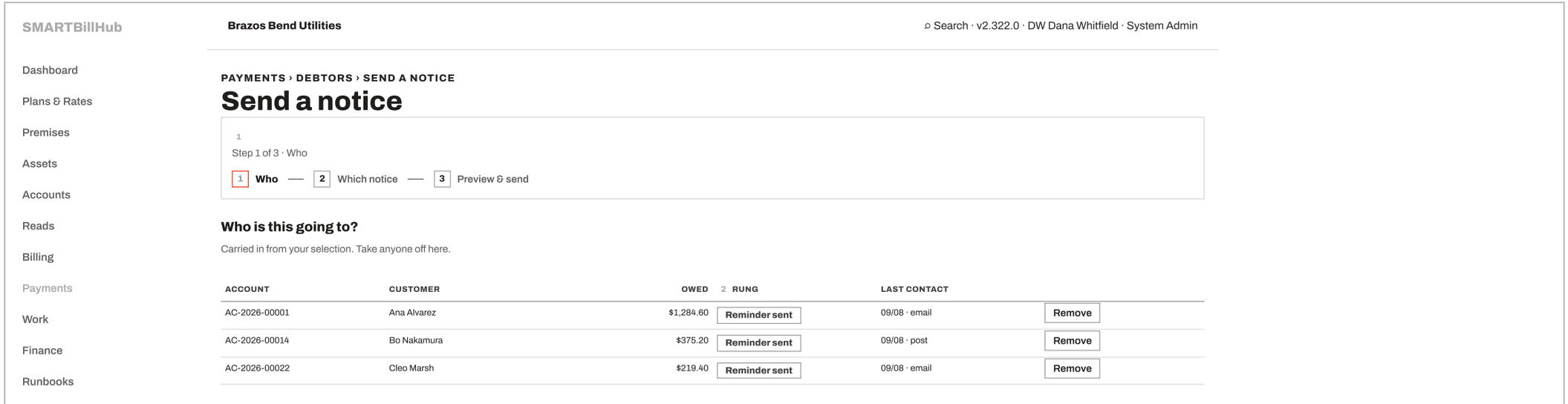
For	The Owner or Finance role giving the second signature.
Doors	Finance → Revenue & AR. **1 door, and it works.**
Primary action	**Approve**, with **Refuse** beside it in the danger tier.
Write surfaces	**Update** → approve · reject · reverse. **All three served.**
TODAY **Today**	**The panel is built and reachable.** What it lacks is a toolbar and paging — rule 16, the weakest of the eighteen, because an approval queue is short by nature.
TASK **To build**	The toolbar and the pagination footer. **And the retirement in 7·3c.**
Rules	B14 **6** · **12** ✗ · **16** ✗ · **31** · **34** · BR-PAY-020

7·3d Notice NEW S35 · PAY-B

`proposed POST /:tenant/collections/notices — the route is not mounted today`

The question: "Three of these have ignored a reminder. Send them the notice."

DESKTOP · 1440 · **A THREE-STEP WIZARD ON THE SHARED FRAME** — WHO, WHICH LETTER, AND THE LETTER ITSELF



Reports

Settings

Cancel Cancel

3 accounts · $1,879.20 Continue →

SMARTBillHub

Brazos Bend Utilities

⌕ Search · v2.322.0 · DW Dana Whitfield · System Admin

Dashboard

Plans & Rates

Premises

Assets

Accounts

Reads

Billing

Payments

Work

Finance

Runbooks

Reports

Settings

PAYMENTS › DEBTORS › SEND A NOTICE

Send a notice

Step 2 of 3 · Which notice

1 Who — 2 **Which notice** — 3 Preview & send

Which letter goes out?

3 **A tone is a letter, not a manner. Firm** sends the regulated 10-day disconnect notice — with its cut date and its statutory footer.

4 **Which notice** *

Firm ⌕ ▾

Four tones. Each one selects a different letter.

Sends: 10-day disconnect notice · cut date 10/02 · statutory footer · post and email

5 **Why**

Kept on every row, sent or skipped.

← Back Cancel

Continue →

SMARTBillHub

Brazos Bend Utilities

⌕ Search · v2.322.0 · DW Dana Whitfield · System Admin

Dashboard

Plans & Rates

Premises

Assets

Accounts

Reads

Billing

Payments

Work

Finance

Runbooks

Reports

PAYMENTS › DEBTORS › SEND A NOTICE

Send a notice

Step 3 of 3 · Preview & send

1 Who — 2 Which notice — 3 **Preview & send**

This is the letter.

Read it before it costs postage.

BRAZOS BEND UTILITIES

NOTICE OF DISCONNECTION

Ana Alvarez · AC-2026-00001
1420 Lamar Street, Apt 3B

Amount now due **$1,284.60**.
Service will be disconnected on or after **October 2, 2026** unless paid.

ACCOUNT	6 GOES OUT	HOW
Ana Alvarez	10-day notice	Post · email
Bo Nakamura	10-day notice	Post
Cleo Marsh	Skipped — medical hold	—



① **WizardFrame** · the shared rail — numbered, a done step clicks BACK, a forward click is a no-op · B14 **36** · **EXISTS** · components/WizardFrame · BR-SHL-018 ② **Rung and last contact** · so a clerk can see who has already been chased, before sending again · **NEW** · LADDER_RUNGS ③ **What a tone actually sends** · the single most dangerous sentence on this screen · B14 **33** · **NEW** · BR-NOT-008 ④ **Which notice** · four tones, and each selects a different LETTER · B14 **8** · **NEW** · GET /notice-tones — served ⑤ **Why** · kept on every row, sent or skipped · B14 **10** · **NEW** ⑥ **Per row, sent or skipped** · **with the reason** — a silent exclusion is the one thing a collections trail may not contain · B14 **11** · **NEW** ⑦ **What it will do** · two letters, one hold, no recall · B14 **11** · **NEW**

For	A clerk chasing accounts that have ignored the automatic ladder.
Doors	**Send notice** on 7·1's selection toolbar. **1 door.**
Steps	Three, on `WizardFrame`: **Who** (carried selection, removable) · **Which notice** (the tone, and what it sends) · **Preview & send** (the rendered letter). The step is URL state — `?step=` — and clamps to the highest validated step.
Primary action	**Send 2**, danger tier — it reaches a customer, costs postage, and cannot be recalled.
Fields	Which notice (required) · Why (kept on every row, sent or skipped).
Write surfaces	**Add** → `POST /collections/notices`, **one transactional commit at the last step** (BR-SHL-019). **NOT SERVED** — only `collections/run` and `collections/ladder` are mounted.
Today	**Two dead buttons.** *Send a reminder* and *Send a disconnection notice* both render disabled with *"Sending reminders and disconnection notices is a separate piece of work and no route accepts one yet."*
To build	The endpoint, and this wizard. **The vocabulary, the tone mapping, the skip reasons and the frame all already exist** — `LADDER_RUNGS`, `GET /notice-tones`, the five skip codes, and `WizardFrame`.
Rules	B14 **4** · **6** · **8** · **10** · **11** · **31** · **33** · **36** · **38** · `BR-PAY-017` · `BR-PAY-035` · `BR-NOT-008` · `BR-SHL-018`/**019**

Why this is a wizard and not a modal — it was drawn as one until 2026-09-22

Because the letter has to be seen before it is sent, and a modal had nowhere to put it. `BR-NOT-008`, verbatim: *"regulated ladder steps ALWAYS render the disconnect-notice template class regardless of the tone picked in the reminder modal."* So **choosing *Firm* does not send a firmer reminder — it sends the regulated 10-day disconnect**

notice, with its cut date and its statutory footer. Choosing *Final* sends the 48-hour certified letter, at real postage.

`GET /notice-tones` was built for exactly this: its own docstring says that until it landed, ***"no surface anywhere showed a clerk which letter a tone selects."*** A line of helper text under a picker is the minimum; **a rendered preview is the thing itself**, and it needs a step of its own.

The day numbers come from **the tenant's own ladder configuration**, never constants (`BR-REP-016`) — and when the settings cannot be read the route reports the days unavailable **and still renders the tones**, because which class a tone selects is a rule and does not depend on any day number. **The wizard follows it:** the tone list never goes empty, the day line does.

⑥ Every skip is shown, and that is a rule rather than a courtesy

`ladder.py` writes a `notice_history` row for every send **and every skip** — five reasons: below the ladder floor · medical hold · payment plan paused · partial payment · nothing overdue. Its own comment says why: ***"a silent exclusion is the one thing a collections trail may not contain."***

So the last step lists the skipped row **with its reason**, and the button says **Send 2**, not Send 3. **A clerk who believes three letters went out when two did will chase the third customer for a notice they never received.**

Why this is a manual screen when the ladder already sends notices

The ladder (7·3a) runs the sequence automatically, by rung. **This is the manual override** — and the two share **one vocabulary on purpose**: `LADDER_RUNGS`'s comment says it is *"ONE vocabulary across the automatic engine and the manual modals, which is what makes no-double-send possible at all."* **A second vocabulary here would break that**, which is why the picker is served rather than invented.

The frame is not ours to invent either. `WizardFrame` carries forty flows — move-in, new account, the bill run, a work order — and owns the rail, the forward fence, the working state and the single transactional commit. **Payments mounts it nowhere today**, which is how this screen came to be drawn as a modal in the first place.

7·3c **Retire** REMOVE S-NEW · PAY-K

`/:tenant/finance/write-offs — mounted, rendered, tested, and superseded`

The question: "There are two write-off queues. Which one is the product?"

GAP **I had this wrong twice, and the correction matters**

I reported this as **the single worst finding in the audit** — an approval queue nobody can reach. **Half of that was wrong.** The *route* is orphaned; the *capability* is not.

What actually happened: `CR-060 §8b` folded write-offs from a destination into a panel inside Revenue & AR, *"where the receivable it gives up already lives."* **The panel landed** — `RevenueArPage.tsx:713`, rendered at `:1230`, and Revenue & AR is one of the six Finance tabs. It imports `approveWriteOff` and holds the approvals. **It is the real queue and it works.**

What did not happen is the deletion. `WriteOffQueuePage` is still mounted, still rendered, still tested — and outside its own declaration it is referenced by exactly two non-test lines: the import and the mount. **So this is not a missing door. It is two implementations of one queue**, and the orphan is the one that was meant to go.

The fix is a deletion, not a link. Adding a door to the standalone page would make the duplication permanent — two screens, two visual decisions, drifting from the first defect onwards. **Delete the route and the component**, or redirect it to the panel.

Why no check caught it: every test asks whether a screen renders *at its own address*, which is the one place a door is not needed — and a test doing exactly that (`renderedCopy.test.tsx`) is what kept the orphan looking alive. **Nothing asks whether two components answer the same question.**

4 What it changes

Chasing, spreading, and giving up

Act	What it writes	The ledger	What changes elsewhere
Run the ladder	A notice row per account, at its rung	Nothing	Letters, emails and texts queue · the account's ladder state moves on
Offer a plan	A payment plan and its instalments	Nothing until money arrives	**The ladder stops** · missed instalments have their own events
Request a write-off	A write-off, requested	Nothing yet	It lands in the approval queue — **which nobody can reach**
Approve it	Who approved, when	A balanced pair to bad debt	The debt leaves the debtor list · `write_off_approved` is a live event
Reverse it	A new row — never an un-picking	A reversing pair	The debt returns to the list
Take a payment	Tab 2's path, unchanged	A balanced pair	**The ladder stops** · the account may leave this list entirely

GAP **The breach CLAUDE.md names by number, and the one it does not**

The debtor list fetches every debtor and sorts them in Python, excused in the code by a ceiling that has since moved from 3,000 connections to 40,000. That one is known. **The collections ladder is unpaginated too, and runs one query per debtor** — that one is not named anywhere, and this audit found it.

04

Journey issues — what happened when the product was driven

Everything before this section is a read. **This is the only section whose findings could not be reconstructed from the source**, and it is where three of this document's own errors were caught.

TODAY **The app was driven. Here is exactly how.**

The full stack was brought up with the repository's own `08-qa/01-e2e/stack/boot_stack.sh` — PostgreSQL 16, 389 migrations, the demo fixture, Django, and the **built** SPA served same-origin at `http://127.0.0.1:8080`. Signed in through the real sign-in screen as the fixture's published owner (**Dana Whitfield, System Admin**, tenant `brazosbend`), and driven with Playwright against the pre-provisioned Chromium.

Tenant data as walked: 30 accounts owing, $8,870.58 outstanding, **zero payments recorded**.

GAP **What was NOT walked, and why — read this before the table**

The payments fixture did not seed. It refused *by name* rather than crashing: *"no multi-property cheque was recorded: one of the landlord's three properties owes nothing, so a cheque covering all three would be a cheque for less than it claims and the tender's own sum check would refuse it."* That is the fixture's honest-refusal mechanism working as designed — but it means the tenant has **no payment rows**.

So of the journeys this surface needs, **6 were walked and 3 were not**:

- **Not walked — recording a payment end to end**, which is the journey that would have shown **PAY-A**'s raw key on screen. **The defect is nonetheless proved**, by reading every link of the chain (§02, screen 02) — but it was proved by reading, not by seeing, and this document does not blur those.
- **Not walked — reversing a payment** and **correcting a reference**: both need a payment row.

A register of issues from six journeys is a different document depending on whether the other three passed or were never opened. **They were never opened.**

Journey · step	Screen	Did	Expected	Got	Severity
J-0 sign in 1 of 1	S3 sign-in	Filled the fixture credentials and submitted	Lands on the dashboard inside the shell	**Passed.** `/brazosbend/dashboard`, shell framed, 13-item rail rendered	ok
J-A reach payments 1 of 1	01	Opened `/brazosbend/payments`	The Payments head and four tabs	**Passed.** `h1 = "Payments"`, four tabs, no 4xx	ok

J-A measure title size	01	`getComputedStyle` on the page-head `h1`	32px per B14 rule 40	**32px.** Rule 40 verified on rendered output — **and it settles the divergence flagged in §01 in B14's favour**	ok
J-A measure the rail	shell	Counted rail links	13 items (R8)	**13.** Dashboard · Plans & Rates · Premises · Assets · Accounts · Reads · Billing · Payments · Work · Finance · Runbooks · Reports · Settings	ok
J-B controls Record tab	02	Looked for a table search and a pagination footer	Both present and server-side	**Both present.** A real toolbar search — placeholder *"Search account #, customer, reference or address..."*, inside `table-toolbar < data-table-frame` — plus a pagination footer and a sorted `POSTED` ▼ header	ok
J-B controls Outstandings	07	Same probe, and inspected each input's DOM ancestry	—	**No table search, no pagination.** The one search box on the page is the shell's global `shell-search < topbar`, not a table control. **30 rows rendered, all of them.** Confirms the §05 violations against rules 12/13/14/16 **on rendered output**	Journey completes, wrongly — at 30 rows. Not at 40,000
J-B controls Collections	08	Same probe	—	**Same: global search only, no pagination, 30 rows rendered.** Confirms the second NFR-3 breach	Journey completes, wrongly
J-C the door 1 of 2	Revenue & AR	Opened the screen that holds the write-off panel and listed every anchor whose `href` contains `write-off`	At least one link to the queue	**0 links.** `[]`	**Blocks the journey**
J-C the door 2 of 2	six screens	Loaded Dashboard, Payments, Outstandings, Finance, Revenue & AR and Deposits — signed in — and counted anchors to `/finance/write-offs` on each	A door somewhere in the product	**Zero on every one.** `{"/dashboard":0, "/payments":0, "/payments/outstandings":0, "/finance":0, "/finance/revenue":0, "/finance/deposits":0}`	**Blocks the journey**
J-D typed URL 1 of 1	09	Typed `/brazosbend/finance/write-offs` directly	The queue renders, proving only the door is missing	**It renders.** `h1 = "Finance"`, the write-off heading present, no 4xx. **The screen works perfectly. Nothing can reach it.**	ok — and that is the point
J-E the notice circle	07 · 08 + account	Followed the product's own instruction for sending a notice, screen to screen	To arrive somewhere that can send one	**A circle ending at a dead button.** See the finding below	**Blocks the journey**

GAP **★ J-E · a finding only the walk could produce — the product sends a clerk in a circle**

Three screens each say where notices are sent from. **No two of them agree, and the end of the trail is a button that does nothing.** All three strings verified in `en.json`:

```
Account screen    "Notices go out from Collections."
                  customers.account.communication.refused.notice_is_the_ladder
                        |
                        ▼
Collections       "This screen sends nothing. Notices go out from the outstandings list."
                  payments.collections.previewOnly
                        |
                        ▼
Outstandings      "Send a reminder"  ·  "Send a disconnection notice"   ← BOTH DISABLED
                  "Sending reminders and disconnection notices is a separate piece
                   of work and no route accepts one yet."
                        |
                        ▼
                  DEAD END — and the reason is a title tooltip, so a keyboard
                  or screen-reader user never learns why.
```

Each sentence is individually defensible. The account screen is right that the ladder owns notices; Collections is right that it is a preview board; Outstandings is right that no route accepts a send. **Nobody owns the path through them**, which is exactly the class of failure the cross-module pass exists for — and it is invisible to every check in the repository, because each screen's own tests pass on its own copy.

Not covered by any commitment above. It is a routing-of-intent question for the PO — where *should* a notice be sent from, once a send route exists? — not something to fix blind. It is filed here as a description, deliberately, and per this document's own rule **nobody has undertaken to fix it.**

GAP **Three corrections the walk forced on this document**

Step 5 exists because no gate in this repository reads rendered output. It earned its place three times here — **every one of these is a place where my source read produced a drawing the running app contradicted**, and all three are corrected in §02 rather than left standing.

1. **Outstandings has 15 columns and an ageing band strip; I had drawn 9 columns and no strip.** The real columns include *Rank*, *Open bills*, *Notices sent*, *Collection step*, *Next step* and *Payment plan*. Drawing it short would have made the remedy's carry-over table reconcile against the wrong baseline — the precise failure the carry-over arithmetic exists to catch.
2. **Days sales outstanding is not a gap.** The screen renders *"Reported on the Finance page"*, and Finance owns it — where it is separately unavailable with its own stated reason (`dso_needs_twelve_months_of_billed`). **My first draft of PAY-J proposed adding a DSO figure to this screen, which would have created a second home for one fact** —

the exact anti-pattern §07 is written to catch. PAY-J now covers the selected total only.

3. **The KPI tiles foot to the rows exactly** — $389.64 + $6,712.26 + $108.88 + $1,659.80 + $0.00 = **$8,870.58**, and the band shares sum to 100.0%. That is the *real* reason the endpoint is unpaginated, and it means **PAY-B cannot simply add a pager**: the tiles must be served independently of the page, or they start describing a page instead of the debt. PAY-B now says so.

05

Cross-module — Settings, and what these 35 screens do to other modules

Everything here is work that is **not a payments screen**. It sits in its own section because a payments screen cannot own it, and because the cheapest way to lose a dependency is to bury it in a drawing of something else.

TODAY **Settings needs almost nothing, and that was a finding**

An earlier pass of this document listed **seven Settings items, three of them new screens**. Read against the code, **five already existed** and one turned out not to be a Settings change at all. **The whole Settings change for all 35 screens is one row in Number series.**

Settings

What	Where exactly		Why, and what was wrong with the first answer
Service centres	Configure → Reference data → Service centres	✅ Exists	Already a list, with name, address, phone, opening hours, what a centre can do and **which payment methods it takes**. It is **portal-published content** — its own docstring says *"where a customer can walk in"* — so operational fields do not belong on it
Counter setup float · auto-close	**Payments** → **Counter**, not Settings	🟢 New	**A counter is not a location — it is a till at one.** One centre can have three windows, each with its own drawer. So the float belongs to the counter, which is payments' own object, and the centre stays a portal record. **This came out of being challenged on the first answer**
Batch numbering	Configure → Number series	🟡 1 row	**Thirty series already exist**, `deposit_batch`, `import_run`, `payment`, `receipt` and `write_off` among them. Under the one-batch-per-session ruling **the counter batch IS the deposit batch**, so it reuses that series; imports reuse `import_run`. **Only settlements need a new one**
Timezone	Company → Regional	🔴 Not	**The setting already exists** and the date component honours it — *"an instant always displays in tenant time"*. The bug

		Settings	is in payments: `isoToday()` returns `new Date().toISOString()`, the **UTC** day. A shift ending after 17:00 local stamps its payments into tomorrow, and the counter makes that urgent
Barcode format	—	🔴 Dropped	Ruled 2026-09-21: **not a tenant choice.** One symbology, shipped. A utility does not pick an encoding
Tender types	Accounting	✅ Exists	Each with its own cash and clearing account
Allocation order	Accounting → rule pages	✅ Exists	A tenant setting, not a per-payment staff choice — which is why the record form deliberately does not offer it

Every other module these screens touch

Module	What		Why, and which screen needs it
Billing	**Remittance barcode** on the standard bill layouts	🟢 New	**The Scan paper source on 4·3a reads nothing without it.** Our bill carries no barcode and the code says so on every result — *"no logo, no remittance barcode, no colour"* — and the mail file prints *"No Intelligent Mail barcode has been applied."* Custom bill design was ruled out for V1, so it goes **into the three to five standard layouts**. Carries account, invoice, amount, check digit
	Credit-only correction	✅ Exists	**6·3c is a door to this, not a copy of it.** Building a second credit path in payments is how a figure starts to drift
Ledger Finance	**Deposit batching rule**	🟡 Update	Today batches group **day × method** and the database enforces it with a unique key. Under the ruling of 2026-09-21 a batch is **one person's session**, so that constraint is replaced. **Reconciliation survives it** — the matching already handles one bank credit paying for several deposits
	Undeposited funds	✅ Exists	A counter receipt already posts a balanced pair the minute money is taken. **The counter needs no new ledger work**
Notifications	**Counter receipt** — paper at the window	🟢 New	The queue is asynchronous. **A counter needs paper in hand now**, which is a different act from queueing a document
	Re-send on demand	🟢 New	2·3h. The templates, the channels and the queue all exist — this is one endpoint that re-queues an existing document
	The four payment events	✅ Exist	`payment_received`, `payment_failed`, `refund_issued` and `credit_issued` are live, on email and text, with a **printed floor** so a utility that switches every electronic channel off still gets paper. **An earlier pass of this document said receipts were not built. That was wrong** — it read the bulk bar's refusal and generalised it
Integrations	**Settlement feed**	🟢 New	Tab 5. Taking a card payment and being paid for it are two different messages, and only the first is handled

	Card terminal at the counter	🟢 New	Tap-and-pay at a window is a **device**, not a webhook. Nothing addresses it
	Payment providers	✅ Exists	Ruled 2026-09-05: one, two or three per utility. **No provider is named in payments anywhere** and a gate refuses any code that names one — adding Worldpay is a manifest and an adapter
Shell	**Doors to Counter and Imports**	🟡 Update	The sidebar renders no submenus, so a tab is reachable only from the page's own strip. Two new tabs need to be in it
	The write-off door	🟡 Update	7·3c. One link. The screen is built, routed and tested, and **nothing anywhere points at it**
Metering the ONE runner	**An eighth `ImportTarget` — `payments`**	🟡 Update	Tab 4. One dict entry: column set, aliases, default mapping template, dedupe key, rule set. `BR-MET-096` (PO 2026-08-21) says ***"this forbids building two importers"***, and the runner has absorbed seven targets on exactly this shape
	The LOAD writer seam	🟡 Update	**Metering must not write payments' tables.** The Load resolves payments' own `import_writer` through the seam registry, exactly as `parts` resolves work's — so the boundary is structural, not a thing to be careful about
Payments	`payment_import.py` — the second pipeline	🔴 Decision	**Retire it, or amend the rule.** Six endpoints and two staging tables that do four of the runner's five stages — with **no Transform**, so no column mapping. Assets' staged path was retired for exactly this and its two write verbs now refuse. **A PO call, not a lane's**
Customers	Account credit position	✅ Exists	Already shows where a credit came from rather than one unexplained number
Scanner	—	⚪ None	**A barcode scanner is a keyboard.** It types what it reads and presses Enter — no driver, no integration, no pairing. The cost is entirely in billing's barcode

> GAP **The dependency that gates the most**
>
> **The ledger's batching rule.** Six of tab 3's screens and the batch column on 2·1 all assume a batch is a session. Until the day × method constraint is replaced, a counter close has nowhere valid to write. **It is the first thing to do, and it is not in payments.**

05

Register — every screen, classified

One row per screen, linked to its drawing. **35 screens: 17 new, 16 updated, 2**

removed, 0 kept. The **Repo id** column is A1's ticket vocabulary and is never invented here — `S-NEW` means a screen that needs a number allocated, which is a governance act and not a drawing decision.

> **2 of these need no backend at all**
>
> Move, Reapply and all three Imports screens are **finished, tested engines with no caller** — the endpoints are served, and in two cases the API client function is written and called by nothing. They need a screen and a door. **Cheapest work on this list, and among the most valuable**: the Imports three are the 4,000–6,000 receipts a month.

Tab	Screen	Address	Class	Repo id	Commits	What it needs
1	**Summary**	1·1	UPDATE	S32	PAY-C · PAY-D	**Redrawn as a worklist** — the counter, then what is waiting, then today
2	**List**	2·1	UPDATE	S33	PAY-M	Source column, Batch column, two filters, Start counter
2	**Bulk**	2·1b	UPDATE	S33	PAY-E	Wire Reapply and Send receipts, or take them off the bar
2	**Detail**	2·2	NEW	S-NEW	PAY-H	An address, GET /payments/{id}, header strip, history
2	**Row panel**	2·2r	REMOVE	S33	PAY-H	Goes when 2·2 lands. Not before
2	**Record**	2·3a	UPDATE	S35	PAY-M	Counter banner, segmented tender, consequence box
2	**Edit**	2·3b	UPDATE	S35	—	Consequence box only
2	**Correction**	2·3c	NEW	S35	PAY-P	Modal, and a column linking it to what it corrects
2	**Cancel**	2·3d	NEW	S35	PAY-P	**A sixth payment state, an endpoint, and a rule**
2	**Reverse**	2·3e	UPDATE	S35	—	Danger tier on the commit, and a consequence box
2	**Move**	2·3f	NEW	S35	PAY-P	**Screen only** — engine and client both written
2	**Reapply**	2·3g	NEW	S35	PAY-E	**Screen only** — and delete the bulk refusal
2	**Resend**	2·3h	NEW	S35	PAY-E	One endpoint that re-queues an existing document
3	**Counters**	3·1	NEW	S-NEW	PAY-N	The session table, and GET /counters
3	**My counter**	3·2	NEW	S-NEW	PAY-N	Running totals by method

3	**Start**	3·3a	NEW	S-NEW	PAY-N	**Opens the session AND its batch**
3	**Close**	3·3b	NEW	S-NEW	PAY-N	Counted against system, per method
3	**Force close**	3·3c	NEW	S-NEW	PAY-N	Supervisor only, reason required
3	**Declare**	3·3d	NEW	S-NEW	PAY-N	A count after the fact
4	**Batches**	4·1	UPDATE	S35	PAY-F	**Reuse** — the existing imports list, target=payments
4	**Run**	4·2	UPDATE	S35	PAY-F	**Reuse** — StageBar, RuleResultsTable, RejectList, DryRunPanel, Receipt
4	**Import**	4·3a	UPDATE	S35	PAY-F	**One dict entry** + 7 catalogue keys + a Scan source card
5	**Settlements**	5·1	NEW	S-NEW	PAY-P	The table, the feed, one number series
5	**Settlement**	5·2	NEW	S-NEW	PAY-P	The read, and the fee posting
6	**Refunds**	6·1	UPDATE	S58	PAY-P	**Move the route**, add toolbar and paging
6	**Refund**	6·2	NEW	S-NEW	PAY-P	GET /refunds/{id} and the screen
6	**Raise**	6·3a	UPDATE	S58	PAY-P	Consequence box
6	**Issue**	6·3b	UPDATE	S58	PAY-P	Danger tier on the commit, consequence box
6	**Credit**	6·3c	UPDATE	S22	PAY-P	**A door only** — billing already has the engine
7	**Debtors**	7·1	UPDATE	S34	PAY-J	Toolbar, sorting, columns, paging — and **NFR-3**
7	**Debtor**	7·2	NEW	S-NEW	PAY-J	One read gathering bills, ladder and history
7	**Ladder**	7·3a	UPDATE	S34	PAY-J	Toolbar and paging, and the one-query-per-debtor read
7	**Write-offs**	7·3b	UPDATE	S36	PAY-K	Toolbar and paging on the panel
7	**Notice**	7·3d	NEW	S35	PAY-B	**The endpoint**, and a 3-step wizard on WizardFrame
7	**Retire**	7·3c	REMOVE	S-NEW	PAY-K	**Delete the orphan.** Two write-off queues exist; the panel in Revenue & AR is the real one

A description is not a commitment, and the two columns say which is which

Repo id says a screen exists in A1's agreed vocabulary. **Commits** names the specification someone has undertaken to build. A row with a `PAY-` id is **promised**; a row with a dash is **described**. **If a thing is not in the Commits column, nobody has undertaken to build it**, however carefully it is drawn above.

Note what **S35** already covers: *Record, Credit, Adjust, Lockbox, Reverse, Reapply, Receipts*. Most of what this document calls new was **specified and never built**, which is a different fact from nobody having thought of it.

Completeness, scoped

This document is **exhaustive over itself** — every rule it states, every figure it prints, every screen it draws, every decision it records. It is **not exhaustive over the product**: the deposits and bank reconciliation family (four routes), the refunds screen, the customer portal's own payment flow, and every account-side copy of the Record modal have behaviour these screens never touch.

It is as stale as the read it was made from: **2.322.0, commit `e75771af`, 2026-09-21**, on branch `claude/upbeat-cannon-xln69g` — which could not be confirmed as the current product.

What was deliberately left out

The argument behind each screen (it lives in that screen's own spec table), the source catalogues themselves, line numbers in prose where a path would do — **they move, and a citation that rots is worse than none** — the planning of this pass, and the sample figures, which are illustrations rather than fixtures.

It is fine for those to be out; it is not fine for them to be out silently.